REM Offline POSIX and Linux System Interface Reference

REM Project

Contents

REM Offline POSIX and Linux System Interface Reference **1**

REM Offline POSIX and Linux System Interface Reference

Prepared from the locally installed Linux man-pages for offline use. This is a practical reference for the interfaces discussed in *Inside the REM Computer*; it is not a replacement for the REM ABI documentation. Linux syscall numbers, register conventions, and availability on REM remain target-specific.

How to use this reference

Use the POSIX/standards sections to identify the portable contract, then read the Linux-specific sections for flags, errors, extensions, and implementation behavior. Where a page describes glibc or mainstream Linux behavior, verify the corresponding REM headers and regression tests before relying on it.

Contents

- read(2)
- write(2)
- open(2) and openat(2)
- ioctl(2)
- TTY ioctl requests
- termios(3)
- poll(2)
- mmap(2)
- brk(2)
- clone(2)
- execve(2)
- wait4(2)
- signal(7)
- signal-safety(7)
- ELF(5)
- errno(3)
- fcntl(2)
- fsync(2)
- rename(2)
- nanosleep(2)

---

read(2)

```
read(2)                        System Calls Manual                        read(2)
```

```
NAME
       read - read from a file descriptor

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       ssize_t read(int fd, void buf[.count], size_t count);

DESCRIPTION
       read() attempts to read up to count bytes from file descriptor fd into
       the buffer starting at buf.

       On files that support seeking, the read operation commences at the file
       offset, and the file offset is incremented by the number of bytes read.
       If the file offset is at or past the end of file, no bytes are read,
       and read() returns zero.

       If count is zero, read() may detect the errors described below.  In the
       absence of any errors, or if read() does not check for errors, a read()
       with a count of 0 returns zero and has no other effects.

       According to POSIX.1, if count is greater than SSIZE_MAX, the result is
       implementation-defined; see NOTES for the upper limit on Linux.

RETURN VALUE
       On success, the number of bytes read is returned (zero indicates end of
       file), and the file position is advanced by this number.  It is not an
       error if this number is smaller than the number of bytes requested;
       this may happen for example because fewer bytes are actually available
       right now (maybe because we were close to end-of-file, or because we
       are reading from a pipe, or from a terminal), or because read() was
       interrupted by a signal.  See also NOTES.

       On error, -1 is returned, and errno is set to indicate the error.  In
       this case, it is left unspecified whether the file position (if any)
       changes.

ERRORS
       EAGAIN The file descriptor fd refers to a file other than a socket and
              has been marked nonblocking (O_NONBLOCK), and the read would
              block.  See open(2) for further details on the O_NONBLOCK flag.

       EAGAIN or EWOULDBLOCK
              The file descriptor fd refers to a socket and has been marked
              nonblocking (O_NONBLOCK), and the read would block.
              POSIX.1-2001 allows either error to be returned for this case,
              and does not require these constants to have the same value, so
              a portable application should check for both possibilities.

       EBADF  fd is not a valid file descriptor or is not open for reading.

       EFAULT buf is outside your accessible address space.
```

```
       EINTR  The call was interrupted by a signal before any data was read;
              see signal(7).

       EINVAL fd is attached to an object which is unsuitable for reading; or
              the file was opened with the O_DIRECT flag, and either the
              address specified in buf, the value specified in count, or the
              file offset is not suitably aligned.

       EINVAL fd was created via a call to timerfd_create(2) and the wrong
              size buffer was given to read(); see timerfd_create(2) for
              further information.

       EIO    I/O error.  This will happen for example when the process is in
              a background process group, tries to read from its controlling
              terminal, and either it is ignoring or blocking SIGTTIN or its
              process group is orphaned.  It may also occur when there is a
              low-level I/O error while reading from a disk or tape.  A
              further possible cause of EIO on networked filesystems is when
              an advisory lock had been taken out on the file descriptor and
              this lock has been lost.  See the Lost locks section of fcntl(2)
              for further details.

       EISDIR fd refers to a directory.

       Other errors may occur, depending on the object connected to fd.

STANDARDS
       POSIX.1-2008.

HISTORY
       SVr4, 4.3BSD, POSIX.1-2001.

NOTES
       On Linux, read() (and similar system calls) will transfer at most
       0x7ffff000 (2,147,479,552) bytes, returning the number of bytes
       actually transferred.  (This is true on both 32-bit and 64-bit
       systems.)

       On NFS filesystems, reading small amounts of data will update the
       timestamp only the first time, subsequent calls may not do so.  This is
       caused by client side attribute caching, because most if not all NFS
       clients leave st_atime (last file access time) updates to the server,
       and client side reads satisfied from the client's cache will not cause
       st_atime updates on the server as there are no server-side reads.  UNIX
       semantics can be obtained by disabling client-side attribute caching,
       but in most situations this will substantially increase server load and
       decrease performance.

BUGS
       According to POSIX.1-2008/SUSv4 Section XSI 2.9.7 ("Thread Interactions
       with Regular File Operations"):

           All of the following functions shall be atomic with respect to each
           other in the effects specified in POSIX.1-2008 when they operate on
```

```
           regular files or symbolic links: ...

       Among the APIs subsequently listed are read() and readv(2).  And among
       the effects that should be atomic across threads (and processes) are
       updates of the file offset.  However, before Linux 3.14, this was not
       the case: if two processes that share an open file description (see
       open(2)) perform a read() (or readv(2)) at the same time, then the I/O
       operations were not atomic with respect to updating the file offset,
       with the result that the reads in the two processes might (incorrectly)
       overlap in the blocks of data that they obtained.  This problem was
       fixed in Linux 3.14.

SEE ALSO
       close(2), fcntl(2), ioctl(2), lseek(2), open(2), pread(2), readdir(2),
       readlink(2), readv(2), select(2), write(2), fread(3)

Linux man-pages 6.7              2024-03-12                          read(2)
```

---

write(2)

```
write(2)                      System Calls Manual                     write(2)

NAME
       write - write to a file descriptor

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       ssize_t write(int fd, const void buf[.count], size_t count);

DESCRIPTION
       write() writes up to count bytes from the buffer starting at buf to the
       file referred to by the file descriptor fd.

       The number of bytes written may be less than count if, for example,
       there is insufficient space on the underlying physical medium, or the
       RLIMIT_FSIZE resource limit is encountered (see setrlimit(2)), or the
       call was interrupted by a signal handler after having written less than
       count bytes.  (See also pipe(7).)

       For a seekable file (i.e., one to which lseek(2) may be applied, for
       example, a regular file) writing takes place at the file offset, and
       the file offset is incremented by the number of bytes actually written.
       If the file was open(2)ed with O_APPEND, the file offset is first set
       to the end of the file before writing.  The adjustment of the file
       offset and the write operation are performed as an atomic step.

       POSIX requires that a read(2) that can be proved to occur after a
       write() has returned will return the new data.  Note that not all
```

```
       filesystems are POSIX conforming.

       According to POSIX.1, if count is greater than SSIZE_MAX, the result is
       implementation-defined; see NOTES for the upper limit on Linux.

RETURN VALUE
       On success, the number of bytes written is returned.  On error, -1 is
       returned, and errno is set to indicate the error.

       Note that a successful write() may transfer fewer than count bytes.
       Such partial writes can occur for various reasons; for example, because
       there was insufficient space on the disk device to write all of the
       requested bytes, or because a blocked write() to a socket, pipe, or
       similar was interrupted by a signal handler after it had transferred
       some, but before it had transferred all of the requested bytes.  In the
       event of a partial write, the caller can make another write() call to
       transfer the remaining bytes.  The subsequent call will either transfer
       further bytes or may result in an error (e.g., if the disk is now
       full).

       If count is zero and fd refers to a regular file, then write() may
       return a failure status if one of the errors below is detected.  If no
       errors are detected, or error detection is not performed, 0 is returned
       without causing any other effect.  If count is zero and fd refers to a
       file other than a regular file, the results are not specified.

ERRORS
       EAGAIN The file descriptor fd refers to a file other than a socket and
              has been marked nonblocking (O_NONBLOCK), and the write would
              block.  See open(2) for further details on the O_NONBLOCK flag.

       EAGAIN or EWOULDBLOCK
              The file descriptor fd refers to a socket and has been marked
              nonblocking (O_NONBLOCK), and the write would block.
              POSIX.1-2001 allows either error to be returned for this case,
              and does not require these constants to have the same value, so
              a portable application should check for both possibilities.

       EBADF  fd is not a valid file descriptor or is not open for writing.

       EDESTADDRREQ
              fd refers to a datagram socket for which a peer address has not
              been set using connect(2).

       EDQUOT The user's quota of disk blocks on the filesystem containing the
              file referred to by fd has been exhausted.

       EFAULT buf is outside your accessible address space.

       EFBIG  An attempt was made to write a file that exceeds the
              implementation-defined maximum file size or the process's file
              size limit, or to write at a position past the maximum allowed
              offset.

       EINTR  The call was interrupted by a signal before any data was
```

```
              written; see signal(7).

       EINVAL fd is attached to an object which is unsuitable for writing; or
              the file was opened with the O_DIRECT flag, and either the
              address specified in buf, the value specified in count, or the
              file offset is not suitably aligned.

       EIO    A low-level I/O error occurred while modifying the inode.  This
              error may relate to the write-back of data written by an earlier
              write(), which may have been issued to a different file
              descriptor on the same file.  Since Linux 4.13, errors from
              write-back come with a promise that they may be reported by
              subsequent.  write() requests, and will be reported by a
              subsequent fsync(2) (whether or not they were also reported by
              write()).  An alternate cause of EIO on networked filesystems is
              when an advisory lock had been taken out on the file descriptor
              and this lock has been lost.  See the Lost locks section of
              fcntl(2) for further details.

       ENOSPC The device containing the file referred to by fd has no room for
              the data.

       EPERM  The operation was prevented by a file seal; see fcntl(2).

       EPIPE  fd is connected to a pipe or socket whose reading end is closed.
              When this happens the writing process will also receive a
              SIGPIPE signal.  (Thus, the write return value is seen only if
              the program catches, blocks or ignores this signal.)

       Other errors may occur, depending on the object connected to fd.

STANDARDS
       POSIX.1-2008.

HISTORY
       SVr4, 4.3BSD, POSIX.1-2001.

       Under SVr4 a write may be interrupted and return EINTR at any point,
       not just before any data is written.

NOTES
       A successful return from write() does not make any guarantee that data
       has been committed to disk.  On some filesystems, including NFS, it
       does not even guarantee that space has successfully been reserved for
       the data.  In this case, some errors might be delayed until a future
       write(), fsync(2), or even close(2).  The only way to be sure is to
       call fsync(2) after you are done writing all your data.

       If a write() is interrupted by a signal handler before any bytes are
       written, then the call fails with the error EINTR; if it is interrupted
       after at least one byte has been written, the call succeeds, and
       returns the number of bytes written.

       On Linux, write() (and similar system calls) will transfer at most
       0x7ffff000 (2,147,479,552) bytes, returning the number of bytes
```

```
       actually transferred.  (This is true on both 32-bit and 64-bit
       systems.)

       An error return value while performing write() using direct I/O does
       not mean the entire write has failed.  Partial data may be written and
       the data at the file offset on which the write() was attempted should
       be considered inconsistent.

BUGS
       According to POSIX.1-2008/SUSv4 Section XSI 2.9.7 ("Thread Interactions
       with Regular File Operations"):

           All of the following functions shall be atomic with respect to each
           other in the effects specified in POSIX.1-2008 when they operate on
           regular files or symbolic links: ...

       Among the APIs subsequently listed are write() and writev(2).  And
       among the effects that should be atomic across threads (and processes)
       are updates of the file offset.  However, before Linux 3.14, this was
       not the case: if two processes that share an open file description (see
       open(2)) perform a write() (or writev(2)) at the same time, then the
       I/O operations were not atomic with respect to updating the file
       offset, with the result that the blocks of data output by the two
       processes might (incorrectly) overlap.  This problem was fixed in Linux
       3.14.

SEE ALSO
       close(2), fcntl(2), fsync(2), ioctl(2), lseek(2), open(2), pwrite(2),
       read(2), select(2), writev(2), fwrite(3)

Linux man-pages 6.7              2023-10-31                          write(2)
```

---

open(2) and openat(2)

```
open(2)                       System Calls Manual                       open(2)

NAME
       open, openat, creat - open and possibly create a file

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <fcntl.h>

       int open(const char *pathname, int flags, ...
                   /* mode_t mode */ );

       int creat(const char *pathname, mode_t mode);

       int openat(int dirfd, const char *pathname, int flags, ...
                   /* mode_t mode */ );
```

```
       /* Documented separately, in openat2(2): */
       int openat2(int dirfd, const char *pathname,
                   const struct open_how *how, size_t size);

   Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

       openat():
           Since glibc 2.10:
               _POSIX_C_SOURCE >= 200809L
           Before glibc 2.10:
               _ATFILE_SOURCE

DESCRIPTION
       The open() system call opens the file specified by pathname.  If the
       specified file does not exist, it may optionally (if O_CREAT is
       specified in flags) be created by open().

       The return value of open() is a file descriptor, a small, nonnegative
       integer that is an index to an entry in the process's table of open
       file descriptors.  The file descriptor is used in subsequent system
       calls (read(2), write(2), lseek(2), fcntl(2), etc.)  to refer to the
       open file.  The file descriptor returned by a successful call will be
       the lowest-numbered file descriptor not currently open for the process.

       By default, the new file descriptor is set to remain open across an
       execve(2) (i.e., the FD_CLOEXEC file descriptor flag described in
       fcntl(2) is initially disabled); the O_CLOEXEC flag, described below,
       can be used to change this default.  The file offset is set to the
       beginning of the file (see lseek(2)).

       A call to open() creates a new open file description, an entry in the
       system-wide table of open files.  The open file description records the
       file offset and the file status flags (see below).  A file descriptor
       is a reference to an open file description; this reference is
       unaffected if pathname is subsequently removed or modified to refer to
       a different file.  For further details on open file descriptions, see
       NOTES.

       The argument flags must include one of the following access modes:
       O_RDONLY, O_WRONLY, or O_RDWR.  These request opening the file read-
       only, write-only, or read/write, respectively.

       In addition, zero or more file creation flags and file status flags can
       be bitwise ORed in flags.  The file creation flags are O_CLOEXEC,
       O_CREAT, O_DIRECTORY, O_EXCL, O_NOCTTY, O_NOFOLLOW, O_TMPFILE, and
       O_TRUNC.  The file status flags are all of the remaining flags listed
       below.  The distinction between these two groups of flags is that the
       file creation flags affect the semantics of the open operation itself,
       while the file status flags affect the semantics of subsequent I/O
       operations.  The file status flags can be retrieved and (in some cases)
       modified; see fcntl(2) for details.

       The full list of file creation flags and file status flags is as
       follows:
```

```
O_APPEND
       The file is opened in append mode.  Before each write(2), the
       file offset is positioned at the end of the file, as if with
       lseek(2).  The modification of the file offset and the write
       operation are performed as a single atomic step.

       O_APPEND may lead to corrupted files on NFS filesystems if more
       than one process appends data to a file at once.  This is
       because NFS does not support appending to a file, so the client
       kernel has to simulate it, which can't be done without a race
       condition.

O_ASYNC
       Enable signal-driven I/O: generate a signal (SIGIO by default,
       but this can be changed via fcntl(2)) when input or output
       becomes possible on this file descriptor.  This feature is
       available only for terminals, pseudoterminals, sockets, and
       (since Linux 2.6) pipes and FIFOs.  See fcntl(2) for further
       details.  See also BUGS, below.

O_CLOEXEC (since Linux 2.6.23)
       Enable the close-on-exec flag for the new file descriptor.
       Specifying this flag permits a program to avoid additional
       fcntl(2) F_SETFD operations to set the FD_CLOEXEC flag.

       Note that the use of this flag is essential in some
       multithreaded programs, because using a separate fcntl(2)
       F_SETFD operation to set the FD_CLOEXEC flag does not suffice to
       avoid race conditions where one thread opens a file descriptor
       and attempts to set its close-on-exec flag using fcntl(2) at the
       same time as another thread does a fork(2) plus execve(2).
       Depending on the order of execution, the race may lead to the
       file descriptor returned by open() being unintentionally leaked
       to the program executed by the child process created by fork(2).
       (This kind of race is in principle possible for any system call
       that creates a file descriptor whose close-on-exec flag should
       be set, and various other Linux system calls provide an
       equivalent of the O_CLOEXEC flag to deal with this problem.)

O_CREAT
       If pathname does not exist, create it as a regular file.

       The owner (user ID) of the new file is set to the effective user
       ID of the process.

       The group ownership (group ID) of the new file is set either to
       the effective group ID of the process (System V semantics) or to
       the group ID of the parent directory (BSD semantics).  On Linux,
       the behavior depends on whether the set-group-ID mode bit is set
       on the parent directory: if that bit is set, then BSD semantics
       apply; otherwise, System V semantics apply.  For some
       filesystems, the behavior also depends on the bsdgroups and
       sysvgroups mount options described in mount(8).
```

```
       The mode argument specifies the file mode bits to be applied
       when a new file is created.  If neither O_CREAT nor O_TMPFILE is
       specified in flags, then mode is ignored (and can thus be
       specified as 0, or simply omitted).  The mode argument must be
       supplied if O_CREAT or O_TMPFILE is specified in flags; if it is
       not supplied, some arbitrary bytes from the stack will be
       applied as the file mode.

       The effective mode is modified by the process's umask in the
       usual way: in the absence of a default ACL, the mode of the
       created file is (mode & ~umask).

       Note that mode applies only to future accesses of the newly
       created file; the open() call that creates a read-only file may
       well return a read/write file descriptor.

       The following symbolic constants are provided for mode:

       S_IRWXU  00700 user (file owner) has read, write, and execute
                permission

       S_IRUSR  00400 user has read permission

       S_IWUSR  00200 user has write permission

       S_IXUSR  00100 user has execute permission

       S_IRWXG  00070 group has read, write, and execute permission

       S_IRGRP  00040 group has read permission

       S_IWGRP  00020 group has write permission

       S_IXGRP  00010 group has execute permission

       S_IRWXO  00007 others have read, write, and execute permission

       S_IROTH  00004 others have read permission

       S_IWOTH  00002 others have write permission

       S_IXOTH  00001 others have execute permission

       According to POSIX, the effect when other bits are set in mode
       is unspecified.  On Linux, the following bits are also honored
       in mode:

       S_ISUID  0004000 set-user-ID bit

       S_ISGID  0002000 set-group-ID bit (see inode(7)).

       S_ISVTX  0001000 sticky bit (see inode(7)).

  O_DIRECT (since Linux 2.4.10)
       Try to minimize cache effects of the I/O to and from this file.
```

```
       In general this will degrade performance, but it is useful in
       special situations, such as when applications do their own
       caching.  File I/O is done directly to/from user-space buffers.
       The O_DIRECT flag on its own makes an effort to transfer data
       synchronously, but does not give the guarantees of the O_SYNC
       flag that data and necessary metadata are transferred.  To
       guarantee synchronous I/O, O_SYNC must be used in addition to
       O_DIRECT.  See NOTES below for further discussion.

       A semantically similar (but deprecated) interface for block
       devices is described in raw(8).

O_DIRECTORY
       If pathname is not a directory, cause the open to fail.  This
       flag was added in Linux 2.1.126, to avoid denial-of-service
       problems if opendir(3) is called on a FIFO or tape device.

O_DSYNC
       Write operations on the file will complete according to the
       requirements of synchronized I/O data integrity completion.

       By the time write(2) (and similar) return, the output data has
       been transferred to the underlying hardware, along with any file
       metadata that would be required to retrieve that data (i.e., as
       though each write(2) was followed by a call to fdatasync(2)).
       See NOTES below.

O_EXCL Ensure that this call creates the file: if this flag is
       specified in conjunction with O_CREAT, and pathname already
       exists, then open() fails with the error EEXIST.

       When these two flags are specified, symbolic links are not
       followed: if pathname is a symbolic link, then open() fails
       regardless of where the symbolic link points.

       In general, the behavior of O_EXCL is undefined if it is used
       without O_CREAT.  There is one exception: on Linux 2.6 and
       later, O_EXCL can be used without O_CREAT if pathname refers to
       a block device.  If the block device is in use by the system
       (e.g., mounted), open() fails with the error EBUSY.

       On NFS, O_EXCL is supported only when using NFSv3 or later on
       kernel 2.6 or later.  In NFS environments where O_EXCL support
       is not provided, programs that rely on it for performing locking
       tasks will contain a race condition.  Portable programs that
       want to perform atomic file locking using a lockfile, and need
       to avoid reliance on NFS support for O_EXCL, can create a unique
       file on the same filesystem (e.g., incorporating hostname and
       PID), and use link(2) to make a link to the lockfile.  If
       link(2) returns 0, the lock is successful.  Otherwise, use
       stat(2) on the unique file to check if its link count has
       increased to 2, in which case the lock is also successful.

O_LARGEFILE
       (LFS) Allow files whose sizes cannot be represented in an off_t
```

```
       (but can be represented in an off64_t) to be opened.  The
       _LARGEFILE64_SOURCE macro must be defined (before including any
       header files) in order to obtain this definition.  Setting the
       _FILE_OFFSET_BITS feature test macro to 64 (rather than using
       O_LARGEFILE) is the preferred method of accessing large files on
       32-bit systems (see feature_test_macros(7)).

O_NOATIME (since Linux 2.6.8)
       Do not update the file last access time (st_atime in the inode)
       when the file is read(2).

       This flag can be employed only if one of the following
       conditions is true:

       • The effective UID of the process matches the owner UID of the
         file.

       • The calling process has the CAP_FOWNER capability in its user
         namespace and the owner UID of the file has a mapping in the
         namespace.

       This flag is intended for use by indexing or backup programs,
       where its use can significantly reduce the amount of disk
       activity.  This flag may not be effective on all filesystems.
       One example is NFS, where the server maintains the access time.

O_NOCTTY
       If pathname refers to a terminal device-see tty(4)-it will not
       become the process's controlling terminal even if the process
       does not have one.

O_NOFOLLOW
       If the trailing component (i.e., basename) of pathname is a
       symbolic link, then the open fails, with the error ELOOP.
       Symbolic links in earlier components of the pathname will still
       be followed.  (Note that the ELOOP error that can occur in this
       case is indistinguishable from the case where an open fails
       because there are too many symbolic links found while resolving
       components in the prefix part of the pathname.)

       This flag is a FreeBSD extension, which was added in Linux
       2.1.126, and has subsequently been standardized in POSIX.1-2008.

       See also O_PATH below.

O_NONBLOCK or O_NDELAY
       When possible, the file is opened in nonblocking mode.  Neither
       the open() nor any subsequent I/O operations on the file
       descriptor which is returned will cause the calling process to
       wait.

       Note that the setting of this flag has no effect on the
       operation of poll(2), select(2), epoll(7), and similar, since
       those interfaces merely inform the caller about whether a file
       descriptor is "ready", meaning that an I/O operation performed
```

```
              on the file descriptor with the O_NONBLOCK flag clear would not
              block.

              Note that this flag has no effect for regular files and block
              devices; that is, I/O operations will (briefly) block when
              device activity is required, regardless of whether O_NONBLOCK is
              set.  Since O_NONBLOCK semantics might eventually be
              implemented, applications should not depend upon blocking
              behavior when specifying this flag for regular files and block
              devices.

              For the handling of FIFOs (named pipes), see also fifo(7).  For
              a discussion of the effect of O_NONBLOCK in conjunction with
              mandatory file locks and with file leases, see fcntl(2).

       O_PATH (since Linux 2.6.39)
              Obtain a file descriptor that can be used for two purposes: to
              indicate a location in the filesystem tree and to perform
              operations that act purely at the file descriptor level.  The
              file itself is not opened, and other file operations (e.g.,
              read(2), write(2), fchmod(2), fchown(2), fgetxattr(2), ioctl(2),
              mmap(2)) fail with the error EBADF.

              The following operations can be performed on the resulting file
              descriptor:

              •  close(2).

              •  fchdir(2), if the file descriptor refers to a directory
                 (since Linux 3.5).

              •  fstat(2) (since Linux 3.6).

              •  fstatfs(2) (since Linux 3.12).

              •  Duplicating the file descriptor (dup(2), fcntl(2) F_DUPFD,
                 etc.).

              •  Getting and setting file descriptor flags (fcntl(2) F_GETFD
                 and F_SETFD).

              •  Retrieving open file status flags using the fcntl(2) F_GETFL
                 operation: the returned flags will include the bit O_PATH.

              •  Passing the file descriptor as the dirfd argument of openat()
                 and the other "*at()" system calls.  This includes linkat(2)
                 with AT_EMPTY_PATH (or via procfs using AT_SYMLINK_FOLLOW)
                 even if the file is not a directory.

              •  Passing the file descriptor to another process via a UNIX
                 domain socket (see SCM_RIGHTS in unix(7)).

              When O_PATH is specified in flags, flag bits other than
              O_CLOEXEC, O_DIRECTORY, and O_NOFOLLOW are ignored.
```

```
       Opening a file or directory with the O_PATH flag requires no
       permissions on the object itself (but does require execute
       permission on the directories in the path prefix).  Depending on
       the subsequent operation, a check for suitable file permissions
       may be performed (e.g., fchdir(2) requires execute permission on
       the directory referred to by its file descriptor argument).  By
       contrast, obtaining a reference to a filesystem object by
       opening it with the O_RDONLY flag requires that the caller have
       read permission on the object, even when the subsequent
       operation (e.g., fchdir(2), fstat(2)) does not require read
       permission on the object.

       If pathname is a symbolic link and the O_NOFOLLOW flag is also
       specified, then the call returns a file descriptor referring to
       the symbolic link.  This file descriptor can be used as the
       dirfd argument in calls to fchownat(2), fstatat(2), linkat(2),
       and readlinkat(2) with an empty pathname to have the calls
       operate on the symbolic link.

       If pathname refers to an automount point that has not yet been
       triggered, so no other filesystem is mounted on it, then the
       call returns a file descriptor referring to the automount
       directory without triggering a mount.  fstatfs(2) can then be
       used to determine if it is, in fact, an untriggered automount
       point (.f_type == AUTOFS_SUPER_MAGIC).

       One use of O_PATH for regular files is to provide the equivalent
       of POSIX.1's O_EXEC functionality.  This permits us to open a
       file for which we have execute permission but not read
       permission, and then execute that file, with steps something
       like the following:

           char buf[PATH_MAX];
           fd = open("some_prog", O_PATH);
           snprintf(buf, PATH_MAX, "/proc/self/fd/%d", fd);
           execl(buf, "some_prog", (char *) NULL);

       An O_PATH file descriptor can also be passed as the argument of
       fexecve(3).

O_SYNC Write operations on the file will complete according to the
       requirements of synchronized I/O file integrity completion (by
       contrast with the synchronized I/O data integrity completion
       provided by O_DSYNC.)

       By the time write(2) (or similar) returns, the output data and
       associated file metadata have been transferred to the underlying
       hardware (i.e., as though each write(2) was followed by a call
       to fsync(2)).  See NOTES below.

O_TMPFILE (since Linux 3.11)
       Create an unnamed temporary regular file.  The pathname argument
       specifies a directory; an unnamed inode will be created in that
       directory's filesystem.  Anything written to the resulting file
       will be lost when the last file descriptor is closed, unless the
```

```
file is given a name.

O_TMPFILE must be specified with one of O_RDWR or O_WRONLY and,
optionally, O_EXCL.  If O_EXCL is not specified, then linkat(2)
can be used to link the temporary file into the filesystem,
making it permanent, using code like the following:

    char path[PATH_MAX];
    fd = open("/path/to/dir", O_TMPFILE | O_RDWR,
                            S_IRUSR | S_IWUSR);

    /* File I/O on 'fd'... */

    linkat(fd, "", AT_FDCWD, "/path/for/file", AT_EMPTY_PATH);

    /* If the caller doesn't have the CAP_DAC_READ_SEARCH
       capability (needed to use AT_EMPTY_PATH with linkat(2)),
       and there is a proc(5) filesystem mounted, then the
       linkat(2) call above can be replaced with:

    snprintf(path, PATH_MAX,  "/proc/self/fd/%d", fd);
    linkat(AT_FDCWD, path, AT_FDCWD, "/path/for/file",
                            AT_SYMLINK_FOLLOW);
    */

In this case, the open() mode argument determines the file
permission mode, as with O_CREAT.

Specifying O_EXCL in conjunction with O_TMPFILE prevents a
temporary file from being linked into the filesystem in the
above manner.  (Note that the meaning of O_EXCL in this case is
different from the meaning of O_EXCL otherwise.)

There are two main use cases for O_TMPFILE:

•  Improved tmpfile(3) functionality: race-free creation of
   temporary files that (1) are automatically deleted when
   closed; (2) can never be reached via any pathname; (3) are
   not subject to symlink attacks; and (4) do not require the
   caller to devise unique names.

•  Creating a file that is initially invisible, which is then
   populated with data and adjusted to have appropriate
   filesystem attributes (fchown(2), fchmod(2), fsetxattr(2),
   etc.)  before being atomically linked into the filesystem in
   a fully formed state (using linkat(2) as described above).

O_TMPFILE requires support by the underlying filesystem; only a
subset of Linux filesystems provide that support.  In the
initial implementation, support was provided in the ext2, ext3,
ext4, UDF, Minix, and tmpfs filesystems.  Support for other
filesystems has subsequently been added as follows: XFS (Linux
3.15); Btrfs (Linux 3.16); F2FS (Linux 3.16); and ubifs (Linux
4.9)
```

```
       O_TRUNC
              If the file already exists and is a regular file and the access
              mode allows writing (i.e., is O_RDWR or O_WRONLY) it will be
              truncated to length 0.  If the file is a FIFO or terminal device
              file, the O_TRUNC flag is ignored.  Otherwise, the effect of
              O_TRUNC is unspecified.

   creat()
       A call to creat() is equivalent to calling open() with flags equal to
       O_CREAT|O_WRONLY|O_TRUNC.

   openat()
       The openat() system call operates in exactly the same way as open(),
       except for the differences described here.

       The dirfd argument is used in conjunction with the pathname argument as
       follows:

       • If the pathname given in pathname is absolute, then dirfd is
         ignored.

       • If the pathname given in pathname is relative and dirfd is the
         special value AT_FDCWD, then pathname is interpreted relative to the
         current working directory of the calling process (like open()).

       • If the pathname given in pathname is relative, then it is
         interpreted relative to the directory referred to by the file
         descriptor dirfd (rather than relative to the current working
         directory of the calling process, as is done by open() for a
         relative pathname).  In this case, dirfd must be a directory that
         was opened for reading (O_RDONLY) or using the O_PATH flag.

       If the pathname given in pathname is relative, and dirfd is not a valid
       file descriptor, an error (EBADF) results.  (Specifying an invalid file
       descriptor number in dirfd can be used as a means to ensure that
       pathname is absolute.)

   openat2(2)
       The openat2(2) system call is an extension of openat(), and provides a
       superset of the features of openat().  It is documented separately, in
       openat2(2).

RETURN VALUE
       On success, open(), openat(), and creat() return the new file
       descriptor (a nonnegative integer).  On error, -1 is returned and errno
       is set to indicate the error.

ERRORS
       open(), openat(), and creat() can fail with the following errors:

       EACCES The requested access to the file is not allowed, or search
              permission is denied for one of the directories in the path
              prefix of pathname, or the file did not exist yet and write
              access to the parent directory is not allowed.  (See also
              path_resolution(7).)
```

```
       EACCES Where O_CREAT is specified, the protected_fifos or
              protected_regular sysctl is enabled, the file already exists and
              is a FIFO or regular file, the owner of the file is neither the
              current user nor the owner of the containing directory, and the
              containing directory is both world- or group-writable and
              sticky.  For details, see the descriptions of
              /proc/sys/fs/protected_fifos and /proc/sys/fs/protected_regular
              in proc(5).

       EBADF  (openat()) pathname is relative but dirfd is neither AT_FDCWD
              nor a valid file descriptor.

       EBUSY  O_EXCL was specified in flags and pathname refers to a block
              device that is in use by the system (e.g., it is mounted).

       EDQUOT Where O_CREAT is specified, the file does not exist, and the
              user's quota of disk blocks or inodes on the filesystem has been
              exhausted.

       EEXIST pathname already exists and O_CREAT and O_EXCL were used.

       EFAULT pathname points outside your accessible address space.

       EFBIG  See EOVERFLOW.

       EINTR  While blocked waiting to complete an open of a slow device
              (e.g., a FIFO; see fifo(7)), the call was interrupted by a
              signal handler; see signal(7).

       EINVAL The filesystem does not support the O_DIRECT flag.  See NOTES
              for more information.

       EINVAL Invalid value in flags.

       EINVAL O_TMPFILE was specified in flags, but neither O_WRONLY nor
              O_RDWR was specified.

       EINVAL O_CREAT was specified in flags and the final component
              ("basename") of the new file's pathname is invalid (e.g., it
              contains characters not permitted by the underlying filesystem).

       EINVAL The final component ("basename") of pathname is invalid (e.g.,
              it contains characters not permitted by the underlying
              filesystem).

       EISDIR pathname refers to a directory and the access requested involved
              writing (that is, O_WRONLY or O_RDWR is set).

       EISDIR pathname refers to an existing directory, O_TMPFILE and one of
              O_WRONLY or O_RDWR were specified in flags, but this kernel
              version does not provide the O_TMPFILE functionality.

       ELOOP  Too many symbolic links were encountered in resolving pathname.
```

```
       ELOOP  pathname was a symbolic link, and flags specified O_NOFOLLOW but
              not O_PATH.

       EMFILE The per-process limit on the number of open file descriptors has
              been reached (see the description of RLIMIT_NOFILE in
              getrlimit(2)).

       ENAMETOOLONG
              pathname was too long.

       ENFILE The system-wide limit on the total number of open files has been
              reached.

       ENODEV pathname refers to a device special file and no corresponding
              device exists.  (This is a Linux kernel bug; in this situation
              ENXIO must be returned.)

       ENOENT O_CREAT is not set and the named file does not exist.

       ENOENT A directory component in pathname does not exist or is a
              dangling symbolic link.

       ENOENT pathname refers to a nonexistent directory, O_TMPFILE and one of
              O_WRONLY or O_RDWR were specified in flags, but this kernel
              version does not provide the O_TMPFILE functionality.

       ENOMEM The named file is a FIFO, but memory for the FIFO buffer can't
              be allocated because the per-user hard limit on memory
              allocation for pipes has been reached and the caller is not
              privileged; see pipe(7).

       ENOMEM Insufficient kernel memory was available.

       ENOSPC pathname was to be created but the device containing pathname
              has no room for the new file.

       ENOTDIR
              A component used as a directory in pathname is not, in fact, a
              directory, or O_DIRECTORY was specified and pathname was not a
              directory.

       ENOTDIR
              (openat()) pathname is a relative pathname and dirfd is a file
              descriptor referring to a file other than a directory.

       ENXIO  O_NONBLOCK | O_WRONLY is set, the named file is a FIFO, and no
              process has the FIFO open for reading.

       ENXIO  The file is a device special file and no corresponding device
              exists.

       ENXIO  The file is a UNIX domain socket.

       EOPNOTSUPP
              The filesystem containing pathname does not support O_TMPFILE.
```

```
       EOVERFLOW
              pathname refers to a regular file that is too large to be
              opened.  The usual scenario here is that an application compiled
              on a 32-bit platform without -D_FILE_OFFSET_BITS=64 tried to
              open a file whose size exceeds (1<<31)-1 bytes; see also
              O_LARGEFILE above.  This is the error specified by POSIX.1;
              before Linux 2.6.24, Linux gave the error EFBIG for this case.

       EPERM  The O_NOATIME flag was specified, but the effective user ID of
              the caller did not match the owner of the file and the caller
              was not privileged.

       EPERM  The operation was prevented by a file seal; see fcntl(2).

       EROFS  pathname refers to a file on a read-only filesystem and write
              access was requested.

       ETXTBSY
              pathname refers to an executable image which is currently being
              executed and write access was requested.

       ETXTBSY
              pathname refers to a file that is currently in use as a swap
              file, and the O_TRUNC flag was specified.

       ETXTBSY
              pathname refers to a file that is currently being read by the
              kernel (e.g., for module/firmware loading), and write access was
              requested.

       EWOULDBLOCK
              The O_NONBLOCK flag was specified, and an incompatible lease was
              held on the file (see fcntl(2)).

VERSIONS
       The (undefined) effect of O_RDONLY | O_TRUNC varies among
       implementations.  On many systems the file is actually truncated.

   Synchronized I/O
       The POSIX.1-2008 "synchronized I/O" option specifies different variants
       of synchronized I/O, and specifies the open() flags O_SYNC, O_DSYNC,
       and O_RSYNC for controlling the behavior.  Regardless of whether an
       implementation supports this option, it must at least support the use
       of O_SYNC for regular files.

       Linux implements O_SYNC and O_DSYNC, but not O_RSYNC.  Somewhat
       incorrectly, glibc defines O_RSYNC to have the same value as O_SYNC.
       (O_RSYNC is defined in the Linux header file <asm/fcntl.h> on HP PA-
       RISC, but it is not used.)

       O_SYNC provides synchronized I/O file integrity completion, meaning
       write operations will flush data and all associated metadata to the
       underlying hardware.  O_DSYNC provides synchronized I/O data integrity
       completion, meaning write operations will flush data to the underlying
```

```
       hardware, but will only flush metadata updates that are required to
       allow a subsequent read operation to complete successfully.  Data
       integrity completion can reduce the number of disk operations that are
       required for applications that don't need the guarantees of file
       integrity completion.

       To understand the difference between the two types of completion,
       consider two pieces of file metadata: the file last modification
       timestamp (st_mtime) and the file length.  All write operations will
       update the last file modification timestamp, but only writes that add
       data to the end of the file will change the file length.  The last
       modification timestamp is not needed to ensure that a read completes
       successfully, but the file length is.  Thus, O_DSYNC would only
       guarantee to flush updates to the file length metadata (whereas O_SYNC
       would also always flush the last modification timestamp metadata).

       Before Linux 2.6.33, Linux implemented only the O_SYNC flag for open().
       However, when that flag was specified, most filesystems actually
       provided the equivalent of synchronized I/O data integrity completion
       (i.e., O_SYNC was actually implemented as the equivalent of O_DSYNC).

       Since Linux 2.6.33, proper O_SYNC support is provided.  However, to
       ensure backward binary compatibility, O_DSYNC was defined with the same
       value as the historical O_SYNC, and O_SYNC was defined as a new (two-
       bit) flag value that includes the O_DSYNC flag value.  This ensures
       that applications compiled against new headers get at least O_DSYNC
       semantics before Linux 2.6.33.

   C library/kernel differences
       Since glibc 2.26, the glibc wrapper function for open() employs the
       openat() system call, rather than the kernel's open() system call.  For
       certain architectures, this is also true before glibc 2.26.

STANDARDS
       open()
       creat()
       openat()
              POSIX.1-2008.

       openat2(2) Linux.

       The O_DIRECT, O_NOATIME, O_PATH, and O_TMPFILE flags are Linux-
       specific.  One must define _GNU_SOURCE to obtain their definitions.

       The O_CLOEXEC, O_DIRECTORY, and O_NOFOLLOW flags are not specified in
       POSIX.1-2001, but are specified in POSIX.1-2008.  Since glibc 2.12, one
       can obtain their definitions by defining either _POSIX_C_SOURCE with a
       value greater than or equal to 200809L or _XOPEN_SOURCE with a value
       greater than or equal to 700.  In glibc 2.11 and earlier, one obtains
       the definitions by defining _GNU_SOURCE.

HISTORY
       open()
       creat()
              SVr4, 4.3BSD, POSIX.1-2001.
```

```
       openat()
              POSIX.1-2008.  Linux 2.6.16, glibc 2.4.

NOTES
       Under Linux, the O_NONBLOCK flag is sometimes used in cases where one
       wants to open but does not necessarily have the intention to read or
       write.  For example, this may be used to open a device in order to get
       a file descriptor for use with ioctl(2).

       Note that open() can open device special files, but creat() cannot
       create them; use mknod(2) instead.

       If the file is newly created, its st_atime, st_ctime, st_mtime fields
       (respectively, time of last access, time of last status change, and
       time of last modification; see stat(2)) are set to the current time,
       and so are the st_ctime and st_mtime fields of the parent directory.
       Otherwise, if the file is modified because of the O_TRUNC flag, its
       st_ctime and st_mtime fields are set to the current time.

       The files in the /proc/pid/fd directory show the open file descriptors
       of the process with the PID pid.  The files in the /proc/pid/fdinfo
       directory show even more information about these file descriptors.  See
       proc(5) for further details of both of these directories.

       The Linux header file <asm/fcntl.h> doesn't define O_ASYNC; the (BSD-
       derived) FASYNC synonym is defined instead.

   Open file descriptions
       The term open file description is the one used by POSIX to refer to the
       entries in the system-wide table of open files.  In other contexts,
       this object is variously also called an "open file object", a "file
       handle", an "open file table entry", or-in kernel-developer parlance-a
       struct file.

       When a file descriptor is duplicated (using dup(2) or similar), the
       duplicate refers to the same open file description as the original file
       descriptor, and the two file descriptors consequently share the file
       offset and file status flags.  Such sharing can also occur between
       processes: a child process created via fork(2) inherits duplicates of
       its parent's file descriptors, and those duplicates refer to the same
       open file descriptions.

       Each open() of a file creates a new open file description; thus, there
       may be multiple open file descriptions corresponding to a file inode.

       On Linux, one can use the kcmp(2) KCMP_FILE operation to test whether
       two file descriptors (in the same process or in two different
       processes) refer to the same open file description.

   NFS
       There are many infelicities in the protocol underlying NFS, affecting
       amongst others O_SYNC and O_NDELAY.

       On NFS filesystems with UID mapping enabled, open() may return a file
```

```
       descriptor but, for example, read(2) requests are denied with EACCES.
       This is because the client performs open() by checking the permissions,
       but UID mapping is performed by the server upon read and write
       requests.

   FIFOs
       Opening the read or write end of a FIFO blocks until the other end is
       also opened (by another process or thread).  See fifo(7) for further
       details.

   File access mode
       Unlike the other values that can be specified in flags, the access mode
       values O_RDONLY, O_WRONLY, and O_RDWR do not specify individual bits.
       Rather, they define the low order two bits of flags, and are defined
       respectively as 0, 1, and 2.  In other words, the combination O_RDONLY
       | O_WRONLY is a logical error, and certainly does not have the same
       meaning as O_RDWR.

       Linux reserves the special, nonstandard access mode 3 (binary 11) in
       flags to mean: check for read and write permission on the file and
       return a file descriptor that can't be used for reading or writing.
       This nonstandard access mode is used by some Linux drivers to return a
       file descriptor that is to be used only for device-specific ioctl(2)
       operations.

   Rationale for openat() and other directory file descriptor APIs
       openat() and the other system calls and library functions that take a
       directory file descriptor argument (i.e., execveat(2), faccessat(2),
       fanotify_mark(2), fchmodat(2), fchownat(2), fspick(2), fstatat(2),
       futimesat(2), linkat(2), mkdirat(2), mknodat(2), mount_setattr(2),
       move_mount(2), name_to_handle_at(2), open_tree(2), openat2(2),
       readlinkat(2), renameat(2), renameat2(2), statx(2), symlinkat(2),
       unlinkat(2), utimensat(2), mkfifoat(3), and scandirat(3)) address two
       problems with the older interfaces that preceded them.  Here, the
       explanation is in terms of the openat() call, but the rationale is
       analogous for the other interfaces.

       First, openat() allows an application to avoid race conditions that
       could occur when using open() to open files in directories other than
       the current working directory.  These race conditions result from the
       fact that some component of the directory prefix given to open() could
       be changed in parallel with the call to open().  Suppose, for example,
       that we wish to create the file dir1/dir2/xxx.dep if the file
       dir1/dir2/xxx exists.  The problem is that between the existence check
       and the file-creation step, dir1 or dir2 (which might be symbolic
       links) could be modified to point to a different location.  Such races
       can be avoided by opening a file descriptor for the target directory,
       and then specifying that file descriptor as the dirfd argument of (say)
       fstatat(2) and openat().  The use of the dirfd file descriptor also has
       other benefits:

       •  the file descriptor is a stable reference to the directory, even if
          the directory is renamed; and

       •  the open file descriptor prevents the underlying filesystem from
```

```
          being dismounted, just as when a process has a current working
          directory on a filesystem.

      Second, openat() allows the implementation of a per-thread "current
      working directory", via file descriptor(s) maintained by the
      application.  (This functionality can also be obtained by tricks based
      on the use of /proc/self/fd/dirfd, but less efficiently.)

      The dirfd argument for these APIs can be obtained by using open() or
      openat() to open a directory (with either the O_RDONLY or the O_PATH
      flag).  Alternatively, such a file descriptor can be obtained by
      applying dirfd(3) to a directory stream created using opendir(3).

      When these APIs are given a dirfd argument of AT_FDCWD or the specified
      pathname is absolute, then they handle their pathname argument in the
      same way as the corresponding conventional APIs.  However, in this
      case, several of the APIs have a flags argument that provides access to
      functionality that is not available with the corresponding conventional
      APIs.

  O_DIRECT
      The O_DIRECT flag may impose alignment restrictions on the length and
      address of user-space buffers and the file offset of I/Os.  In Linux
      alignment restrictions vary by filesystem and kernel version and might
      be absent entirely.  The handling of misaligned O_DIRECT I/Os also
      varies; they can either fail with EINVAL or fall back to buffered I/O.

      Since Linux 6.1, O_DIRECT support and alignment restrictions for a file
      can be queried using statx(2), using the STATX_DIOALIGN flag.  Support
      for STATX_DIOALIGN varies by filesystem; see statx(2).

      Some filesystems provide their own interfaces for querying O_DIRECT
      alignment restrictions, for example the XFS_IOC_DIOINFO operation in
      xfsctl(3).  STATX_DIOALIGN should be used instead when it is available.

      If none of the above is available, then direct I/O support and
      alignment restrictions can only be assumed from known characteristics
      of the filesystem, the individual file, the underlying storage
      device(s), and the kernel version.  In Linux 2.4, most filesystems
      based on block devices require that the file offset and the length and
      memory address of all I/O segments be multiples of the filesystem block
      size (typically 4096 bytes).  In Linux 2.6.0, this was relaxed to the
      logical block size of the block device (typically 512 bytes).  A block
      device's logical block size can be determined using the ioctl(2)
      BLKSSZGET operation or from the shell using the command:

          blockdev --getss

      O_DIRECT I/Os should never be run concurrently with the fork(2) system
      call, if the memory buffer is a private mapping (i.e., any mapping
      created with the mmap(2) MAP_PRIVATE flag; this includes memory
      allocated on the heap and statically allocated buffers).  Any such
      I/Os, whether submitted via an asynchronous I/O interface or from
      another thread in the process, should be completed before fork(2) is
      called.  Failure to do so can result in data corruption and undefined
```

```
       behavior in parent and child processes.  This restriction does not
       apply when the memory buffer for the O_DIRECT I/Os was created using
       shmat(2) or mmap(2) with the MAP_SHARED flag.  Nor does this
       restriction apply when the memory buffer has been advised as
       MADV_DONTFORK with madvise(2), ensuring that it will not be available
       to the child after fork(2).

       The O_DIRECT flag was introduced in SGI IRIX, where it has alignment
       restrictions similar to those of Linux 2.4.  IRIX has also a fcntl(2)
       call to query appropriate alignments, and sizes.  FreeBSD 4.x
       introduced a flag of the same name, but without alignment restrictions.

       O_DIRECT support was added in Linux 2.4.10.  Older Linux kernels simply
       ignore this flag.  Some filesystems may not implement the flag, in
       which case open() fails with the error EINVAL if it is used.

       Applications should avoid mixing O_DIRECT and normal I/O to the same
       file, and especially to overlapping byte regions in the same file.
       Even when the filesystem correctly handles the coherency issues in this
       situation, overall I/O throughput is likely to be slower than using
       either mode alone.  Likewise, applications should avoid mixing mmap(2)
       of files with direct I/O to the same files.

       The behavior of O_DIRECT with NFS will differ from local filesystems.
       Older kernels, or kernels configured in certain ways, may not support
       this combination.  The NFS protocol does not support passing the flag
       to the server, so O_DIRECT I/O will bypass the page cache only on the
       client; the server may still cache the I/O.  The client asks the server
       to make the I/O synchronous to preserve the synchronous semantics of
       O_DIRECT.  Some servers will perform poorly under these circumstances,
       especially if the I/O size is small.  Some servers may also be
       configured to lie to clients about the I/O having reached stable
       storage; this will avoid the performance penalty at some risk to data
       integrity in the event of server power failure.  The Linux NFS client
       places no alignment restrictions on O_DIRECT I/O.

       In summary, O_DIRECT is a potentially powerful tool that should be used
       with caution.  It is recommended that applications treat use of
       O_DIRECT as a performance option which is disabled by default.

BUGS
       Currently, it is not possible to enable signal-driven I/O by specifying
       O_ASYNC when calling open(); use fcntl(2) to enable this flag.

       One must check for two different error codes, EISDIR and ENOENT, when
       trying to determine whether the kernel supports O_TMPFILE
       functionality.

       When both O_CREAT and O_DIRECTORY are specified in flags and the file
       specified by pathname does not exist, open() will create a regular file
       (i.e., O_DIRECTORY is ignored).

SEE ALSO
       chmod(2), chown(2), close(2), dup(2), fcntl(2), link(2), lseek(2),
       mknod(2), mmap(2), mount(2), open_by_handle_at(2), openat2(2), read(2),
```

```
       socket(2), stat(2), umask(2), unlink(2), write(2), fopen(3), acl(5),
       fifo(7), inode(7), path_resolution(7), symlink(7)

Linux man-pages 6.7              2024-01-16                          open(2)
```

---

ioctl(2)

```
ioctl(2)                      System Calls Manual                     ioctl(2)

NAME
       ioctl - control device

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/ioctl.h>

       int ioctl(int fd, unsigned long op, ...);  /* glibc, BSD */
       int ioctl(int fd, int op, ...);            /* musl, other UNIX */

DESCRIPTION
       The ioctl() system call manipulates the underlying device parameters of
       special files.  In particular, many operating characteristics of
       character special files (e.g., terminals) may be controlled with
       ioctl() operations.  The argument fd must be an open file descriptor.

       The second argument is a device-dependent operation code.  The third
       argument is an untyped pointer to memory.  It's traditionally char
       *argp (from the days before void * was valid C), and will be so named
       for this discussion.

       An ioctl() op has encoded in it whether the argument is an in parameter
       or out parameter, and the size of the argument argp in bytes.  Macros
       and defines used in specifying an ioctl() op are located in the file
       <sys/ioctl.h>.  See NOTES.

RETURN VALUE
       Usually, on success zero is returned.  A few ioctl() operations use the
       return value as an output parameter and return a nonnegative value on
       success.  On error, -1 is returned, and errno is set to indicate the
       error.

ERRORS
       EBADF  fd is not a valid file descriptor.

       EFAULT argp references an inaccessible memory area.

       EINVAL op or argp is not valid.

       ENOTTY fd is not associated with a character special device.
```

```
       ENOTTY The specified operation does not apply to the kind of object
              that the file descriptor fd references.

VERSIONS
       Arguments, returns, and semantics of ioctl() vary according to the
       device driver in question (the call is used as a catch-all for
       operations that don't cleanly fit the UNIX stream I/O model).

STANDARDS
       None.

HISTORY
       Version 7 AT&T UNIX has
           ioctl(int fildes, int op, struct sgttyb *argp);
       (where struct sgttyb has historically been used by stty(2) and gtty(2),
       and is polymorphic by operation type (like a void * would be, if it had
       been available)).

       SysIII documents arg without a type at all.

       4.3BSD has
           ioctl(int d, unsigned long op, char *argp);
       (with char * similarly in for void *).

       SysVr4 has
           int ioctl(int fildes, int op, ... /* arg */);
NOTES
       In order to use this call, one needs an open file descriptor.  Often
       the open(2) call has unwanted side effects, that can be avoided under
       Linux by giving it the O_NONBLOCK flag.

   ioctl structure
       Ioctl op values are 32-bit constants.  In principle these constants are
       completely arbitrary, but people have tried to build some structure
       into them.

       The old Linux situation was that of mostly 16-bit constants, where the
       last byte is a serial number, and the preceding byte(s) give a type
       indicating the driver.  Sometimes the major number was used: 0x03 for
       the HDIO_* ioctls, 0x06 for the LP* ioctls.  And sometimes one or more
       ASCII letters were used.  For example, TCGETS has value 0x00005401,
       with 0x54 = 'T' indicating the terminal driver, and CYGETTIMEOUT has
       value 0x00435906, with 0x43 0x59 = 'C' 'Y' indicating the cyclades
       driver.

       Later (0.98p5) some more information was built into the number.  One
       has 2 direction bits (00: none, 01: write, 10: read, 11: read/write)
       followed by 14 size bits (giving the size of the argument), followed by
       an 8-bit type (collecting the ioctls in groups for a common purpose or
       a common driver), and an 8-bit serial number.

       The macros describing this structure live in <asm/ioctl.h> and are
       _IO(type,nr) and {_IOR,_IOW,_IOWR}(type,nr,size).  They use
       sizeof(size) so that size is a misnomer here: this third argument is a
       data type.
```

```
       Note that the size bits are very unreliable: in lots of cases they are
       wrong, either because of buggy macros using sizeof(sizeof(struct)), or
       because of legacy values.

       Thus, it seems that the new structure only gave disadvantages: it does
       not help in checking, but it causes varying values for the various
       architectures.

SEE ALSO
       execve(2), fcntl(2), ioctl_console(2), ioctl_fat(2), ioctl_ficlone(2),
       ioctl_ficlonerange(2), ioctl_fideduperange(2), ioctl_fslabel(2),
       ioctl_getfsmap(2), ioctl_iflags(2), ioctl_ns(2), ioctl_tty(2),
       ioctl_userfaultfd(2), open(2), sd(4), tty(4)

Linux man-pages 6.7              2024-03-03                          ioctl(2)
```

TTY ioctl requests

```
ioctl_tty(2)                 System Calls Manual                ioctl_tty(2)

NAME
       ioctl_tty - ioctls for terminals and serial lines

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/ioctl.h>
       #include <asm/termbits.h>   /* Definition of struct termios,
                                      struct termios2, and
                                      Bnnn, BOTHER, CBAUD, CLOCAL,
                                      TC*{FLUSH,ON,OFF} and other constants */

       int ioctl(int fd, int op, ...);

DESCRIPTION
       The ioctl(2) call for terminals and serial ports accepts many possible
       operation arguments.  Most require a third argument, of varying type,
       here called argp or arg.

       Use of ioctl() makes for nonportable programs.  Use the POSIX interface
       described in termios(3) whenever possible.

       Please note that struct termios from <asm/termbits.h> is different and
       incompatible with struct termios from <termios.h>.  These ioctl calls
       require struct termios from <asm/termbits.h>.

   Get and set terminal attributes
       TCGETS Argument: struct termios *argp

              Equivalent to tcgetattr(fd, argp).
```

```
              Get the current serial port settings.

       TCSETS Argument: const struct termios *argp

              Equivalent to tcsetattr(fd, TCSANOW, argp).

              Set the current serial port settings.

       TCSETSW
              Argument: const struct termios *argp

              Equivalent to tcsetattr(fd, TCSADRAIN, argp).

              Allow the output buffer to drain, and set the current serial
              port settings.

       TCSETSF
              Argument: const struct termios *argp

              Equivalent to tcsetattr(fd, TCSAFLUSH, argp).

              Allow the output buffer to drain, discard pending input, and set
              the current serial port settings.

       The following four ioctls, added in Linux 2.6.20, are just like TCGETS,
       TCSETS, TCSETSW, TCSETSF, except that they take a struct termios2 *
       instead of a struct termios *.  If the structure member c_cflag
       contains the flag BOTHER, then the baud rate is stored in the structure
       members c_ispeed and c_ospeed as integer values.  These ioctls are not
       supported on all architectures.
              TCGETS2    struct termios2 *argp
              TCSETS2    const struct termios2 *argp
              TCSETSW2   const struct termios2 *argp
              TCSETSF2   const struct termios2 *argp

       The following four ioctls are just like TCGETS, TCSETS, TCSETSW,
       TCSETSF, except that they take a struct termio * instead of a struct
       termios *.
              TCGETA    struct termio *argp
              TCSETA    const struct termio *argp
              TCSETAW   const struct termio *argp
              TCSETAF   const struct termio *argp

   Locking the termios structure
       The termios structure of a terminal can be locked.  The lock is itself
       a termios structure, with nonzero bits or fields indicating a locked
       value.

       TIOCGLCKTRMIOS
              Argument: struct termios *argp

              Gets the locking status of the termios structure of the
              terminal.

       TIOCSLCKTRMIOS
```

```
            Argument: const struct termios *argp

            Sets the locking status of the termios structure of the
            terminal.  Only a process with the CAP_SYS_ADMIN capability can
            do this.

Get and set window size
    Window sizes are kept in the kernel, but not used by the kernel (except
    in the case of virtual consoles, where the kernel will update the
    window size when the size of the virtual console changes, for example,
    by loading a new font).

    TIOCGWINSZ
           Argument: struct winsize *argp

           Get window size.

    TIOCSWINSZ
           Argument: const struct winsize *argp

           Set window size.

    The struct used by these ioctls is defined as

        struct winsize {
            unsigned short ws_row;
            unsigned short ws_col;
            unsigned short ws_xpixel;   /* unused */
            unsigned short ws_ypixel;   /* unused */
        };

    When the window size changes, a SIGWINCH signal is sent to the
    foreground process group.

Sending a break
    TCSBRK Argument: int arg

           Equivalent to tcsendbreak(fd, arg).

           If the terminal is using asynchronous serial data transmission,
           and arg is zero, then send a break (a stream of zero bits) for
           between 0.25 and 0.5 seconds.  If the terminal is not using
           asynchronous serial data transmission, then either a break is
           sent, or the function returns without doing anything.  When arg
           is nonzero, nobody knows what will happen.

           (SVr4, UnixWare, Solaris, and Linux treat tcsendbreak(fd,arg)
           with nonzero arg like tcdrain(fd).  SunOS treats arg as a
           multiplier, and sends a stream of bits arg times as long as done
           for zero arg.  DG/UX and AIX treat arg (when nonzero) as a time
           interval measured in milliseconds.  HP-UX ignores arg.)

    TCSBRKP
           Argument: int arg
```

```
              So-called "POSIX version" of TCSBRK.  It treats nonzero arg as a
              time interval measured in deciseconds, and does nothing when the
              driver does not support breaks.

       TIOCSBRK
              Argument: void

              Turn break on, that is, start sending zero bits.

       TIOCCBRK
              Argument: void

              Turn break off, that is, stop sending zero bits.

   Software flow control
       TCXONC Argument: int arg

              Equivalent to tcflow(fd, arg).

              See tcflow(3) for the argument values TCOOFF, TCOON, TCIOFF,
              TCION.

   Buffer count and flushing
       FIONREAD
              Argument: int *argp

              Get the number of bytes in the input buffer.

       TIOCINQ
              Argument: int *argp

              Same as FIONREAD.

       TIOCOUTQ
              Argument: int *argp

              Get the number of bytes in the output buffer.

       TCFLSH Argument: int arg

              Equivalent to tcflush(fd, arg).

              See tcflush(3) for the argument values TCIFLUSH, TCOFLUSH,
              TCIOFLUSH.

       TIOCSERGETLSR
              Argument: int *argp

              Get line status register.  Status register has TIOCSER_TEMT bit
              set when output buffer is empty and also hardware transmitter is
              physically empty.

              Does not have to be supported by all serial tty drivers.

              tcdrain(3) does not wait and returns immediately when
```

```
            TIOCSER_TEMT bit is set.

Faking input
    TIOCSTI
            Argument: const char *argp

            Insert the given byte in the input queue.

            Since Linux 6.2, this operation may require the CAP_SYS_ADMIN
            capability (if the dev.tty.legacy_tiocsti sysctl variable is set
            to false).

Redirecting console output
    TIOCCONS
            Argument: void

            Redirect output that would have gone to /dev/console or
            /dev/tty0 to the given terminal.  If that was a pseudoterminal
            master, send it to the slave.  Before Linux 2.6.10, anybody can
            do this as long as the output was not redirected yet; since
            Linux 2.6.10, only a process with the CAP_SYS_ADMIN capability
            may do this.  If output was redirected already, then EBUSY is
            returned, but redirection can be stopped by using this ioctl
            with fd pointing at /dev/console or /dev/tty0.

Controlling terminal
    TIOCSCTTY
            Argument: int arg

            Make the given terminal the controlling terminal of the calling
            process.  The calling process must be a session leader and not
            have a controlling terminal already.  For this case, arg should
            be specified as zero.

            If this terminal is already the controlling terminal of a
            different session group, then the ioctl fails with EPERM, unless
            the caller has the CAP_SYS_ADMIN capability and arg equals 1, in
            which case the terminal is stolen, and all processes that had it
            as controlling terminal lose it.

    TIOCNOTTY
            Argument: void

            If the given terminal was the controlling terminal of the
            calling process, give up this controlling terminal.  If the
            process was session leader, then send SIGHUP and SIGCONT to the
            foreground process group and all processes in the current
            session lose their controlling terminal.

Process group and session ID
    TIOCGPGRP
            Argument: pid_t *argp

            When successful, equivalent to *argp = tcgetpgrp(fd).
```

```
              Get the process group ID of the foreground process group on this
              terminal.

       TIOCSPGRP
              Argument: const pid_t *argp

              Equivalent to tcsetpgrp(fd, *argp).

              Set the foreground process group ID of this terminal.

       TIOCGSID
              Argument: pid_t *argp

              When successful, equivalent to *argp = tcgetsid(fd).

              Get the session ID of the given terminal.  This fails with the
              error ENOTTY if the terminal is not a master pseudoterminal and
              not our controlling terminal.  Strange.

   Exclusive mode
       TIOCEXCL
              Argument: void

              Put the terminal into exclusive mode.  No further open(2)
              operations on the terminal are permitted.  (They fail with
              EBUSY, except for a process with the CAP_SYS_ADMIN capability.)

       TIOCGEXCL
              Argument: int *argp

              (since Linux 3.8) If the terminal is currently in exclusive
              mode, place a nonzero value in the location pointed to by argp;
              otherwise, place zero in *argp.

       TIOCNXCL
              Argument: void

              Disable exclusive mode.

   Line discipline
       TIOCGETD
              Argument: int *argp

              Get the line discipline of the terminal.

       TIOCSETD
              Argument: const int *argp

              Set the line discipline of the terminal.

   Pseudoterminal ioctls
       TIOCPKT
              Argument: const int *argp

              Enable (when *argp is nonzero) or disable packet mode.  Can be
```

```
       applied to the master side of a pseudoterminal only (and will
       return ENOTTY otherwise).  In packet mode, each subsequent
       read(2) will return a packet that either contains a single
       nonzero control byte, or has a single byte containing zero
       ('\0') followed by data written on the slave side of the
       pseudoterminal.  If the first byte is not TIOCPKT_DATA (0), it
       is an OR of one or more of the following bits:

       TIOCPKT_FLUSHREAD    The read queue for the
                            terminal is flushed.
       TIOCPKT_FLUSHWRITE   The write queue for the
                            terminal is flushed.
       TIOCPKT_STOP         Output to the terminal is
                            stopped.
       TIOCPKT_START        Output to the terminal is
                            restarted.
       TIOCPKT_DOSTOP       The start and stop
                            characters are ^S/^Q.
       TIOCPKT_NOSTOP       The start and stop
                            characters are not ^S/^Q.

       While  packet  mode  is  in  use, the presence of control status
       information to be read from the master side may be detected by a
       select(2) for  exceptional  conditions  or  a  poll(2)  for  the
       POLLPRI event.

       This  mode  is  used  by rlogin(1) and rlogind(8) to implement a
       remote-echoed, locally ^S/^Q flow-controlled remote login.

TIOCGPKT
       Argument: const int *argp

       (since Linux 3.8) Return the current packet mode setting in the
       integer pointed to by argp.

TIOCSPTLCK
       Argument: int *argp

       Set (if *argp is nonzero) or remove (if *argp is zero) the lock
       on the pseudoterminal slave device.  (See also unlockpt(3).)

TIOCGPTLCK
       Argument: int *argp

       (since Linux 3.8) Place the current lock state of the
       pseudoterminal slave device in the location pointed to by argp.

TIOCGPTPEER
       Argument: int flags

       (since Linux 4.13) Given a file descriptor in fd that refers to
       a pseudoterminal master, open (with the given open(2)-style
       flags) and return a new file descriptor that refers to the peer
       pseudoterminal slave device.  This operation can be performed
       regardless of whether the pathname of the slave device is
```

```
              accessible through the calling process's mount namespace.

              Security-conscious programs interacting with namespaces may wish
              to use this operation rather than open(2) with the pathname
              returned by ptsname(3), and similar library functions that have
              insecure APIs.  (For example, confusion can occur in some cases
              using ptsname(3) with a pathname where a devpts filesystem has
              been mounted in a different mount namespace.)

       The BSD ioctls TIOCSTOP, TIOCSTART, TIOCUCNTL, and TIOCREMOTE have not
       been implemented under Linux.

   Modem control
       TIOCMGET
              Argument: int *argp

              Get the status of modem bits.

       TIOCMSET
              Argument: const int *argp

              Set the status of modem bits.

       TIOCMBIC
              Argument: const int *argp

              Clear the indicated modem bits.

       TIOCMBIS
              Argument: const int *argp

              Set the indicated modem bits.

       The following bits are used by the above ioctls:

       TIOCM_LE    DSR (data set ready/line enable)
       TIOCM_DTR   DTR (data terminal ready)
       TIOCM_RTS   RTS (request to send)
       TIOCM_ST    Secondary TXD (transmit)
       TIOCM_SR    Secondary RXD (receive)
       TIOCM_CTS   CTS (clear to send)
       TIOCM_CAR   DCD (data carrier detect)
       TIOCM_CD    see TIOCM_CAR
       TIOCM_RNG   RNG (ring)
       TIOCM_RI    see TIOCM_RNG
       TIOCM_DSR   DSR (data set ready)

       TIOCMIWAIT
              Argument: int arg

              Wait for any of the 4 modem bits (DCD, RI, DSR, CTS) to change.
              The bits of interest are specified as a bit mask in arg, by
              ORing together any of the bit values, TIOCM_RNG, TIOCM_DSR,
              TIOCM_CD, and TIOCM_CTS.  The caller should use TIOCGICOUNT to
              see which bit has changed.
```

```
       TIOCGICOUNT
              Argument: struct serial_icounter_struct *argp

              Get counts of input serial line interrupts (DCD, RI, DSR, CTS).
              The counts are written to the serial_icounter_struct structure
              pointed to by argp.

              Note: both 1->0 and 0->1 transitions are counted, except for RI,
              where only 0->1 transitions are counted.

   Marking a line as local
       TIOCGSOFTCAR
              Argument: int *argp

              ("Get software carrier flag") Get the status of the CLOCAL flag
              in the c_cflag field of the termios structure.

       TIOCSSOFTCAR
              Argument: const int *argp

              ("Set software carrier flag") Set the CLOCAL flag in the termios
              structure when *argp is nonzero, and clear it otherwise.

       If the CLOCAL flag for a line is off, the hardware carrier detect (DCD)
       signal is significant, and an open(2) of the corresponding terminal
       will block until DCD is asserted, unless the O_NONBLOCK flag is given.
       If CLOCAL is set, the line behaves as if DCD is always asserted.  The
       software carrier flag is usually turned on for local devices, and is
       off for lines with modems.

   Linux-specific
       For the TIOCLINUX ioctl, see ioctl_console(2).

   Kernel debugging
       #include <linux/tty.h>

       TIOCTTYGSTRUCT
              Argument: struct tty_struct *argp

              Get the tty_struct corresponding to fd.  This operation was
              removed in Linux 2.5.67.

RETURN VALUE
       The ioctl(2) system call returns 0 on success.  On error, it returns -1
       and sets errno to indicate the error.

ERRORS
       EINVAL Invalid operation parameter.

       ENOIOCTLCMD
              Unknown operation.

       ENOTTY Inappropriate fd.
```

```
       EPERM  Insufficient permission.

EXAMPLES
       Check the condition of DTR on the serial port.

       #include <fcntl.h>
       #include <stdio.h>
       #include <sys/ioctl.h>
       #include <unistd.h>

       int
       main(void)
       {
           int fd, serial;

           fd = open("/dev/ttyS0", O_RDONLY);
           ioctl(fd, TIOCMGET, &serial);
           if (serial & TIOCM_DTR)
               puts("TIOCM_DTR is set");
           else
               puts("TIOCM_DTR is not set");
           close(fd);
       }

       Get or set arbitrary baudrate on the serial port.

       /* SPDX-License-Identifier: GPL-2.0-or-later */

       #include <asm/termbits.h>
       #include <fcntl.h>
       #include <stdio.h>
       #include <stdlib.h>
       #include <sys/ioctl.h>
       #include <unistd.h>

       int
       main(int argc, char *argv[])
       {
       #if !defined BOTHER
           fprintf(stderr, "BOTHER is unsupported\n");
           /* Program may fallback to TCGETS/TCSETS with Bnnn constants */
           exit(EXIT_FAILURE);
       #else
           /* Declare tio structure, its type depends on supported ioctl */
       # if defined TCGETS2
           struct termios2 tio;
       # else
           struct termios tio;
       # endif
           int fd, rc;

           if (argc != 2 && argc != 3 && argc != 4) {
               fprintf(stderr, "Usage: %s device [output [input] ]\n", argv[0]);
               exit(EXIT_FAILURE);
           }
```

```
    fd = open(argv[1], O_RDWR | O_NONBLOCK | O_NOCTTY);
    if (fd < 0) {
        perror("open");
        exit(EXIT_FAILURE);
    }

    /* Get the current serial port settings via supported ioctl */
# if defined TCGETS2
    rc = ioctl(fd, TCGETS2, &tio);
# else
    rc = ioctl(fd, TCGETS, &tio);
# endif
    if (rc) {
        perror("TCGETS");
        close(fd);
        exit(EXIT_FAILURE);
    }

    /* Change baud rate when more arguments were provided */
    if (argc == 3 || argc == 4) {
        /* Clear the current output baud rate and fill a new value */
        tio.c_cflag &= ~CBAUD;
        tio.c_cflag |= BOTHER;
        tio.c_ospeed = atoi(argv[2]);

        /* Clear the current input baud rate and fill a new value */
        tio.c_cflag &= ~(CBAUD << IBSHIFT);
        tio.c_cflag |= BOTHER << IBSHIFT;
        /* When 4th argument is not provided reuse output baud rate */
        tio.c_ispeed = (argc == 4) ? atoi(argv[3]) : atoi(argv[2]);

        /* Set new serial port settings via supported ioctl */
# if defined TCSETS2
        rc = ioctl(fd, TCSETS2, &tio);
# else
        rc = ioctl(fd, TCSETS, &tio);
# endif
        if (rc) {
            perror("TCSETS");
            close(fd);
            exit(EXIT_FAILURE);
        }

        /* And get new values which were really configured */
# if defined TCGETS2
        rc = ioctl(fd, TCGETS2, &tio);
# else
        rc = ioctl(fd, TCGETS, &tio);
# endif
        if (rc) {
            perror("TCGETS");
            close(fd);
            exit(EXIT_FAILURE);
        }
```

```
            }

            close(fd);

            printf("output baud rate: %u\n", tio.c_ospeed);
            printf("input baud rate: %u\n", tio.c_ispeed);

            exit(EXIT_SUCCESS);
        #endif
        }

SEE ALSO
        ldattach(8), ioctl(2), ioctl_console(2), termios(3), pty(7)

Linux man-pages 6.7              2024-03-03                      ioctl_tty(2)
```

---

termios(3)

```
termios(3)                Library Functions Manual                termios(3)

NAME
        termios, tcgetattr, tcsetattr, tcsendbreak, tcdrain, tcflush, tcflow,
        cfmakeraw, cfgetospeed, cfgetispeed, cfsetispeed, cfsetospeed,
        cfsetspeed - get and set terminal attributes, line control, get and set
        baud rate

LIBRARY
        Standard C library (libc, -lc)

SYNOPSIS
        #include <termios.h>
        #include <unistd.h>

        int tcgetattr(int fd, struct termios *termios_p);
        int tcsetattr(int fd, int optional_actions,
                      const struct termios *termios_p);

        int tcsendbreak(int fd, int duration);
        int tcdrain(int fd);
        int tcflush(int fd, int queue_selector);
        int tcflow(int fd, int action);

        void cfmakeraw(struct termios *termios_p);

        speed_t cfgetispeed(const struct termios *termios_p);
        speed_t cfgetospeed(const struct termios *termios_p);

        int cfsetispeed(struct termios *termios_p, speed_t speed);
        int cfsetospeed(struct termios *termios_p, speed_t speed);
        int cfsetspeed(struct termios *termios_p, speed_t speed);

    Feature Test Macro Requirements for glibc (see feature_test_macros(7)):
```

```
       cfsetspeed(), cfmakeraw():
           Since glibc 2.19:
               _DEFAULT_SOURCE
           glibc 2.19 and earlier:
               _BSD_SOURCE

DESCRIPTION
       The termios functions describe a general terminal interface that is
       provided to control asynchronous communications ports.

   The termios structure
       Many of the functions described here have a termios_p argument that is
       a pointer to a termios structure.  This structure contains at least the
       following members:

           tcflag_t c_iflag;      /* input modes */
           tcflag_t c_oflag;      /* output modes */
           tcflag_t c_cflag;      /* control modes */
           tcflag_t c_lflag;      /* local modes */
           cc_t     c_cc[NCCS];   /* special characters */

       The values that may be assigned to these fields are described below.
       In the case of the first four bit-mask fields, the definitions of some
       of the associated flags that may be set are exposed only if a specific
       feature test macro (see feature_test_macros(7)) is defined, as noted in
       brackets ("[]").

       In the descriptions below, "not in POSIX" means that the value is not
       specified in POSIX.1-2001, and "XSI" means that the value is specified
       in POSIX.1-2001 as part of the XSI extension.

       c_iflag flag constants:

       IGNBRK Ignore BREAK condition on input.

       BRKINT If IGNBRK is set, a BREAK is ignored.  If it is not set but
              BRKINT is set, then a BREAK causes the input and output queues
              to be flushed, and if the terminal is the controlling terminal
              of a foreground process group, it will cause a SIGINT to be sent
              to this foreground process group.  When neither IGNBRK nor
              BRKINT are set, a BREAK reads as a null byte ('\0'), except when
              PARMRK is set, in which case it reads as the sequence \377 \0
              \0.

       IGNPAR Ignore framing errors and parity errors.

       PARMRK If this bit is set, input bytes with parity or framing errors
              are marked when passed to the program.  This bit is meaningful
              only when INPCK is set and IGNPAR is not set.  The way erroneous
              bytes are marked is with two preceding bytes, \377 and \0.
              Thus, the program actually reads three bytes for one erroneous
              byte received from the terminal.  If a valid byte has the value
              \377, and ISTRIP (see below) is not set, the program might
              confuse it with the prefix that marks a parity error.
```

```
       Therefore, a valid byte \377 is passed to the program as two
       bytes, \377 \377, in this case.

       If neither IGNPAR nor PARMRK is set, read a character with a
       parity error or framing error as \0.

INPCK  Enable input parity checking.

ISTRIP Strip off eighth bit.

INLCR  Translate NL to CR on input.

IGNCR  Ignore carriage return on input.

ICRNL  Translate carriage return to newline on input (unless IGNCR is
       set).

IUCLC  (not in POSIX) Map uppercase characters to lowercase on input.

IXON   Enable XON/XOFF flow control on output.

IXANY  (XSI) Typing any character will restart stopped output.  (The
       default is to allow just the START character to restart output.)

IXOFF  Enable XON/XOFF flow control on input.

IMAXBEL
       (not in POSIX) Ring bell when input queue is full.  Linux does
       not implement this bit, and acts as if it is always set.

IUTF8 (since Linux 2.6.4)
       (not in POSIX) Input is UTF8; this allows character-erase to be
       correctly performed in cooked mode.

c_oflag flag constants:

OPOST  Enable implementation-defined output processing.

OLCUC  (not in POSIX) Map lowercase characters to uppercase on output.

ONLCR  (XSI) Map NL to CR-NL on output.

OCRNL  Map CR to NL on output.

ONOCR  Don't output CR at column 0.

ONLRET The NL character is assumed to do the carriage-return function;
       the kernel's idea of the current column is set to 0 after both
       NL and CR.

OFILL  Send fill characters for a delay, rather than using a timed
       delay.

OFDEL  Fill character is ASCII DEL (0177).  If unset, fill character is
       ASCII NUL ('\0').  (Not implemented on Linux.)
```

```
       NLDLY  Newline delay mask.  Values are NL0 and NL1.  [requires
              _BSD_SOURCE or _SVID_SOURCE or _XOPEN_SOURCE]

       CRDLY  Carriage return delay mask.  Values are CR0, CR1, CR2, or CR3.
              [requires _BSD_SOURCE or _SVID_SOURCE or _XOPEN_SOURCE]

       TABDLY Horizontal tab delay mask.  Values are TAB0, TAB1, TAB2, TAB3
              (or XTABS, but see the BUGS section).  A value of TAB3, that is,
              XTABS, expands tabs to spaces (with tab stops every eight
              columns).  [requires _BSD_SOURCE or _SVID_SOURCE or
              _XOPEN_SOURCE]

       BSDLY  Backspace delay mask.  Values are BS0 or BS1.  (Has never been
              implemented.)  [requires _BSD_SOURCE or _SVID_SOURCE or
              _XOPEN_SOURCE]

       VTDLY  Vertical tab delay mask.  Values are VT0 or VT1.

       FFDLY  Form feed delay mask.  Values are FF0 or FF1.  [requires
              _BSD_SOURCE or _SVID_SOURCE or _XOPEN_SOURCE]

       c_cflag flag constants:

       CBAUD  (not in POSIX) Baud speed mask (4+1 bits).  [requires
              _BSD_SOURCE or _SVID_SOURCE]

       CBAUDEX
              (not in POSIX) Extra baud speed mask (1 bit), included in CBAUD.
              [requires _BSD_SOURCE or _SVID_SOURCE]

              (POSIX says that the baud speed is stored in the termios
              structure without specifying where precisely, and provides
              cfgetispeed() and cfsetispeed() for getting at it.  Some systems
              use bits selected by CBAUD in c_cflag, other systems use
              separate fields, for example, sg_ispeed and sg_ospeed.)

       CSIZE  Character size mask.  Values are CS5, CS6, CS7, or CS8.

       CSTOPB Set two stop bits, rather than one.

       CREAD  Enable receiver.

       PARENB Enable parity generation on output and parity checking for
              input.

       PARODD If set, then parity for input and output is odd; otherwise even
              parity is used.

       HUPCL  Lower modem control lines after last process closes the device
              (hang up).

       CLOCAL Ignore modem control lines.

       LOBLK  (not in POSIX) Block output from a noncurrent shell layer.  For
```

```
       use by shl (shell layers).  (Not implemented on Linux.)

   CIBAUD (not in POSIX) Mask for input speeds.  The values for the CIBAUD
       bits are the same as the values for the CBAUD bits, shifted left
       IBSHIFT bits.  [requires _BSD_SOURCE or _SVID_SOURCE] (Not
       implemented in glibc, supported on Linux via TCGET* and TCSET*
       ioctls; see ioctl_tty(2))

   CMSPAR (not in POSIX) Use "stick" (mark/space) parity (supported on
       certain serial devices): if PARODD is set, the parity bit is
       always 1; if PARODD is not set, then the parity bit is always 0.
       [requires _BSD_SOURCE or _SVID_SOURCE]

   CRTSCTS
       (not in POSIX) Enable RTS/CTS (hardware) flow control.
       [requires _BSD_SOURCE or _SVID_SOURCE]

   c_lflag flag constants:

   ISIG   When any of the characters INTR, QUIT, SUSP, or DSUSP are
       received, generate the corresponding signal.

   ICANON Enable canonical mode (described below).

   XCASE  (not in POSIX; not supported under Linux) If ICANON is also set,
       terminal is uppercase only.  Input is converted to lowercase,
       except for characters preceded by \.  On output, uppercase
       characters are preceded by \ and lowercase characters are
       converted to uppercase.  [requires _BSD_SOURCE or _SVID_SOURCE
       or _XOPEN_SOURCE]

   ECHO   Echo input characters.

   ECHOE  If ICANON is also set, the ERASE character erases the preceding
       input character, and WERASE erases the preceding word.

   ECHOK  If ICANON is also set, the KILL character erases the current
       line.

   ECHONL If ICANON is also set, echo the NL character even if ECHO is not
       set.

   ECHOCTL
       (not in POSIX) If ECHO is also set, terminal special characters
       other than TAB, NL, START, and STOP are echoed as ^X, where X is
       the character with ASCII code 0x40 greater than the special
       character.  For example, character 0x08 (BS) is echoed as ^H.
       [requires _BSD_SOURCE or _SVID_SOURCE]

   ECHOPRT
       (not in POSIX) If ICANON and ECHO are also set, characters are
       printed as they are being erased.  [requires _BSD_SOURCE or
       _SVID_SOURCE]

   ECHOKE (not in POSIX) If ICANON is also set, KILL is echoed by erasing
```

```
       each character on the line, as specified by ECHOE and ECHOPRT.
       [requires _BSD_SOURCE or _SVID_SOURCE]

   DEFECHO
       (not in POSIX) Echo only when a process is reading.  (Not
       implemented on Linux.)

   FLUSHO (not in POSIX; not supported under Linux) Output is being
       flushed.  This flag is toggled by typing the DISCARD character.
       [requires _BSD_SOURCE or _SVID_SOURCE]

   NOFLSH Disable flushing the input and output queues when generating
       signals for the INT, QUIT, and SUSP characters.

   TOSTOP Send the SIGTTOU signal to the process group of a background
       process which tries to write to its controlling terminal.

   PENDIN (not in POSIX; not supported under Linux) All characters in the
       input queue are reprinted when the next character is read.
       (bash(1) handles typeahead this way.)  [requires _BSD_SOURCE or
       _SVID_SOURCE]

   IEXTEN Enable implementation-defined input processing.  This flag, as
       well as ICANON must be enabled for the special characters EOL2,
       LNEXT, REPRINT, WERASE to be interpreted, and for the IUCLC flag
       to be effective.

   The c_cc array defines the terminal special characters.  The symbolic
   indices (initial values) and meaning are:

   VDISCARD
       (not in POSIX; not supported under Linux; 017, SI, Ctrl-O)
       Toggle: start/stop discarding pending output.  Recognized when
       IEXTEN is set, and then not passed as input.

   VDSUSP (not in POSIX; not supported under Linux; 031, EM, Ctrl-Y)
       Delayed suspend character (DSUSP): send SIGTSTP signal when the
       character is read by the user program.  Recognized when IEXTEN
       and ISIG are set, and the system supports job control, and then
       not passed as input.

   VEOF   (004, EOT, Ctrl-D) End-of-file character (EOF).  More precisely:
       this character causes the pending tty buffer to be sent to the
       waiting user program without waiting for end-of-line.  If it is
       the first character of the line, the read(2) in the user program
       returns 0, which signifies end-of-file.  Recognized when ICANON
       is set, and then not passed as input.

   VEOL   (0, NUL) Additional end-of-line character (EOL).  Recognized
       when ICANON is set.

   VEOL2  (not in POSIX; 0, NUL) Yet another end-of-line character (EOL2).
       Recognized when ICANON is set.

   VERASE (0177, DEL, rubout, or 010, BS, Ctrl-H, or also #) Erase
```

```
       character (ERASE).  This erases the previous not-yet-erased
       character, but does not erase past EOF or beginning-of-line.
       Recognized when ICANON is set, and then not passed as input.

VINTR  (003, ETX, Ctrl-C, or also 0177, DEL, rubout) Interrupt
       character (INTR).  Send a SIGINT signal.  Recognized when ISIG
       is set, and then not passed as input.

VKILL  (025, NAK, Ctrl-U, or Ctrl-X, or also @) Kill character (KILL).
       This erases the input since the last EOF or beginning-of-line.
       Recognized when ICANON is set, and then not passed as input.

VLNEXT (not in POSIX; 026, SYN, Ctrl-V) Literal next (LNEXT).  Quotes
       the next input character, depriving it of a possible special
       meaning.  Recognized when IEXTEN is set, and then not passed as
       input.

VMIN   Minimum number of characters for noncanonical read (MIN).

VQUIT  (034, FS, Ctrl-\) Quit character (QUIT).  Send SIGQUIT signal.
       Recognized when ISIG is set, and then not passed as input.

VREPRINT
       (not in POSIX; 022, DC2, Ctrl-R) Reprint unread characters
       (REPRINT).  Recognized when ICANON and IEXTEN are set, and then
       not passed as input.

VSTART (021, DC1, Ctrl-Q) Start character (START).  Restarts output
       stopped by the Stop character.  Recognized when IXON is set, and
       then not passed as input.

VSTATUS
       (not in POSIX; not supported under Linux; status request: 024,
       DC4, Ctrl-T).  Status character (STATUS).  Display status
       information at terminal, including state of foreground process
       and amount of CPU time it has consumed.  Also sends a SIGINFO
       signal (not supported on Linux) to the foreground process group.

VSTOP  (023, DC3, Ctrl-S) Stop character (STOP).  Stop output until
       Start character typed.  Recognized when IXON is set, and then
       not passed as input.

VSUSP  (032, SUB, Ctrl-Z) Suspend character (SUSP).  Send SIGTSTP
       signal.  Recognized when ISIG is set, and then not passed as
       input.

VSWTCH (not in POSIX; not supported under Linux; 0, NUL) Switch
       character (SWTCH).  Used in System V to switch shells in shell
       layers, a predecessor to shell job control.

VTIME  Timeout in deciseconds for noncanonical read (TIME).

VWERASE
       (not in POSIX; 027, ETB, Ctrl-W) Word erase (WERASE).
       Recognized when ICANON and IEXTEN are set, and then not passed
```

```
              as input.

       An individual terminal special character can be disabled by setting the
       value of the corresponding c_cc element to _POSIX_VDISABLE.

       The above symbolic subscript values are all different, except that
       VTIME, VMIN may have the same value as VEOL, VEOF, respectively.  In
       noncanonical mode the special character meaning is replaced by the
       timeout meaning.  For an explanation of VMIN and VTIME, see the
       description of noncanonical mode below.

   Retrieving and changing terminal settings
       tcgetattr() gets the parameters associated with the object referred by
       fd and stores them in the termios structure referenced by termios_p.
       This function may be invoked from a background process; however, the
       terminal attributes may be subsequently changed by a foreground
       process.

       tcsetattr() sets the parameters associated with the terminal (unless
       support is required from the underlying hardware that is not available)
       from the termios structure referred to by termios_p.  optional_actions
       specifies when the changes take effect:

       TCSANOW
              the change occurs immediately.

       TCSADRAIN
              the change occurs after all output written to fd has been
              transmitted.  This option should be used when changing
              parameters that affect output.

       TCSAFLUSH
              the change occurs after all output written to the object
              referred by fd has been transmitted, and all input that has been
              received but not read will be discarded before the change is
              made.

   Canonical and noncanonical mode
       The setting of the ICANON canon flag in c_lflag determines whether the
       terminal is operating in canonical mode (ICANON set) or noncanonical
       mode (ICANON unset).  By default, ICANON is set.

       In canonical mode:

       • Input is made available line by line.  An input line is available
         when one of the line delimiters is typed (NL, EOL, EOL2; or EOF at
         the start of line).  Except in the case of EOF, the line delimiter
         is included in the buffer returned by read(2).

       • Line editing is enabled (ERASE, KILL; and if the IEXTEN flag is set:
         WERASE, REPRINT, LNEXT).  A read(2) returns at most one line of
         input; if the read(2) requested fewer bytes than are available in
         the current line of input, then only as many bytes as requested are
         read, and the remaining characters will be available for a future
         read(2).
```

```
    • The maximum line length is 4096 chars (including the terminating
      newline character); lines longer than 4096 chars are truncated.
      After 4095 characters, input processing (e.g., ISIG and ECHO*
      processing) continues, but any input data after 4095 characters up
      to (but not including) any terminating newline is discarded.  This
      ensures that the terminal can always receive more input until at
      least one line can be read.

In noncanonical mode input is available immediately (without the user
having to type a line-delimiter character), no input processing is
performed, and line editing is disabled.  The read buffer will only
accept 4095 chars; this provides the necessary space for a newline char
if the input mode is switched to canonical.  The settings of MIN
(c_cc[VMIN]) and TIME (c_cc[VTIME]) determine the circumstances in
which a read(2) completes; there are four distinct cases:

MIN == 0, TIME == 0 (polling read)
       If data is available, read(2) returns immediately, with the
       lesser of the number of bytes available, or the number of bytes
       requested.  If no data is available, read(2) returns 0.

MIN > 0, TIME == 0 (blocking read)
       read(2) blocks until MIN bytes are available, and returns up to
       the number of bytes requested.

MIN == 0, TIME > 0 (read with timeout)
       TIME specifies the limit for a timer in tenths of a second.  The
       timer is started when read(2) is called.  read(2) returns either
       when at least one byte of data is available, or when the timer
       expires.  If the timer expires without any input becoming
       available, read(2) returns 0.  If data is already available at
       the time of the call to read(2), the call behaves as though the
       data was received immediately after the call.

MIN > 0, TIME > 0 (read with interbyte timeout)
       TIME specifies the limit for a timer in tenths of a second.
       Once an initial byte of input becomes available, the timer is
       restarted after each further byte is received.  read(2) returns
       when any of the following conditions is met:

       • MIN bytes have been received.

       • The interbyte timer expires.

       • The number of bytes requested by read(2) has been received.
         (POSIX does not specify this termination condition, and on
         some other implementations read(2) does not return in this
         case.)

       Because the timer is started only after the initial byte becomes
       available, at least one byte will be read.  If data is already
       available at the time of the call to read(2), the call behaves
       as though the data was received immediately after the call.
```

```
       POSIX does not specify whether the setting of the O_NONBLOCK file
       status flag takes precedence over the MIN and TIME settings.  If
       O_NONBLOCK is set, a read(2) in noncanonical mode may return
       immediately, regardless of the setting of MIN or TIME.  Furthermore, if
       no data is available, POSIX permits a read(2) in noncanonical mode to
       return either 0, or -1 with errno set to EAGAIN.

   Raw mode
       cfmakeraw() sets the terminal to something like the "raw" mode of the
       old Version 7 terminal driver: input is available character by
       character, echoing is disabled, and all special processing of terminal
       input and output characters is disabled.  The terminal attributes are
       set as follows:

           termios_p->c_iflag &= ~(IGNBRK | BRKINT | PARMRK | ISTRIP
                           | INLCR | IGNCR | ICRNL | IXON);
           termios_p->c_oflag &= ~OPOST;
           termios_p->c_lflag &= ~(ECHO | ECHONL | ICANON | ISIG | IEXTEN);
           termios_p->c_cflag &= ~(CSIZE | PARENB);
           termios_p->c_cflag |= CS8;

   Line control
       tcsendbreak() transmits a continuous stream of zero-valued bits for a
       specific duration, if the terminal is using asynchronous serial data
       transmission.  If duration is zero, it transmits zero-valued bits for
       at least 0.25 seconds, and not more than 0.5 seconds.  If duration is
       not zero, it sends zero-valued bits for some implementation-defined
       length of time.

       If the terminal is not using asynchronous serial data transmission,
       tcsendbreak() returns without taking any action.

       tcdrain() waits until all output written to the object referred to by
       fd has been transmitted.

       tcflush() discards data written to the object referred to by fd but not
       transmitted, or data received but not read, depending on the value of
       queue_selector:

       TCIFLUSH
              flushes data received but not read.

       TCOFLUSH
              flushes data written but not transmitted.

       TCIOFLUSH
              flushes both data received but not read, and data written but
              not transmitted.

       tcflow() suspends transmission or reception of data on the object
       referred to by fd, depending on the value of action:

       TCOOFF suspends output.

       TCOON  restarts suspended output.
```

```
      TCIOFF transmits a STOP character, which stops the terminal device from
             transmitting data to the system.

      TCION  transmits a START character, which starts the terminal device
             transmitting data to the system.

      The default on open of a terminal file is that neither its input nor
      its output is suspended.

  Line speed
      The baud rate functions are provided for getting and setting the values
      of the input and output baud rates in the termios structure.  The new
      values do not take effect until tcsetattr() is successfully called.

      Setting the speed to B0 instructs the modem to "hang up".  The actual
      bit rate corresponding to B38400 may be altered with setserial(8).

      The input and output baud rates are stored in the termios structure.

      cfgetospeed() returns the output baud rate stored in the termios
      structure pointed to by termios_p.

      cfsetospeed() sets the output baud rate stored in the termios structure
      pointed to by termios_p to speed, which must be one of these constants:

             B0
             B50
             B75
             B110
             B134
             B150
             B200
             B300
             B600
             B1200
             B1800
             B2400
             B4800
             B9600
             B19200
             B38400
             B57600
             B115200
             B230400
             B460800
             B500000
             B576000
             B921600
             B1000000
             B1152000
             B1500000
             B2000000

      These constants are additionally supported on the SPARC architecture:
```

```
              B76800
              B153600
              B307200
              B614400

       These constants are additionally supported on non-SPARC architectures:

              B2500000
              B3000000
              B3500000
              B4000000

       Due to differences between architectures, portable applications should
       check if a particular Bnnn constant is defined prior to using it.

       The zero baud rate, B0, is used to terminate the connection.  If B0 is
       specified, the modem control lines shall no longer be asserted.
       Normally, this will disconnect the line.  CBAUDEX is a mask for the
       speeds beyond those defined in POSIX.1 (57600 and above).  Thus, B57600
       & CBAUDEX is nonzero.

       Setting the baud rate to a value other than those defined by Bnnn
       constants is possible via the TCSETS2 ioctl; see ioctl_tty(2).

       cfgetispeed() returns the input baud rate stored in the termios
       structure.

       cfsetispeed() sets the input baud rate stored in the termios structure
       to speed, which must be specified as one of the Bnnn constants listed
       above for cfsetospeed().  If the input baud rate is set to the literal
       constant 0 (not the symbolic constant B0), the input baud rate will be
       equal to the output baud rate.

       cfsetspeed() is a 4.4BSD extension.  It takes the same arguments as
       cfsetispeed(), and sets both input and output speed.

RETURN VALUE
       cfgetispeed() returns the input baud rate stored in the termios
       structure.

       cfgetospeed() returns the output baud rate stored in the termios
       structure.

       All other functions return:

       0      on success.

       -1     on failure and set errno to indicate the error.

       Note that tcsetattr() returns success if any of the requested changes
       could be successfully carried out.  Therefore, when making multiple
       changes it may be necessary to follow this call with a further call to
       tcgetattr() to check that all changes have been performed successfully.
```

```
ATTRIBUTES
       For an explanation of the terms used in this section, see
       attributes(7).

        Interface                                  Attribute       Value

        tcgetattr(), tcsetattr(), tcdrain(),       Thread safety   MT-Safe
        tcflush(), tcflow(), tcsendbreak(),
        cfmakeraw(), cfgetispeed(),
        cfgetospeed(), cfsetispeed(),
        cfsetospeed(), cfsetspeed()


STANDARDS
       tcgetattr()
       tcsetattr()
       tcsendbreak()
       tcdrain()
       tcflush()
       tcflow()
       cfgetispeed()
       cfgetospeed()
       cfsetispeed()
       cfsetospeed()
              POSIX.1-2008.

       cfmakeraw()
       cfsetspeed()
              BSD.

HISTORY
       tcgetattr()
       tcsetattr()
       tcsendbreak()
       tcdrain()
       tcflush()
       tcflow()
       cfgetispeed()
       cfgetospeed()
       cfsetispeed()
       cfsetospeed()
              POSIX.1-2001.

       cfmakeraw()
       cfsetspeed()
              BSD.

NOTES
       UNIX V7 and several later systems have a list of baud rates where after
       the values B0 through B9600 one finds the two constants EXTA, EXTB
       ("External A" and "External B").  Many systems extend the list with
       much higher baud rates.

       The effect of a nonzero duration with tcsendbreak() varies.  SunOS
       specifies a break of duration * N seconds, where N is at least 0.25,
```

```
       and not more than 0.5.  Linux, AIX, DU, Tru64 send a break of duration
       milliseconds.  FreeBSD and NetBSD and HP-UX and MacOS ignore the value
       of duration.  Under Solaris and UnixWare, tcsendbreak() with nonzero
       duration behaves like tcdrain().

BUGS
       On the Alpha architecture before Linux 4.16 (and glibc before glibc
       2.28), the XTABS value was different from TAB3 and it was ignored by
       the N_TTY line discipline code of the terminal driver as a result
       (because as it wasn't part of the TABDLY mask).

SEE ALSO
       reset(1), setterm(1), stty(1), tput(1), tset(1), tty(1),
       ioctl_console(2), ioctl_tty(2), cc_t(3type), speed_t(3type),
       tcflag_t(3type), setserial(8)

Linux man-pages 6.7              2023-10-31                        termios(3)
```

---

poll(2)

```
poll(2)                       System Calls Manual                      poll(2)

NAME
       poll, ppoll - wait for some event on a file descriptor

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <poll.h>

       int poll(struct pollfd *fds, nfds_t nfds, int timeout);

       #define _GNU_SOURCE         /* See feature_test_macros(7) */
       #include <poll.h>

       int ppoll(struct pollfd *fds, nfds_t nfds,
                 const struct timespec *_Nullable tmo_p,
                 const sigset_t *_Nullable sigmask);

DESCRIPTION
       poll() performs a similar task to select(2): it waits for one of a set
       of file descriptors to become ready to perform I/O.  The Linux-specific
       epoll(7) API performs a similar task, but offers features beyond those
       found in poll().

       The set of file descriptors to be monitored is specified in the fds
       argument, which is an array of structures of the following form:

           struct pollfd {
               int   fd;         /* file descriptor */
               short events;     /* requested events */
```

```
        short revents;    /* returned events */
    };
```

The caller should specify the number of items in the fds array in nfds.

The field fd contains a file descriptor for an open file. If this field is negative, then the corresponding events field is ignored and the revents field returns zero. (This provides an easy way of ignoring a file descriptor for a single poll() call: simply set the fd field to its bitwise complement.)

The field events is an input parameter, a bit mask specifying the events the application is interested in for the file descriptor fd. This field may be specified as zero, in which case the only events that can be returned in revents are POLLHUP, POLLERR, and POLLNVAL (see below).

The field revents is an output parameter, filled by the kernel with the events that actually occurred. The bits returned in revents can include any of those specified in events, or one of the values POLLERR, POLLHUP, or POLLNVAL. (These three bits are meaningless in the events field, and will be set in the revents field whenever the corresponding condition is true.)

If none of the events requested (and no error) has occurred for any of the file descriptors, then poll() blocks until one of the events occurs.

The timeout argument specifies the number of milliseconds that poll() should block waiting for a file descriptor to become ready. The call will block until either:

- a file descriptor becomes ready;
- the call is interrupted by a signal handler; or
- the timeout expires.

Being "ready" means that the requested operation will not block; thus, poll()ing regular files, block devices, and other files with no reasonable polling semantic always returns instantly as ready to read and write.

Note that the timeout interval will be rounded up to the system clock granularity, and kernel scheduling delays mean that the blocking interval may overrun by a small amount. Specifying a negative value in timeout means an infinite timeout. Specifying a timeout of zero causes poll() to return immediately, even if no file descriptors are ready.

The bits that may be set/returned in events and revents are defined in <poll.h>:

POLLIN There is data to read.

POLLPRI

```
              There is some exceptional condition on the file descriptor.
              Possibilities include:

              • There is out-of-band data on a TCP socket (see tcp(7)).

              • A pseudoterminal master in packet mode has seen a state
                change on the slave (see ioctl_tty(2)).

              • A cgroup.events file has been modified (see cgroups(7)).

       POLLOUT
              Writing is now possible, though a write larger than the
              available space in a socket or pipe will still block (unless
              O_NONBLOCK is set).

       POLLRDHUP (since Linux 2.6.17)
              Stream socket peer closed connection, or shut down writing half
              of connection.  The _GNU_SOURCE feature test macro must be
              defined (before including any header files) in order to obtain
              this definition.

       POLLERR
              Error condition (only returned in revents; ignored in events).
              This bit is also set for a file descriptor referring to the
              write end of a pipe when the read end has been closed.

       POLLHUP
              Hang up (only returned in revents; ignored in events).  Note
              that when reading from a channel such as a pipe or a stream
              socket, this event merely indicates that the peer closed its end
              of the channel.  Subsequent reads from the channel will return 0
              (end of file) only after all outstanding data in the channel has
              been consumed.

       POLLNVAL
              Invalid request: fd not open (only returned in revents; ignored
              in events).

       When compiling with _XOPEN_SOURCE defined, one also has the following,
       which convey no further information beyond the bits listed above:

       POLLRDNORM
              Equivalent to POLLIN.

       POLLRDBAND
              Priority band data can be read (generally unused on Linux).

       POLLWRNORM
              Equivalent to POLLOUT.

       POLLWRBAND
              Priority data may be written.

       Linux also knows about, but does not use POLLMSG.
```

```
   ppoll()
       The relationship between poll() and ppoll() is analogous to the
       relationship between select(2) and pselect(2): like pselect(2), ppoll()
       allows an application to safely wait until either a file descriptor
       becomes ready or until a signal is caught.

       Other than the difference in the precision of the timeout argument, the
       following ppoll() call:

           ready = ppoll(&fds, nfds, tmo_p, &sigmask);

       is nearly equivalent to atomically executing the following calls:

           sigset_t origmask;
           int timeout;

           timeout = (tmo_p == NULL) ? -1 :
                     (tmo_p->tv_sec * 1000 + tmo_p->tv_nsec / 1000000);
           pthread_sigmask(SIG_SETMASK, &sigmask, &origmask);
           ready = poll(&fds, nfds, timeout);
           pthread_sigmask(SIG_SETMASK, &origmask, NULL);

       The above code segment is described as nearly equivalent because
       whereas a negative timeout value for poll() is interpreted as an
       infinite timeout, a negative value expressed in *tmo_p results in an
       error from ppoll().

       See the description of pselect(2) for an explanation of why ppoll() is
       necessary.

       If the sigmask argument is specified as NULL, then no signal mask
       manipulation is performed (and thus ppoll() differs from poll() only in
       the precision of the timeout argument).

       The tmo_p argument specifies an upper limit on the amount of time that
       ppoll() will block.  This argument is a pointer to a timespec(3)
       structure.

       If tmo_p is specified as NULL, then ppoll() can block indefinitely.

RETURN VALUE
       On success, poll() returns a nonnegative value which is the number of
       elements in the pollfds whose revents fields have been set to a nonzero
       value (indicating an event or an error).  A return value of zero
       indicates that the system call timed out before any file descriptors
       became ready.

       On error, -1 is returned, and errno is set to indicate the error.

ERRORS
       EFAULT fds points outside the process's accessible address space.  The
              array given as argument was not contained in the calling
              program's address space.

       EINTR  A signal occurred before any requested event; see signal(7).
```

```
       EINVAL The nfds value exceeds the RLIMIT_NOFILE value.

       EINVAL (ppoll()) The timeout value expressed in *tmo_p is invalid
              (negative).

       ENOMEM Unable to allocate memory for kernel data structures.

VERSIONS
       On some other UNIX systems, poll() can fail with the error EAGAIN if
       the system fails to allocate kernel-internal resources, rather than
       ENOMEM as Linux does.  POSIX permits this behavior.  Portable programs
       may wish to check for EAGAIN and loop, just as with EINTR.

       Some implementations define the nonstandard constant INFTIM with the
       value -1 for use as a timeout for poll().  This constant is not
       provided in glibc.

   C library/kernel differences
       The Linux ppoll() system call modifies its tmo_p argument.  However,
       the glibc wrapper function hides this behavior by using a local
       variable for the timeout argument that is passed to the system call.
       Thus, the glibc ppoll() function does not modify its tmo_p argument.

       The raw ppoll() system call has a fifth argument, size_t sigsetsize,
       which specifies the size in bytes of the sigmask argument.  The glibc
       ppoll() wrapper function specifies this argument as a fixed value
       (equal to sizeof(kernel_sigset_t)).  See sigprocmask(2) for a
       discussion on the differences between the kernel and the libc notion of
       the sigset.

STANDARDS
       poll() POSIX.1-2008.

       ppoll()
              Linux.

HISTORY
       poll() POSIX.1-2001.  Linux 2.1.23.

              On older kernels that lack this system call, the glibc poll()
              wrapper function provides emulation using select(2).

       ppoll()
              Linux 2.6.16, glibc 2.4.

NOTES
       The operation of poll() and ppoll() is not affected by the O_NONBLOCK
       flag.

       For a discussion of what may happen if a file descriptor being
       monitored by poll() is closed in another thread, see select(2).

BUGS
       See the discussion of spurious readiness notifications under the BUGS
```

```
       section of select(2).

EXAMPLES
       The program below opens each of the files named in its command-line
       arguments and monitors the resulting file descriptors for readiness to
       read (POLLIN).  The program loops, repeatedly using poll() to monitor
       the file descriptors, printing the number of ready file descriptors on
       return.  For each ready file descriptor, the program:

       •  displays the returned revents field in a human-readable form;

       •  if the file descriptor is readable, reads some data from it, and
          displays that data on standard output; and

       •  if the file descriptor was not readable, but some other event
          occurred (presumably POLLHUP), closes the file descriptor.

       Suppose we run the program in one terminal, asking it to open a FIFO:

           $ mkfifo myfifo
           $ ./poll_input myfifo

       In a second terminal window, we then open the FIFO for writing, write
       some data to it, and close the FIFO:

           $ echo aaaaabbbbbccccc > myfifo

       In the terminal where we are running the program, we would then see:

           Opened "myfifo" on fd 3
           About to poll()
           Ready: 1
             fd=3; events: POLLIN POLLHUP
               read 10 bytes: aaaaabbbbb
           About to poll()
           Ready: 1
             fd=3; events: POLLIN POLLHUP
               read 6 bytes: ccccc

           About to poll()
           Ready: 1
             fd=3; events: POLLHUP
               closing fd 3
           All file descriptors closed; bye

       In the above output, we see that poll() returned three times:

       •  On the first return, the bits returned in the revents field were
          POLLIN, indicating that the file descriptor is readable, and
          POLLHUP, indicating that the other end of the FIFO has been closed.
          The program then consumed some of the available input.

       •  The second return from poll() also indicated POLLIN and POLLHUP; the
          program then consumed the last of the available input.
```

- On the final return, poll() indicated only POLLHUP on the FIFO, at which point the file descriptor was closed and the program terminated.

Program source

```
    /* poll_input.c

       Licensed under GNU General Public License v2 or later.
    */
    #include <fcntl.h>
    #include <poll.h>
    #include <stdio.h>
    #include <stdlib.h>
    #include <unistd.h>

    #define errExit(msg)    do { perror(msg); exit(EXIT_FAILURE); \
                            } while (0)

    int
    main(int argc, char *argv[])
    {
        int            ready;
        char           buf[10];
        nfds_t         num_open_fds, nfds;
        ssize_t        s;
        struct pollfd  *pfds;

        if (argc < 2) {
           fprintf(stderr, "Usage: %s file...\n", argv[0]);
           exit(EXIT_FAILURE);
        }

        num_open_fds = nfds = argc - 1;
        pfds = calloc(nfds, sizeof(struct pollfd));
        if (pfds == NULL)
            errExit("malloc");

        /* Open each file on command line, and add it to 'pfds' array. */

        for (nfds_t j = 0; j < nfds; j++) {
            pfds[j].fd = open(argv[j + 1], O_RDONLY);
            if (pfds[j].fd == -1)
                errExit("open");

            printf("Opened \"%s\" on fd %d\n", argv[j + 1], pfds[j].fd);

            pfds[j].events = POLLIN;
        }

        /* Keep calling poll() as long as at least one file descriptor is
           open. */

        while (num_open_fds > 0) {
            printf("About to poll()\n");
```

```
               ready = poll(pfds, nfds, -1);
               if (ready == -1)
                   errExit("poll");

               printf("Ready: %d\n", ready);

               /* Deal with array returned by poll(). */

               for (nfds_t j = 0; j < nfds; j++) {
                   if (pfds[j].revents != 0) {
                       printf("  fd=%d; events: %s%s%s\n", pfds[j].fd,
                              (pfds[j].revents & POLLIN)  ? "POLLIN "  : "",
                              (pfds[j].revents & POLLHUP) ? "POLLHUP " : "",
                              (pfds[j].revents & POLLERR) ? "POLLERR " : "");

                       if (pfds[j].revents & POLLIN) {
                           s = read(pfds[j].fd, buf, sizeof(buf));
                           if (s == -1)
                               errExit("read");
                           printf("    read %zd bytes: %.*s\n",
                                  s, (int) s, buf);
                       } else {                /* POLLERR | POLLHUP */
                           printf("    closing fd %d\n", pfds[j].fd);
                           if (close(pfds[j].fd) == -1)
                               errExit("close");
                           num_open_fds--;
                       }
                   }
               }

               printf("All file descriptors closed; bye\n");
               exit(EXIT_SUCCESS);
           }

SEE ALSO
       restart_syscall(2), select(2), select_tut(2), timespec(3), epoll(7),
       time(7)

Linux man-pages 6.7              2023-10-31                          poll(2)
```

---

mmap(2)

```
mmap(2)                      System Calls Manual                     mmap(2)

NAME
       mmap, munmap - map or unmap files or devices into memory

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
```

```
       #include <sys/mman.h>

       void *mmap(void addr[.length], size_t length, int prot, int flags,
                  int fd, off_t offset);
       int munmap(void addr[.length], size_t length);

       See NOTES for information on feature test macro requirements.

DESCRIPTION
       mmap() creates a new mapping in the virtual address space of the
       calling process.  The starting address for the new mapping is specified
       in addr.  The length argument specifies the length of the mapping
       (which must be greater than 0).

       If addr is NULL, then the kernel chooses the (page-aligned) address at
       which to create the mapping; this is the most portable method of
       creating a new mapping.  If addr is not NULL, then the kernel takes it
       as a hint about where to place the mapping; on Linux, the kernel will
       pick a nearby page boundary (but always above or equal to the value
       specified by /proc/sys/vm/mmap_min_addr) and attempt to create the
       mapping there.  If another mapping already exists there, the kernel
       picks a new address that may or may not depend on the hint.  The
       address of the new mapping is returned as the result of the call.

       The contents of a file mapping (as opposed to an anonymous mapping; see
       MAP_ANONYMOUS below), are initialized using length bytes starting at
       offset offset in the file (or other object) referred to by the file
       descriptor fd.  offset must be a multiple of the page size as returned
       by sysconf(_SC_PAGE_SIZE).

       After the mmap() call has returned, the file descriptor, fd, can be
       closed immediately without invalidating the mapping.

       The prot argument describes the desired memory protection of the
       mapping (and must not conflict with the open mode of the file).  It is
       either PROT_NONE or the bitwise OR of one or more of the following
       flags:

       PROT_EXEC  Pages may be executed.

       PROT_READ  Pages may be read.

       PROT_WRITE Pages may be written.

       PROT_NONE  Pages may not be accessed.

   The flags argument
       The flags argument determines whether updates to the mapping are
       visible to other processes mapping the same region, and whether updates
       are carried through to the underlying file.  This behavior is
       determined by including exactly one of the following values in flags:

       MAP_SHARED
              Share this mapping.  Updates to the mapping are visible to other
              processes mapping the same region, and (in the case of file-
```

```
       backed mappings) are carried through to the underlying file.
       (To precisely control when updates are carried through to the
       underlying file requires the use of msync(2).)

MAP_SHARED_VALIDATE (since Linux 4.15)
       This flag provides the same behavior as MAP_SHARED except that
       MAP_SHARED mappings ignore unknown flags in flags.  By contrast,
       when creating a mapping using MAP_SHARED_VALIDATE, the kernel
       verifies all passed flags are known and fails the mapping with
       the error EOPNOTSUPP for unknown flags.  This mapping type is
       also required to be able to use some mapping flags (e.g.,
       MAP_SYNC).

MAP_PRIVATE
       Create a private copy-on-write mapping.  Updates to the mapping
       are not visible to other processes mapping the same file, and
       are not carried through to the underlying file.  It is
       unspecified whether changes made to the file after the mmap()
       call are visible in the mapped region.

Both MAP_SHARED and MAP_PRIVATE are described in POSIX.1-2001 and
POSIX.1-2008.  MAP_SHARED_VALIDATE is a Linux extension.

In addition, zero or more of the following values can be ORed in flags:

MAP_32BIT (since Linux 2.4.20, 2.6)
       Put the mapping into the first 2 Gigabytes of the process
       address space.  This flag is supported only on x86-64, for
       64-bit programs.  It was added to allow thread stacks to be
       allocated somewhere in the first 2 GB of memory, so as to
       improve context-switch performance on some early 64-bit
       processors.  Modern x86-64 processors no longer have this
       performance problem, so use of this flag is not required on
       those systems.  The MAP_32BIT flag is ignored when MAP_FIXED is
       set.

MAP_ANON
       Synonym for MAP_ANONYMOUS; provided for compatibility with other
       implementations.

MAP_ANONYMOUS
       The mapping is not backed by any file; its contents are
       initialized to zero.  The fd argument is ignored; however, some
       implementations require fd to be -1 if MAP_ANONYMOUS (or
       MAP_ANON) is specified, and portable applications should ensure
       this.  The offset argument should be zero.  Support for
       MAP_ANONYMOUS in conjunction with MAP_SHARED was added in Linux
       2.4.

MAP_DENYWRITE
       This flag is ignored.  (Long ago-Linux 2.0 and earlier-it
       signaled that attempts to write to the underlying file should
       fail with ETXTBSY.  But this was a source of denial-of-service
       attacks.)
```

```
       MAP_EXECUTABLE
              This flag is ignored.

       MAP_FILE
              Compatibility flag.  Ignored.

       MAP_FIXED
              Don't interpret addr as a hint: place the mapping at exactly
              that address.  addr must be suitably aligned: for most
              architectures a multiple of the page size is sufficient;
              however, some architectures may impose additional restrictions.
              If the memory region specified by addr and length overlaps pages
              of any existing mapping(s), then the overlapped part of the
              existing mapping(s) will be discarded.  If the specified address
              cannot be used, mmap() will fail.

              Software that aspires to be portable should use the MAP_FIXED
              flag with care, keeping in mind that the exact layout of a
              process's memory mappings is allowed to change significantly
              between Linux versions, C library versions, and operating system
              releases.  Carefully read the discussion of this flag in NOTES!

       MAP_FIXED_NOREPLACE (since Linux 4.17)
              This flag provides behavior that is similar to MAP_FIXED with
              respect to the addr enforcement, but differs in that
              MAP_FIXED_NOREPLACE never clobbers a preexisting mapped range.
              If the requested range would collide with an existing mapping,
              then this call fails with the error EEXIST.  This flag can
              therefore be used as a way to atomically (with respect to other
              threads) attempt to map an address range: one thread will
              succeed; all others will report failure.

              Note that older kernels which do not recognize the
              MAP_FIXED_NOREPLACE flag will typically (upon detecting a
              collision with a preexisting mapping) fall back to a "non-
              MAP_FIXED" type of behavior: they will return an address that is
              different from the requested address.  Therefore, backward-
              compatible software should check the returned address against
              the requested address.

       MAP_GROWSDOWN
              This flag is used for stacks.  It indicates to the kernel
              virtual memory system that the mapping should extend downward in
              memory.  The return address is one page lower than the memory
              area that is actually created in the process's virtual address
              space.  Touching an address in the "guard" page below the
              mapping will cause the mapping to grow by a page.  This growth
              can be repeated until the mapping grows to within a page of the
              high end of the next lower mapping, at which point touching the
              "guard" page will result in a SIGSEGV signal.

       MAP_HUGETLB (since Linux 2.6.32)
              Allocate the mapping using "huge" pages.  See the Linux kernel
              source file Documentation/admin-guide/mm/hugetlbpage.rst for
              further information, as well as NOTES, below.
```

```
       MAP_HUGE_2MB
       MAP_HUGE_1GB (since Linux 3.8)
              Used in conjunction with MAP_HUGETLB to select alternative
              hugetlb page sizes (respectively, 2 MB and 1 GB) on systems that
              support multiple hugetlb page sizes.

              More generally, the desired huge page size can be configured by
              encoding the base-2 logarithm of the desired page size in the
              six bits at the offset MAP_HUGE_SHIFT.  (A value of zero in this
              bit field provides the default huge page size; the default huge
              page size can be discovered via the Hugepagesize field exposed
              by /proc/meminfo.)  Thus, the above two constants are defined
              as:

                  #define MAP_HUGE_2MB    (21 << MAP_HUGE_SHIFT)
                  #define MAP_HUGE_1GB    (30 << MAP_HUGE_SHIFT)

              The range of huge page sizes that are supported by the system
              can be discovered by listing the subdirectories in
              /sys/kernel/mm/hugepages.

       MAP_LOCKED (since Linux 2.5.37)
              Mark the mapped region to be locked in the same way as mlock(2).
              This implementation will try to populate (prefault) the whole
              range but the mmap() call doesn't fail with ENOMEM if this
              fails.  Therefore major faults might happen later on.  So the
              semantic is not as strong as mlock(2).  One should use mmap()
              plus mlock(2) when major faults are not acceptable after the
              initialization of the mapping.  The MAP_LOCKED flag is ignored
              in older kernels.

       MAP_NONBLOCK (since Linux 2.5.46)
              This flag is meaningful only in conjunction with MAP_POPULATE.
              Don't perform read-ahead: create page tables entries only for
              pages that are already present in RAM.  Since Linux 2.6.23, this
              flag causes MAP_POPULATE to do nothing.  One day, the
              combination of MAP_POPULATE and MAP_NONBLOCK may be
              reimplemented.

       MAP_NORESERVE
              Do not reserve swap space for this mapping.  When swap space is
              reserved, one has the guarantee that it is possible to modify
              the mapping.  When swap space is not reserved one might get
              SIGSEGV upon a write if no physical memory is available.  See
              also the discussion of the file /proc/sys/vm/overcommit_memory
              in proc(5).  Before Linux 2.6, this flag had effect only for
              private writable mappings.

       MAP_POPULATE (since Linux 2.5.46)
              Populate (prefault) page tables for a mapping.  For a file
              mapping, this causes read-ahead on the file.  This will help to
              reduce blocking on page faults later.  The mmap() call doesn't
              fail if the mapping cannot be populated (for example, due to
              limitations on the number of mapped huge pages when using
```

```
              MAP_HUGETLB).  Support for MAP_POPULATE in conjunction with
              private mappings was added in Linux 2.6.23.

       MAP_STACK (since Linux 2.6.27)
              Allocate the mapping at an address suitable for a process or
              thread stack.

              This flag is currently a no-op on Linux.  However, by employing
              this flag, applications can ensure that they transparently
              obtain support if the flag is implemented in the future.  Thus,
              it is used in the glibc threading implementation to allow for
              the fact that some architectures may (later) require special
              treatment for stack allocations.  A further reason to employ
              this flag is portability: MAP_STACK exists (and has an effect)
              on some other systems (e.g., some of the BSDs).

       MAP_SYNC (since Linux 4.15)
              This flag is available only with the MAP_SHARED_VALIDATE mapping
              type; mappings of type MAP_SHARED will silently ignore this
              flag.  This flag is supported only for files supporting DAX
              (direct mapping of persistent memory).  For other files,
              creating a mapping with this flag results in an EOPNOTSUPP
              error.

              Shared file mappings with this flag provide the guarantee that
              while some memory is mapped writable in the address space of the
              process, it will be visible in the same file at the same offset
              even after the system crashes or is rebooted.  In conjunction
              with the use of appropriate CPU instructions, this provides
              users of such mappings with a more efficient way of making data
              modifications persistent.

       MAP_UNINITIALIZED (since Linux 2.6.33)
              Don't clear anonymous pages.  This flag is intended to improve
              performance on embedded devices.  This flag is honored only if
              the kernel was configured with the
              CONFIG_MMAP_ALLOW_UNINITIALIZED option.  Because of the security
              implications, that option is normally enabled only on embedded
              devices (i.e., devices where one has complete control of the
              contents of user memory).

       Of the above flags, only MAP_FIXED is specified in POSIX.1-2001 and
       POSIX.1-2008.  However, most systems also support MAP_ANONYMOUS (or its
       synonym MAP_ANON).

   munmap()
       The munmap() system call deletes the mappings for the specified address
       range, and causes further references to addresses within the range to
       generate invalid memory references.  The region is also automatically
       unmapped when the process is terminated.  On the other hand, closing
       the file descriptor does not unmap the region.

       The address addr must be a multiple of the page size (but length need
       not be).  All pages containing a part of the indicated range are
       unmapped, and subsequent references to these pages will generate
```

```
       SIGSEGV.  It is not an error if the indicated range does not contain
       any mapped pages.

RETURN VALUE
       On success, mmap() returns a pointer to the mapped area.  On error, the
       value MAP_FAILED (that is, (void *) -1) is returned, and errno is set
       to indicate the error.

       On success, munmap() returns 0.  On failure, it returns -1, and errno
       is set to indicate the error (probably to EINVAL).

ERRORS
       EACCES A file descriptor refers to a non-regular file.  Or a file
              mapping was requested, but fd is not open for reading.  Or
              MAP_SHARED was requested and PROT_WRITE is set, but fd is not
              open in read/write (O_RDWR) mode.  Or PROT_WRITE is set, but the
              file is append-only.

       EAGAIN The file has been locked, or too much memory has been locked
              (see setrlimit(2)).

       EBADF  fd is not a valid file descriptor (and MAP_ANONYMOUS was not
              set).

       EEXIST MAP_FIXED_NOREPLACE was specified in flags, and the range
              covered by addr and length clashes with an existing mapping.

       EINVAL We don't like addr, length, or offset (e.g., they are too large,
              or not aligned on a page boundary).

       EINVAL (since Linux 2.6.12) length was 0.

       EINVAL flags contained none of MAP_PRIVATE, MAP_SHARED, or
              MAP_SHARED_VALIDATE.

       ENFILE The system-wide limit on the total number of open files has been
              reached.

       ENODEV The underlying filesystem of the specified file does not support
              memory mapping.

       ENOMEM No memory is available.

       ENOMEM The process's maximum number of mappings would have been
              exceeded.  This error can also occur for munmap(), when
              unmapping a region in the middle of an existing mapping, since
              this results in two smaller mappings on either side of the
              region being unmapped.

       ENOMEM (since Linux 4.7) The process's RLIMIT_DATA limit, described in
              getrlimit(2), would have been exceeded.

       ENOMEM We don't like addr, because it exceeds the virtual address space
              of the CPU.
```

```
       EOVERFLOW
              On 32-bit architecture together with the large file extension
              (i.e., using 64-bit off_t): the number of pages used for length
              plus number of pages used for offset would overflow unsigned
              long (32 bits).

       EPERM  The prot argument asks for PROT_EXEC but the mapped area belongs
              to a file on a filesystem that was mounted no-exec.

       EPERM  The operation was prevented by a file seal; see fcntl(2).

       EPERM  The MAP_HUGETLB flag was specified, but the caller was not
              privileged (did not have the CAP_IPC_LOCK capability) and is not
              a member of the sysctl_hugetlb_shm_group group; see the
              description of /proc/sys/vm/sysctl_hugetlb_shm_group in
              proc_sys(5).

       ETXTBSY
              MAP_DENYWRITE was set but the object specified by fd is open for
              writing.

       Use of a mapped region can result in these signals:

       SIGSEGV
              Attempted write into a region mapped as read-only.

       SIGBUS Attempted access to a page of the buffer that lies beyond the
              end of the mapped file.  For an explanation of the treatment of
              the bytes in the page that corresponds to the end of a mapped
              file that is not a multiple of the page size, see NOTES.

ATTRIBUTES
       For an explanation of the terms used in this section, see
       attributes(7).

        Interface                                Attribute       Value

        mmap(), munmap()                         Thread safety   MT-Safe


VERSIONS
       On some hardware architectures (e.g., i386), PROT_WRITE implies
       PROT_READ.  It is architecture dependent whether PROT_READ implies
       PROT_EXEC or not.  Portable programs should always set PROT_EXEC if
       they intend to execute code in the new mapping.

       The portable way to create a mapping is to specify addr as 0 (NULL),
       and omit MAP_FIXED from flags.  In this case, the system chooses the
       address for the mapping; the address is chosen so as not to conflict
       with any existing mapping, and will not be 0.  If the MAP_FIXED flag is
       specified, and addr is 0 (NULL), then the mapped address will be 0
       (NULL).

       Certain flags constants are defined only if suitable feature test
       macros are defined (possibly by default): _DEFAULT_SOURCE with glibc
```

```
       2.19 or later; or _BSD_SOURCE or _SVID_SOURCE in glibc 2.19 and
       earlier.  (Employing _GNU_SOURCE also suffices, and requiring that
       macro specifically would have been more logical, since these flags are
       all Linux-specific.)  The relevant flags are: MAP_32BIT, MAP_ANONYMOUS
       (and the synonym MAP_ANON), MAP_DENYWRITE, MAP_EXECUTABLE, MAP_FILE,
       MAP_GROWSDOWN, MAP_HUGETLB, MAP_LOCKED, MAP_NONBLOCK, MAP_NORESERVE,
       MAP_POPULATE, and MAP_STACK.

   C library/kernel differences
       This page describes the interface provided by the glibc mmap() wrapper
       function.  Originally, this function invoked a system call of the same
       name.  Since Linux 2.4, that system call has been superseded by
       mmap2(2), and nowadays the glibc mmap() wrapper function invokes
       mmap2(2) with a suitably adjusted value for offset.

STANDARDS
       POSIX.1-2008.

HISTORY
       POSIX.1-2001, SVr4, 4.4BSD.

       On POSIX systems on which mmap(), msync(2), and munmap() are available,
       _POSIX_MAPPED_FILES is defined in <unistd.h> to a value greater than 0.
       (See also sysconf(3).)

NOTES
       Memory mapped by mmap() is preserved across fork(2), with the same
       attributes.

       A file is mapped in multiples of the page size.  For a file that is not
       a multiple of the page size, the remaining bytes in the partial page at
       the end of the mapping are zeroed when mapped, and modifications to
       that region are not written out to the file.  The effect of changing
       the size of the underlying file of a mapping on the pages that
       correspond to added or removed regions of the file is unspecified.

       An application can determine which pages of a mapping are currently
       resident in the buffer/page cache using mincore(2).

   Using MAP_FIXED safely
       The only safe use for MAP_FIXED is where the address range specified by
       addr and length was previously reserved using another mapping;
       otherwise, the use of MAP_FIXED is hazardous because it forcibly
       removes preexisting mappings, making it easy for a multithreaded
       process to corrupt its own address space.

       For example, suppose that thread A looks through /proc/pid/maps in
       order to locate an unused address range that it can map using
       MAP_FIXED, while thread B simultaneously acquires part or all of that
       same address range.  When thread A subsequently employs
       mmap(MAP_FIXED), it will effectively clobber the mapping that thread B
       created.  In this scenario, thread B need not create a mapping
       directly; simply making a library call that, internally, uses dlopen(3)
       to load some other shared library, will suffice.  The dlopen(3) call
       will map the library into the process's address space.  Furthermore,
```

```
       almost any library call may be implemented in a way that adds memory
       mappings to the address space, either with this technique, or by simply
       allocating memory.  Examples include brk(2), malloc(3),
       pthread_create(3), and the PAM libraries http://www.linux-pam.org.

       Since Linux 4.17, a multithreaded program can use the
       MAP_FIXED_NOREPLACE flag to avoid the hazard described above when
       attempting to create a mapping at a fixed address that has not been
       reserved by a preexisting mapping.

   Timestamps changes for file-backed mappings
       For file-backed mappings, the st_atime field for the mapped file may be
       updated at any time between the mmap() and the corresponding unmapping;
       the first reference to a mapped page will update the field if it has
       not been already.

       The st_ctime and st_mtime field for a file mapped with PROT_WRITE and
       MAP_SHARED will be updated after a write to the mapped region, and
       before a subsequent msync(2) with the MS_SYNC or MS_ASYNC flag, if one
       occurs.

   Huge page (Huge TLB) mappings
       For mappings that employ huge pages, the requirements for the arguments
       of mmap() and munmap() differ somewhat from the requirements for
       mappings that use the native system page size.

       For mmap(), offset must be a multiple of the underlying huge page size.
       The system automatically aligns length to be a multiple of the
       underlying huge page size.

       For munmap(), addr, and length must both be a multiple of the
       underlying huge page size.

BUGS
       On Linux, there are no guarantees like those suggested above under
       MAP_NORESERVE.  By default, any process can be killed at any moment
       when the system runs out of memory.

       Before Linux 2.6.7, the MAP_POPULATE flag has effect only if prot is
       specified as PROT_NONE.

       SUSv3 specifies that mmap() should fail if length is 0.  However,
       before Linux 2.6.12, mmap() succeeded in this case: no mapping was
       created and the call returned addr.  Since Linux 2.6.12, mmap() fails
       with the error EINVAL for this case.

       POSIX specifies that the system shall always zero fill any partial page
       at the end of the object and that system will never write any
       modification of the object beyond its end.  On Linux, when you write
       data to such partial page after the end of the object, the data stays
       in the page cache even after the file is closed and unmapped and even
       though the data is never written to the file itself, subsequent
       mappings may see the modified content.  In some cases, this could be
       fixed by calling msync(2) before the unmap takes place; however, this
       doesn't work on tmpfs(5) (for example, when using the POSIX shared
```

```
       memory interface documented in shm_overview(7)).

EXAMPLES
       The following program prints part of the file specified in its first
       command-line argument to standard output.  The range of bytes to be
       printed is specified via offset and length values in the second and
       third command-line arguments.  The program creates a memory mapping of
       the required pages of the file and then uses write(2) to output the
       desired bytes.

   Program source
       #include <fcntl.h>
       #include <stdio.h>
       #include <stdlib.h>
       #include <sys/mman.h>
       #include <sys/stat.h>
       #include <unistd.h>

       #define handle_error(msg) \
           do { perror(msg); exit(EXIT_FAILURE); } while (0)

       int
       main(int argc, char *argv[])
       {
           int          fd;
           char         *addr;
           off_t        offset, pa_offset;
           size_t       length;
           ssize_t      s;
           struct stat  sb;

           if (argc < 3 || argc > 4) {
               fprintf(stderr, "%s file offset [length]\n", argv[0]);
               exit(EXIT_FAILURE);
           }

           fd = open(argv[1], O_RDONLY);
           if (fd == -1)
               handle_error("open");

           if (fstat(fd, &sb) == -1)           /* To obtain file size */
               handle_error("fstat");

           offset = atoi(argv[2]);
           pa_offset = offset & ~(sysconf(_SC_PAGE_SIZE) - 1);
               /* offset for mmap() must be page aligned */

           if (offset >= sb.st_size) {
               fprintf(stderr, "offset is past end of file\n");
               exit(EXIT_FAILURE);
           }

           if (argc == 4) {
               length = atoi(argv[3]);
               if (offset + length > sb.st_size)
```

```
                    length = sb.st_size - offset;
                        /* Can't display bytes past end of file */

            } else {    /* No length arg ==> display to end of file */
                length = sb.st_size - offset;
            }

            addr = mmap(NULL, length + offset - pa_offset, PROT_READ,
                        MAP_PRIVATE, fd, pa_offset);
            if (addr == MAP_FAILED)
                handle_error("mmap");

            s = write(STDOUT_FILENO, addr + offset - pa_offset, length);
            if (s != length) {
                if (s == -1)
                    handle_error("write");

                fprintf(stderr, "partial write");
                exit(EXIT_FAILURE);
            }

            munmap(addr, length + offset - pa_offset);
            close(fd);

            exit(EXIT_SUCCESS);
        }

SEE ALSO
       ftruncate(2), getpagesize(2), memfd_create(2), mincore(2), mlock(2),
       mmap2(2), mprotect(2), mremap(2), msync(2), remap_file_pages(2),
       setrlimit(2), shmat(2), userfaultfd(2), shm_open(3), shm_overview(7)

       The descriptions of the following files in proc(5): /proc/pid/maps,
       /proc/pid/map_files, and /proc/pid/smaps.

       B.O. Gallmeister, POSIX.4, O'Reilly, pp. 128-129 and 389-391.

Linux man-pages 6.7              2023-10-31                          mmap(2)
```

---

brk(2)

```
brk(2)                        System Calls Manual                        brk(2)

NAME
       brk, sbrk - change data segment size

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>
```

```
       int brk(void *addr);
       void *sbrk(intptr_t increment);

   Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

       brk(), sbrk():
           Since glibc 2.19:
               _DEFAULT_SOURCE
                   || ((_XOPEN_SOURCE >= 500) &&
                       ! (_POSIX_C_SOURCE >= 200112L))
           From glibc 2.12 to glibc 2.19:
               _BSD_SOURCE || _SVID_SOURCE
                   || ((_XOPEN_SOURCE >= 500) &&
                       ! (_POSIX_C_SOURCE >= 200112L))
           Before glibc 2.12:
               _BSD_SOURCE || _SVID_SOURCE || _XOPEN_SOURCE >= 500

DESCRIPTION
       brk() and sbrk() change the location of the program break, which
       defines the end of the process's data segment (i.e., the program break
       is the first location after the end of the uninitialized data segment).
       Increasing the program break has the effect of allocating memory to the
       process; decreasing the break deallocates memory.

       brk() sets the end of the data segment to the value specified by addr,
       when that value is reasonable, the system has enough memory, and the
       process does not exceed its maximum data size (see setrlimit(2)).

       sbrk() increments the program's data space by increment bytes.  Calling
       sbrk() with an increment of 0 can be used to find the current location
       of the program break.

RETURN VALUE
       On success, brk() returns zero.  On error, -1 is returned, and errno is
       set to ENOMEM.

       On success, sbrk() returns the previous program break.  (If the break
       was increased, then this value is a pointer to the start of the newly
       allocated memory).  On error, (void *) -1 is returned, and errno is set
       to ENOMEM.

STANDARDS
       None.

HISTORY
       4.3BSD; SUSv1, marked LEGACY in SUSv2, removed in POSIX.1-2001.

NOTES
       Avoid using brk() and sbrk(): the malloc(3) memory allocation package
       is the portable and comfortable way of allocating memory.

       Various systems use various types for the argument of sbrk().  Common
       are int, ssize_t, ptrdiff_t, intptr_t.

   C library/kernel differences
```

```
       The return value described above for brk() is the behavior provided by
       the glibc wrapper function for the Linux brk() system call.  (On most
       other implementations, the return value from brk() is the same; this
       return value was also specified in SUSv2.)  However, the actual Linux
       system call returns the new program break on success.  On failure, the
       system call returns the current break.  The glibc wrapper function does
       some work (i.e., checks whether the new break is less than addr) to
       provide the 0 and -1 return values described above.

       On Linux, sbrk() is implemented as a library function that uses the
       brk() system call, and does some internal bookkeeping so that it can
       return the old break value.

SEE ALSO
       execve(2), getrlimit(2), end(3), malloc(3)

Linux man-pages 6.7              2023-10-31                          brk(2)
```

---

clone(2)

```
clone(2)                      System Calls Manual                     clone(2)

NAME
       clone, __clone2, clone3 - create a child process

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       /* Prototype for the glibc wrapper function */

       #define _GNU_SOURCE
       #include <sched.h>

       int clone(int (*fn)(void *_Nullable), void *stack, int flags,
                 void *_Nullable arg, ...  /* pid_t *_Nullable parent_tid,
                                              void *_Nullable tls,
                                              pid_t *_Nullable child_tid */ );

       /* For the prototype of the raw clone() system call, see NOTES */

       #include <linux/sched.h>    /* Definition of struct clone_args */
       #include <sched.h>          /* Definition of CLONE_* constants */
       #include <sys/syscall.h>    /* Definition of SYS_* constants */
       #include <unistd.h>

       long syscall(SYS_clone3, struct clone_args *cl_args, size_t size);

       Note: glibc provides no wrapper for clone3(), necessitating the use of
       syscall(2).

DESCRIPTION
```

```
    These system calls create a new ("child") process, in a manner similar
    to fork(2).

    By contrast with fork(2), these system calls provide more precise
    control over what pieces of execution context are shared between the
    calling process and the child process.  For example, using these system
    calls, the caller can control whether or not the two processes share
    the virtual address space, the table of file descriptors, and the table
    of signal handlers.  These system calls also allow the new child
    process to be placed in separate namespaces(7).

    Note that in this manual page, "calling process" normally corresponds
    to "parent process".  But see the descriptions of CLONE_PARENT and
    CLONE_THREAD below.

    This page describes the following interfaces:

    • The glibc clone() wrapper function and the underlying system call on
      which it is based.  The main text describes the wrapper function;
      the differences for the raw system call are described toward the end
      of this page.

    • The newer clone3() system call.

    In the remainder of this page, the terminology "the clone call" is used
    when noting details that apply to all of these interfaces.

The clone() wrapper function
    When the child process is created with the clone() wrapper function, it
    commences execution by calling the function pointed to by the argument
    fn.  (This differs from fork(2), where execution continues in the child
    from the point of the fork(2) call.)  The arg argument is passed as the
    argument of the function fn.

    When the fn(arg) function returns, the child process terminates.  The
    integer returned by fn is the exit status for the child process.  The
    child process may also terminate explicitly by calling exit(2) or after
    receiving a fatal signal.

    The stack argument specifies the location of the stack used by the
    child process.  Since the child and calling process may share memory,
    it is not possible for the child process to execute in the same stack
    as the calling process.  The calling process must therefore set up
    memory space for the child stack and pass a pointer to this space to
    clone().  Stacks grow downward on all processors that run Linux (except
    the HP PA processors), so stack usually points to the topmost address
    of the memory space set up for the child stack.  Note that clone() does
    not provide a means whereby the caller can inform the kernel of the
    size of the stack area.

    The remaining arguments to clone() are discussed below.

clone3()
    The clone3() system call provides a superset of the functionality of
    the older clone() interface.  It also provides a number of API
```

```
    improvements, including: space for additional flags bits; cleaner
    separation in the use of various arguments; and the ability to specify
    the size of the child's stack area.

    As with fork(2), clone3() returns in both the parent and the child.  It
    returns 0 in the child process and returns the PID of the child in the
    parent.

    The cl_args argument of clone3() is a structure of the following form:

        struct clone_args {
            u64 flags;        /* Flags bit mask */
            u64 pidfd;        /* Where to store PID file descriptor
                                 (int *) */
            u64 child_tid;    /* Where to store child TID,
                                 in child's memory (pid_t *) */
            u64 parent_tid;   /* Where to store child TID,
                                 in parent's memory (pid_t *) */
            u64 exit_signal;  /* Signal to deliver to parent on
                                 child termination */
            u64 stack;        /* Pointer to lowest byte of stack */
            u64 stack_size;   /* Size of stack */
            u64 tls;          /* Location of new TLS */
            u64 set_tid;      /* Pointer to a pid_t array
                                 (since Linux 5.5) */
            u64 set_tid_size; /* Number of elements in set_tid
                                 (since Linux 5.5) */
            u64 cgroup;       /* File descriptor for target cgroup
                                 of child (since Linux 5.7) */
        };

    The size argument that is supplied to clone3() should be initialized to
    the size of this structure.  (The existence of the size argument
    permits future extensions to the clone_args structure.)

    The stack for the child process is specified via cl_args.stack, which
    points to the lowest byte of the stack area, and cl_args.stack_size,
    which specifies the size of the stack in bytes.  In the case where the
    CLONE_VM flag (see below) is specified, a stack must be explicitly
    allocated and specified.  Otherwise, these two fields can be specified
    as NULL and 0, which causes the child to use the same stack area as the
    parent (in the child's own virtual address space).

    The remaining fields in the cl_args argument are discussed below.

Equivalence between clone() and clone3() arguments
    Unlike the older clone() interface, where arguments are passed
    individually, in the newer clone3() interface the arguments are
    packaged into the clone_args structure shown above.  This structure
    allows for a superset of the information passed via the clone()
    arguments.

    The following table shows the equivalence between the arguments of
    clone() and the fields in the clone_args argument supplied to clone3():
        clone()         clone3()        Notes
```

```
                        cl_args field
        flags & ~0xff   flags           For most flags; details
                                        below
        parent_tid      pidfd           See CLONE_PIDFD
        child_tid       child_tid       See CLONE_CHILD_SETTID
        parent_tid      parent_tid      See CLONE_PARENT_SETTID
        flags & 0xff    exit_signal
        stack           stack
        ---             stack_size
        tls             tls             See CLONE_SETTLS
        ---             set_tid         See below for details
        ---             set_tid_size
        ---             cgroup          See CLONE_INTO_CGROUP

The child termination signal
    When the child process terminates, a signal may be sent to the parent.
    The termination signal is specified in the low byte of flags (clone())
    or in cl_args.exit_signal (clone3()).  If this signal is specified as
    anything other than SIGCHLD, then the parent process must specify the
    __WALL or __WCLONE options when waiting for the child with wait(2).  If
    no signal (i.e., zero) is specified, then the parent process is not
    signaled when the child terminates.

The set_tid array
    By default, the kernel chooses the next sequential PID for the new
    process in each of the PID namespaces where it is present.  When
    creating a process with clone3(), the set_tid array (available since
    Linux 5.5) can be used to select specific PIDs for the process in some
    or all of the PID namespaces where it is present.  If the PID of the
    newly created process should be set only for the current PID namespace
    or in the newly created PID namespace (if flags contains CLONE_NEWPID)
    then the first element in the set_tid array has to be the desired PID
    and set_tid_size needs to be 1.

    If the PID of the newly created process should have a certain value in
    multiple PID namespaces, then the set_tid array can have multiple
    entries.  The first entry defines the PID in the most deeply nested PID
    namespace and each of the following entries contains the PID in the
    corresponding ancestor PID namespace.  The number of PID namespaces in
    which a PID should be set is defined by set_tid_size which cannot be
    larger than the number of currently nested PID namespaces.

    To create a process with the following PIDs in a PID namespace
    hierarchy:
        PID NS level   Requested PID   Notes
        0              31496           Outermost PID namespace
        1              42
        2              7               Innermost PID namespace

    Set the array to:

        set_tid[0] = 7;
        set_tid[1] = 42;
        set_tid[2] = 31496;
        set_tid_size = 3;
```

```
    If only the PIDs in the two innermost PID namespaces need to be
    specified, set the array to:

        set_tid[0] = 7;
        set_tid[1] = 42;
        set_tid_size = 2;

    The PID in the PID namespaces outside the two innermost PID namespaces
    is selected the same way as any other PID is selected.

    The set_tid feature requires CAP_SYS_ADMIN or (since Linux 5.9)
    CAP_CHECKPOINT_RESTORE in all owning user namespaces of the target PID
    namespaces.

    Callers may only choose a PID greater than 1 in a given PID namespace
    if an init process (i.e., a process with PID 1) already exists in that
    namespace.  Otherwise the PID entry for this PID namespace must be 1.

The flags mask
    Both clone() and clone3() allow a flags bit mask that modifies their
    behavior and allows the caller to specify what is shared between the
    calling process and the child process.  This bit mask-the flags
    argument of clone() or the cl_args.flags field passed to clone3()-is
    referred to as the flags mask in the remainder of this page.

    The flags mask is specified as a bitwise OR of zero or more of the
    constants listed below.  Except as noted below, these flags are
    available (and have the same effect) in both clone() and clone3().

    CLONE_CHILD_CLEARTID (since Linux 2.5.49)
           Clear (zero) the child thread ID at the location pointed to by
           child_tid (clone()) or cl_args.child_tid (clone3()) in child
           memory when the child exits, and do a wakeup on the futex at
           that address.  The address involved may be changed by the
           set_tid_address(2) system call.  This is used by threading
           libraries.

    CLONE_CHILD_SETTID (since Linux 2.5.49)
           Store the child thread ID at the location pointed to by
           child_tid (clone()) or cl_args.child_tid (clone3()) in the
           child's memory.  The store operation completes before the clone
           call returns control to user space in the child process.  (Note
           that the store operation may not have completed before the clone
           call returns in the parent process, which is relevant if the
           CLONE_VM flag is also employed.)

    CLONE_CLEAR_SIGHAND (since Linux 5.5)
           By default, signal dispositions in the child thread are the same
           as in the parent.  If this flag is specified, then all signals
           that are handled in the parent (and not set to SIG_IGN) are
           reset to their default dispositions (SIG_DFL) in the child.

           Specifying this flag together with CLONE_SIGHAND is nonsensical
           and disallowed.
```

```
   CLONE_DETACHED (historical)
          For a while (during the Linux 2.5 development series) there was
          a CLONE_DETACHED flag, which caused the parent not to receive a
          signal when the child terminated.  Ultimately, the effect of
          this flag was subsumed under the CLONE_THREAD flag and by the
          time Linux 2.6.0 was released, this flag had no effect.
          Starting in Linux 2.6.2, the need to give this flag together
          with CLONE_THREAD disappeared.

          This flag is still defined, but it is usually ignored when
          calling clone().  However, see the description of CLONE_PIDFD
          for some exceptions.

   CLONE_FILES (since Linux 2.0)
          If CLONE_FILES is set, the calling process and the child process
          share the same file descriptor table.  Any file descriptor
          created by the calling process or by the child process is also
          valid in the other process.  Similarly, if one of the processes
          closes a file descriptor, or changes its associated flags (using
          the fcntl(2) F_SETFD operation), the other process is also
          affected.  If a process sharing a file descriptor table calls
          execve(2), its file descriptor table is duplicated (unshared).

          If CLONE_FILES is not set, the child process inherits a copy of
          all file descriptors opened in the calling process at the time
          of the clone call.  Subsequent operations that open or close
          file descriptors, or change file descriptor flags, performed by
          either the calling process or the child process do not affect
          the other process.  Note, however, that the duplicated file
          descriptors in the child refer to the same open file
          descriptions as the corresponding file descriptors in the
          calling process, and thus share file offsets and file status
          flags (see open(2)).

   CLONE_FS (since Linux 2.0)
          If CLONE_FS is set, the caller and the child process share the
          same filesystem information.  This includes the root of the
          filesystem, the current working directory, and the umask.  Any
          call to chroot(2), chdir(2), or umask(2) performed by the
          calling process or the child process also affects the other
          process.

          If CLONE_FS is not set, the child process works on a copy of the
          filesystem information of the calling process at the time of the
          clone call.  Calls to chroot(2), chdir(2), or umask(2) performed
          later by one of the processes do not affect the other process.

   CLONE_INTO_CGROUP (since Linux 5.7)
          By default, a child process is placed in the same version 2
          cgroup as its parent.  The CLONE_INTO_CGROUP flag allows the
          child process to be created in a different version 2 cgroup.
          (Note that CLONE_INTO_CGROUP has effect only for version 2
          cgroups.)
```

```
       In order to place the child process in a different cgroup, the
       caller specifies CLONE_INTO_CGROUP in cl_args.flags and passes a
       file descriptor that refers to a version 2 cgroup in the
       cl_args.cgroup field.  (This file descriptor can be obtained by
       opening a cgroup v2 directory using either the O_RDONLY or the
       O_PATH flag.)  Note that all of the usual restrictions
       (described in cgroups(7)) on placing a process into a version 2
       cgroup apply.

       Among the possible use cases for CLONE_INTO_CGROUP are the
       following:

       • Spawning a process into a cgroup different from the parent's
         cgroup makes it possible for a service manager to directly
         spawn new services into dedicated cgroups.  This eliminates
         the accounting jitter that would be caused if the child
         process was first created in the same cgroup as the parent
         and then moved into the target cgroup.  Furthermore, spawning
         the child process directly into a target cgroup is
         significantly cheaper than moving the child process into the
         target cgroup after it has been created.

       • The CLONE_INTO_CGROUP flag also allows the creation of frozen
         child processes by spawning them into a frozen cgroup.  (See
         cgroups(7) for a description of the freezer controller.)

       • For threaded applications (or even thread implementations
         which make use of cgroups to limit individual threads), it is
         possible to establish a fixed cgroup layout before spawning
         each thread directly into its target cgroup.

CLONE_IO (since Linux 2.6.25)
       If CLONE_IO is set, then the new process shares an I/O context
       with the calling process.  If this flag is not set, then (as
       with fork(2)) the new process has its own I/O context.

       The I/O context is the I/O scope of the disk scheduler (i.e.,
       what the I/O scheduler uses to model scheduling of a process's
       I/O).  If processes share the same I/O context, they are treated
       as one by the I/O scheduler.  As a consequence, they get to
       share disk time.  For some I/O schedulers, if two processes
       share an I/O context, they will be allowed to interleave their
       disk access.  If several threads are doing I/O on behalf of the
       same process (aio_read(3), for instance), they should employ
       CLONE_IO to get better I/O performance.

       If the kernel is not configured with the CONFIG_BLOCK option,
       this flag is a no-op.

CLONE_NEWCGROUP (since Linux 4.6)
       Create the process in a new cgroup namespace.  If this flag is
       not set, then (as with fork(2)) the process is created in the
       same cgroup namespaces as the calling process.

       For further information on cgroup namespaces, see
```

```
              cgroup_namespaces(7).

              Only a privileged process (CAP_SYS_ADMIN) can employ
              CLONE_NEWCGROUP.

       CLONE_NEWIPC (since Linux 2.6.19)
              If CLONE_NEWIPC is set, then create the process in a new IPC
              namespace.  If this flag is not set, then (as with fork(2)), the
              process is created in the same IPC namespace as the calling
              process.

              For further information on IPC namespaces, see
              ipc_namespaces(7).

              Only a privileged process (CAP_SYS_ADMIN) can employ
              CLONE_NEWIPC.  This flag can't be specified in conjunction with
              CLONE_SYSVSEM.

       CLONE_NEWNET (since Linux 2.6.24)
              (The implementation of this flag was completed only by about
              Linux 2.6.29.)

              If CLONE_NEWNET is set, then create the process in a new network
              namespace.  If this flag is not set, then (as with fork(2)) the
              process is created in the same network namespace as the calling
              process.

              For further information on network namespaces, see
              network_namespaces(7).

              Only a privileged process (CAP_SYS_ADMIN) can employ
              CLONE_NEWNET.

       CLONE_NEWNS (since Linux 2.4.19)
              If CLONE_NEWNS is set, the cloned child is started in a new
              mount namespace, initialized with a copy of the namespace of the
              parent.  If CLONE_NEWNS is not set, the child lives in the same
              mount namespace as the parent.

              For further information on mount namespaces, see namespaces(7)
              and mount_namespaces(7).

              Only a privileged process (CAP_SYS_ADMIN) can employ
              CLONE_NEWNS.  It is not permitted to specify both CLONE_NEWNS
              and CLONE_FS in the same clone call.

       CLONE_NEWPID (since Linux 2.6.24)
              If CLONE_NEWPID is set, then create the process in a new PID
              namespace.  If this flag is not set, then (as with fork(2)) the
              process is created in the same PID namespace as the calling
              process.

              For further information on PID namespaces, see namespaces(7) and
              pid_namespaces(7).
```

```
              Only a privileged process (CAP_SYS_ADMIN) can employ
              CLONE_NEWPID.  This flag can't be specified in conjunction with
              CLONE_THREAD.

       CLONE_NEWUSER
              (This flag first became meaningful for clone() in Linux 2.6.23,
              the current clone() semantics were merged in Linux 3.5, and the
              final pieces to make the user namespaces completely usable were
              merged in Linux 3.8.)

              If CLONE_NEWUSER is set, then create the process in a new user
              namespace.  If this flag is not set, then (as with fork(2)) the
              process is created in the same user namespace as the calling
              process.

              For further information on user namespaces, see namespaces(7)
              and user_namespaces(7).

              Before Linux 3.8, use of CLONE_NEWUSER required that the caller
              have three capabilities: CAP_SYS_ADMIN, CAP_SETUID, and
              CAP_SETGID.  Starting with Linux 3.8, no privileges are needed
              to create a user namespace.

              This flag can't be specified in conjunction with CLONE_THREAD or
              CLONE_PARENT.  For security reasons, CLONE_NEWUSER cannot be
              specified in conjunction with CLONE_FS.

       CLONE_NEWUTS (since Linux 2.6.19)
              If CLONE_NEWUTS is set, then create the process in a new UTS
              namespace, whose identifiers are initialized by duplicating the
              identifiers from the UTS namespace of the calling process.  If
              this flag is not set, then (as with fork(2)) the process is
              created in the same UTS namespace as the calling process.

              For further information on UTS namespaces, see
              uts_namespaces(7).

              Only a privileged process (CAP_SYS_ADMIN) can employ
              CLONE_NEWUTS.

       CLONE_PARENT (since Linux 2.3.12)
              If CLONE_PARENT is set, then the parent of the new child (as
              returned by getppid(2)) will be the same as that of the calling
              process.

              If CLONE_PARENT is not set, then (as with fork(2)) the child's
              parent is the calling process.

              Note that it is the parent process, as returned by getppid(2),
              which is signaled when the child terminates, so that if
              CLONE_PARENT is set, then the parent of the calling process,
              rather than the calling process itself, is signaled.

              The CLONE_PARENT flag can't be used in clone calls by the global
              init process (PID 1 in the initial PID namespace) and init
```

```
       processes in other PID namespaces.  This restriction prevents
       the creation of multi-rooted process trees as well as the
       creation of unreapable zombies in the initial PID namespace.

CLONE_PARENT_SETTID (since Linux 2.5.49)
       Store the child thread ID at the location pointed to by
       parent_tid (clone()) or cl_args.parent_tid (clone3()) in the
       parent's memory.  (In Linux 2.5.32-2.5.48 there was a flag
       CLONE_SETTID that did this.)  The store operation completes
       before the clone call returns control to user space.

CLONE_PID (Linux 2.0 to Linux 2.5.15)
       If CLONE_PID is set, the child process is created with the same
       process ID as the calling process.  This is good for hacking the
       system, but otherwise of not much use.  From Linux 2.3.21
       onward, this flag could be specified only by the system boot
       process (PID 0).  The flag disappeared completely from the
       kernel sources in Linux 2.5.16.  Subsequently, the kernel
       silently ignored this bit if it was specified in the flags mask.
       Much later, the same bit was recycled for use as the CLONE_PIDFD
       flag.

CLONE_PIDFD (since Linux 5.2)
       If this flag is specified, a PID file descriptor referring to
       the child process is allocated and placed at a specified
       location in the parent's memory.  The close-on-exec flag is set
       on this new file descriptor.  PID file descriptors can be used
       for the purposes described in pidfd_open(2).

       • When using clone3(), the PID file descriptor is placed at the
         location pointed to by cl_args.pidfd.

       • When using clone(), the PID file descriptor is placed at the
         location pointed to by parent_tid.  Since the parent_tid
         argument is used to return the PID file descriptor,
         CLONE_PIDFD cannot be used with CLONE_PARENT_SETTID when
         calling clone().

       It is currently not possible to use this flag together with
       CLONE_THREAD.  This means that the process identified by the PID
       file descriptor will always be a thread group leader.

       If the obsolete CLONE_DETACHED flag is specified alongside
       CLONE_PIDFD when calling clone(), an error is returned.  An
       error also results if CLONE_DETACHED is specified when calling
       clone3().  This error behavior ensures that the bit
       corresponding to CLONE_DETACHED can be reused for further PID
       file descriptor features in the future.

CLONE_PTRACE (since Linux 2.2)
       If CLONE_PTRACE is specified, and the calling process is being
       traced, then trace the child also (see ptrace(2)).

CLONE_SETTLS (since Linux 2.5.32)
       The TLS (Thread Local Storage) descriptor is set to tls.
```

```
       The interpretation of tls and the resulting effect is
       architecture dependent.  On x86, tls is interpreted as a struct
       user_desc * (see set_thread_area(2)).  On x86-64 it is the new
       value to be set for the %fs base register (see the ARCH_SET_FS
       argument to arch_prctl(2)).  On architectures with a dedicated
       TLS register, it is the new value of that register.

       Use of this flag requires detailed knowledge and generally it
       should not be used except in libraries implementing threading.

CLONE_SIGHAND (since Linux 2.0)
       If CLONE_SIGHAND is set, the calling process and the child
       process share the same table of signal handlers.  If the calling
       process or child process calls sigaction(2) to change the
       behavior associated with a signal, the behavior is changed in
       the other process as well.  However, the calling process and
       child processes still have distinct signal masks and sets of
       pending signals.  So, one of them may block or unblock signals
       using sigprocmask(2) without affecting the other process.

       If CLONE_SIGHAND is not set, the child process inherits a copy
       of the signal handlers of the calling process at the time of the
       clone call.  Calls to sigaction(2) performed later by one of the
       processes have no effect on the other process.

       Since Linux 2.6.0, the flags mask must also include CLONE_VM if
       CLONE_SIGHAND is specified.

CLONE_STOPPED (since Linux 2.6.0)
       If CLONE_STOPPED is set, then the child is initially stopped (as
       though it was sent a SIGSTOP signal), and must be resumed by
       sending it a SIGCONT signal.

       This flag was deprecated from Linux 2.6.25 onward, and was
       removed altogether in Linux 2.6.38.  Since then, the kernel
       silently ignores it without error.  Starting with Linux 4.6, the
       same bit was reused for the CLONE_NEWCGROUP flag.

CLONE_SYSVSEM (since Linux 2.5.10)
       If CLONE_SYSVSEM is set, then the child and the calling process
       share a single list of System V semaphore adjustment (semadj)
       values (see semop(2)).  In this case, the shared list
       accumulates semadj values across all processes sharing the list,
       and semaphore adjustments are performed only when the last
       process that is sharing the list terminates (or ceases sharing
       the list using unshare(2)).  If this flag is not set, then the
       child has a separate semadj list that is initially empty.

CLONE_THREAD (since Linux 2.4.0)
       If CLONE_THREAD is set, the child is placed in the same thread
       group as the calling process.  To make the remainder of the
       discussion of CLONE_THREAD more readable, the term "thread" is
       used to refer to the processes within a thread group.
```

```
Thread groups were a feature added in Linux 2.4 to support the
POSIX threads notion of a set of threads that share a single
PID.  Internally, this shared PID is the so-called thread group
identifier (TGID) for the thread group.  Since Linux 2.4, calls
to getpid(2) return the TGID of the caller.

The threads within a group can be distinguished by their
(system-wide) unique thread IDs (TID).  A new thread's TID is
available as the function result returned to the caller, and a
thread can obtain its own TID using gettid(2).

When a clone call is made without specifying CLONE_THREAD, then
the resulting thread is placed in a new thread group whose TGID
is the same as the thread's TID.  This thread is the leader of
the new thread group.

A new thread created with CLONE_THREAD has the same parent
process as the process that made the clone call (i.e., like
CLONE_PARENT), so that calls to getppid(2) return the same value
for all of the threads in a thread group.  When a CLONE_THREAD
thread terminates, the thread that created it is not sent a
SIGCHLD (or other termination) signal; nor can the status of
such a thread be obtained using wait(2).  (The thread is said to
be detached.)

After all of the threads in a thread group terminate the parent
process of the thread group is sent a SIGCHLD (or other
termination) signal.

If any of the threads in a thread group performs an execve(2),
then all threads other than the thread group leader are
terminated, and the new program is executed in the thread group
leader.

If one of the threads in a thread group creates a child using
fork(2), then any thread in the group can wait(2) for that
child.

Since Linux 2.5.35, the flags mask must also include
CLONE_SIGHAND if CLONE_THREAD is specified (and note that, since
Linux 2.6.0, CLONE_SIGHAND also requires CLONE_VM to be
included).

Signal dispositions and actions are process-wide: if an
unhandled signal is delivered to a thread, then it will affect
(terminate, stop, continue, be ignored in) all members of the
thread group.

Each thread has its own signal mask, as set by sigprocmask(2).

A signal may be process-directed or thread-directed.  A process-
directed signal is targeted at a thread group (i.e., a TGID),
and is delivered to an arbitrarily selected thread from among
those that are not blocking the signal.  A signal may be
process-directed because it was generated by the kernel for
```

```
              reasons other than a hardware exception, or because it was sent
              using kill(2) or sigqueue(3).  A thread-directed signal is
              targeted at (i.e., delivered to) a specific thread.  A signal
              may be thread directed because it was sent using tgkill(2) or
              pthread_sigqueue(3), or because the thread executed a machine
              language instruction that triggered a hardware exception (e.g.,
              invalid memory access triggering SIGSEGV or a floating-point
              exception triggering SIGFPE).

              A call to sigpending(2) returns a signal set that is the union
              of the pending process-directed signals and the signals that are
              pending for the calling thread.

              If a process-directed signal is delivered to a thread group, and
              the thread group has installed a handler for the signal, then
              the handler is invoked in exactly one, arbitrarily selected
              member of the thread group that has not blocked the signal.  If
              multiple threads in a group are waiting to accept the same
              signal using sigwaitinfo(2), the kernel will arbitrarily select
              one of these threads to receive the signal.

       CLONE_UNTRACED (since Linux 2.5.46)
              If CLONE_UNTRACED is specified, then a tracing process cannot
              force CLONE_PTRACE on this child process.

       CLONE_VFORK (since Linux 2.2)
              If CLONE_VFORK is set, the execution of the calling process is
              suspended until the child releases its virtual memory resources
              via a call to execve(2) or _exit(2) (as with vfork(2)).

              If CLONE_VFORK is not set, then both the calling process and the
              child are schedulable after the call, and an application should
              not rely on execution occurring in any particular order.

       CLONE_VM (since Linux 2.0)
              If CLONE_VM is set, the calling process and the child process
              run in the same memory space.  In particular, memory writes
              performed by the calling process or by the child process are
              also visible in the other process.  Moreover, any memory mapping
              or unmapping performed with mmap(2) or munmap(2) by the child or
              calling process also affects the other process.

              If CLONE_VM is not set, the child process runs in a separate
              copy of the memory space of the calling process at the time of
              the clone call.  Memory writes or file mappings/unmappings
              performed by one of the processes do not affect the other, as
              with fork(2).

              If the CLONE_VM flag is specified and the CLONE_VFORK flag is
              not specified, then any alternate signal stack that was
              established by sigaltstack(2) is cleared in the child process.

RETURN VALUE
       On success, the thread ID of the child process is returned in the
       caller's thread of execution.  On failure, -1 is returned in the
```

```
       caller's context, no child process is created, and errno is set to
       indicate the error.

ERRORS
       EACCES (clone3() only)
              CLONE_INTO_CGROUP was specified in cl_args.flags, but the
              restrictions (described in cgroups(7)) on placing the child
              process into the version 2 cgroup referred to by cl_args.cgroup
              are not met.

       EAGAIN Too many processes are already running; see fork(2).

       EBUSY (clone3() only)
              CLONE_INTO_CGROUP was specified in cl_args.flags, but the file
              descriptor specified in cl_args.cgroup refers to a version 2
              cgroup in which a domain controller is enabled.

       EEXIST (clone3() only)
              One (or more) of the PIDs specified in set_tid already exists in
              the corresponding PID namespace.

       EINVAL Both CLONE_SIGHAND and CLONE_CLEAR_SIGHAND were specified in the
              flags mask.

       EINVAL CLONE_SIGHAND was specified in the flags mask, but CLONE_VM was
              not.  (Since Linux 2.6.0.)

       EINVAL CLONE_THREAD was specified in the flags mask, but CLONE_SIGHAND
              was not.  (Since Linux 2.5.35.)

       EINVAL CLONE_THREAD was specified in the flags mask, but the current
              process previously called unshare(2) with the CLONE_NEWPID flag
              or used setns(2) to reassociate itself with a PID namespace.

       EINVAL Both CLONE_FS and CLONE_NEWNS were specified in the flags mask.

       EINVAL (since Linux 3.9)
              Both CLONE_NEWUSER and CLONE_FS were specified in the flags
              mask.

       EINVAL Both CLONE_NEWIPC and CLONE_SYSVSEM were specified in the flags
              mask.

       EINVAL CLONE_NEWPID and one (or both) of CLONE_THREAD or CLONE_PARENT
              were specified in the flags mask.

       EINVAL CLONE_NEWUSER and CLONE_THREAD were specified in the flags mask.

       EINVAL (since Linux 2.6.32)
              CLONE_PARENT was specified, and the caller is an init process.

       EINVAL Returned by the glibc clone() wrapper function when fn or stack
              is specified as NULL.

       EINVAL CLONE_NEWIPC was specified in the flags mask, but the kernel was
```

```
              not configured with the CONFIG_SYSVIPC and CONFIG_IPC_NS
              options.

       EINVAL CLONE_NEWNET was specified in the flags mask, but the kernel was
              not configured with the CONFIG_NET_NS option.

       EINVAL CLONE_NEWPID was specified in the flags mask, but the kernel was
              not configured with the CONFIG_PID_NS option.

       EINVAL CLONE_NEWUSER was specified in the flags mask, but the kernel
              was not configured with the CONFIG_USER_NS option.

       EINVAL CLONE_NEWUTS was specified in the flags mask, but the kernel was
              not configured with the CONFIG_UTS_NS option.

       EINVAL stack is not aligned to a suitable boundary for this
              architecture.  For example, on aarch64, stack must be a multiple
              of 16.

       EINVAL (clone3() only)
              CLONE_DETACHED was specified in the flags mask.

       EINVAL (clone() only)
              CLONE_PIDFD was specified together with CLONE_DETACHED in the
              flags mask.

       EINVAL CLONE_PIDFD was specified together with CLONE_THREAD in the
              flags mask.

       EINVAL (clone() only)
              CLONE_PIDFD was specified together with CLONE_PARENT_SETTID in
              the flags mask.

       EINVAL (clone3() only)
              set_tid_size is greater than the number of nested PID
              namespaces.

       EINVAL (clone3() only)
              One of the PIDs specified in set_tid was an invalid.

       EINVAL (clone3() only)
              CLONE_THREAD or CLONE_PARENT was specified in the flags mask,
              but a signal was specified in exit_signal.

       EINVAL (AArch64 only, Linux 4.6 and earlier)
              stack was not aligned to a 128-bit boundary.

       ENOMEM Cannot allocate sufficient memory to allocate a task structure
              for the child, or to copy those parts of the caller's context
              that need to be copied.

       ENOSPC (since Linux 3.7)
              CLONE_NEWPID was specified in the flags mask, but the limit on
              the nesting depth of PID namespaces would have been exceeded;
              see pid_namespaces(7).
```

```
       ENOSPC (since Linux 4.9; beforehand EUSERS)
              CLONE_NEWUSER was specified in the flags mask, and the call
              would cause the limit on the number of nested user namespaces to
              be exceeded.  See user_namespaces(7).

              From Linux 3.11 to Linux 4.8, the error diagnosed in this case
              was EUSERS.

       ENOSPC (since Linux 4.9)
              One of the values in the flags mask specified the creation of a
              new user namespace, but doing so would have caused the limit
              defined by the corresponding file in /proc/sys/user to be
              exceeded.  For further details, see namespaces(7).

       EOPNOTSUPP (clone3() only)
              CLONE_INTO_CGROUP was specified in cl_args.flags, but the file
              descriptor specified in cl_args.cgroup refers to a version 2
              cgroup that is in the domain invalid state.

       EPERM  CLONE_NEWCGROUP, CLONE_NEWIPC, CLONE_NEWNET, CLONE_NEWNS,
              CLONE_NEWPID, or CLONE_NEWUTS was specified by an unprivileged
              process (process without CAP_SYS_ADMIN).

       EPERM  CLONE_PID was specified by a process other than process 0.
              (This error occurs only on Linux 2.5.15 and earlier.)

       EPERM  CLONE_NEWUSER was specified in the flags mask, but either the
              effective user ID or the effective group ID of the caller does
              not have a mapping in the parent namespace (see
              user_namespaces(7)).

       EPERM (since Linux 3.9)
              CLONE_NEWUSER was specified in the flags mask and the caller is
              in a chroot environment (i.e., the caller's root directory does
              not match the root directory of the mount namespace in which it
              resides).

       EPERM (clone3() only)
              set_tid_size was greater than zero, and the caller lacks the
              CAP_SYS_ADMIN capability in one or more of the user namespaces
              that own the corresponding PID namespaces.

       ERESTARTNOINTR (since Linux 2.6.17)
              System call was interrupted by a signal and will be restarted.
              (This can be seen only during a trace.)

       EUSERS (Linux 3.11 to Linux 4.8)
              CLONE_NEWUSER was specified in the flags mask, and the limit on
              the number of nested user namespaces would be exceeded.  See the
              discussion of the ENOSPC error above.

VERSIONS
       The glibc clone() wrapper function makes some changes in the memory
       pointed to by stack (changes required to set the stack up correctly for
```

```
       the child) before invoking the clone() system call.  So, in cases where
       clone() is used to recursively create children, do not use the buffer
       employed for the parent's stack as the stack of the child.

       On i386, clone() should not be called through vsyscall, but directly
       through int $0x80.

   C library/kernel differences
       The raw clone() system call corresponds more closely to fork(2) in that
       execution in the child continues from the point of the call.  As such,
       the fn and arg arguments of the clone() wrapper function are omitted.

       In contrast to the glibc wrapper, the raw clone() system call accepts
       NULL as a stack argument (and clone3() likewise allows cl_args.stack to
       be NULL).  In this case, the child uses a duplicate of the parent's
       stack.  (Copy-on-write semantics ensure that the child gets separate
       copies of stack pages when either process modifies the stack.)  In this
       case, for correct operation, the CLONE_VM option should not be
       specified.  (If the child shares the parent's memory because of the use
       of the CLONE_VM flag, then no copy-on-write duplication occurs and
       chaos is likely to result.)

       The order of the arguments also differs in the raw system call, and
       there are variations in the arguments across architectures, as detailed
       in the following paragraphs.

       The raw system call interface on x86-64 and some other architectures
       (including sh, tile, and alpha) is:

           long clone(unsigned long flags, void *stack,
                      int *parent_tid, int *child_tid,
                      unsigned long tls);

       On x86-32, and several other common architectures (including score,
       ARM, ARM 64, PA-RISC, arc, Power PC, xtensa, and MIPS), the order of
       the last two arguments is reversed:

           long clone(unsigned long flags, void *stack,
                     int *parent_tid, unsigned long tls,
                     int *child_tid);

       On the cris and s390 architectures, the order of the first two
       arguments is reversed:

           long clone(void *stack, unsigned long flags,
                      int *parent_tid, int *child_tid,
                      unsigned long tls);

       On the microblaze architecture, an additional argument is supplied:

           long clone(unsigned long flags, void *stack,
                      int stack_size,         /* Size of stack */
                      int *parent_tid, int *child_tid,
                      unsigned long tls);
```

```
   blackfin, m68k, and sparc
       The argument-passing conventions on blackfin, m68k, and sparc are
       different from the descriptions above.  For details, see the kernel
       (and glibc) source.

   ia64
       On ia64, a different interface is used:

           int __clone2(int (*fn)(void *),
                        void *stack_base, size_t stack_size,
                        int flags, void *arg, ...
                      /* pid_t *parent_tid, struct user_desc *tls,
                         pid_t *child_tid */ );

       The prototype shown above is for the glibc wrapper function; for the
       system call itself, the prototype can be described as follows (it is
       identical to the clone() prototype on microblaze):

           long clone2(unsigned long flags, void *stack_base,
                       int stack_size,         /* Size of stack */
                       int *parent_tid, int *child_tid,
                       unsigned long tls);

       __clone2() operates in the same way as clone(), except that stack_base
       points to the lowest address of the child's stack area, and stack_size
       specifies the size of the stack pointed to by stack_base.

STANDARDS
       Linux.

HISTORY
       clone3()
              Linux 5.3.

   Linux 2.4 and earlier
       In the Linux 2.4.x series, CLONE_THREAD generally does not make the
       parent of the new thread the same as the parent of the calling process.
       However, from Linux 2.4.7 to Linux 2.4.18 the CLONE_THREAD flag implied
       the CLONE_PARENT flag (as in Linux 2.6.0 and later).

       In Linux 2.4 and earlier, clone() does not take arguments parent_tid,
       tls, and child_tid.

NOTES
       One use of these system calls is to implement threads: multiple flows
       of control in a program that run concurrently in a shared address
       space.

       The kcmp(2) system call can be used to test whether two processes share
       various resources such as a file descriptor table, System V semaphore
       undo operations, or a virtual address space.

       Handlers registered using pthread_atfork(3) are not executed during a
       clone call.
```

```
BUGS
       GNU C library versions 2.3.4 up to and including 2.24 contained a
       wrapper function for getpid(2) that performed caching of PIDs.  This
       caching relied on support in the glibc wrapper for clone(), but
       limitations in the implementation meant that the cache was not up to
       date in some circumstances.  In particular, if a signal was delivered
       to the child immediately after the clone() call, then a call to
       getpid(2) in a handler for the signal could return the PID of the
       calling process ("the parent"), if the clone wrapper had not yet had a
       chance to update the PID cache in the child.  (This discussion ignores
       the case where the child was created using CLONE_THREAD, when getpid(2)
       should return the same value in the child and in the process that
       called clone(), since the caller and the child are in the same thread
       group.  The stale-cache problem also does not occur if the flags
       argument includes CLONE_VM.)  To get the truth, it was sometimes
       necessary to use code such as the following:

           #include <syscall.h>

           pid_t mypid;

           mypid = syscall(SYS_getpid);

       Because of the stale-cache problem, as well as other problems noted in
       getpid(2), the PID caching feature was removed in glibc 2.25.

EXAMPLES
       The following program demonstrates the use of clone() to create a child
       process that executes in a separate UTS namespace.  The child changes
       the hostname in its UTS namespace.  Both parent and child then display
       the system hostname, making it possible to see that the hostname
       differs in the UTS namespaces of the parent and child.  For an example
       of the use of this program, see setns(2).

       Within the sample program, we allocate the memory that is to be used
       for the child's stack using mmap(2) rather than malloc(3) for the
       following reasons:

       •  mmap(2) allocates a block of memory that starts on a page boundary
          and is a multiple of the page size.  This is useful if we want to
          establish a guard page (a page with protection PROT_NONE) at the end
          of the stack using mprotect(2).

       •  We can specify the MAP_STACK flag to request a mapping that is
          suitable for a stack.  For the moment, this flag is a no-op on
          Linux, but it exists and has effect on some other systems, so we
          should include it for portability.

   Program source
       #define _GNU_SOURCE
       #include <err.h>
       #include <sched.h>
       #include <signal.h>
       #include <stdint.h>
       #include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/utsname.h>
#include <sys/wait.h>
#include <unistd.h>

static int              /* Start function for cloned child */
childFunc(void *arg)
{
    struct utsname uts;

    /* Change hostname in UTS namespace of child. */

    if (sethostname(arg, strlen(arg)) == -1)
        err(EXIT_FAILURE, "sethostname");

    /* Retrieve and display hostname. */

    if (uname(&uts) == -1)
        err(EXIT_FAILURE, "uname");
    printf("uts.nodename in child:  %s\n", uts.nodename);

    /* Keep the namespace open for a while, by sleeping.
       This allows some experimentation--for example, another
       process might join the namespace. */

    sleep(200);

    return 0;           /* Child terminates now */
}

#define STACK_SIZE (1024 * 1024)    /* Stack size for cloned child */

int
main(int argc, char *argv[])
{
    char            *stack;         /* Start of stack buffer */
    char            *stackTop;      /* End of stack buffer */
    pid_t           pid;
    struct utsname  uts;

    if (argc < 2) {
        fprintf(stderr, "Usage: %s <child-hostname>\n", argv[0]);
        exit(EXIT_SUCCESS);
    }

    /* Allocate memory to be used for the stack of the child. */

    stack = mmap(NULL, STACK_SIZE, PROT_READ | PROT_WRITE,
                 MAP_PRIVATE | MAP_ANONYMOUS | MAP_STACK, -1, 0);
    if (stack == MAP_FAILED)
        err(EXIT_FAILURE, "mmap");

    stackTop = stack + STACK_SIZE;  /* Assume stack grows downward */
```

```
           /* Create child that has its own UTS namespace;
              child commences execution in childFunc(). */

           pid = clone(childFunc, stackTop, CLONE_NEWUTS | SIGCHLD, argv[1]);
           if (pid == -1)
               err(EXIT_FAILURE, "clone");
           printf("clone() returned %jd\n", (intmax_t) pid);

           /* Parent falls through to here */

           sleep(1);           /* Give child time to change its hostname */

           /* Display hostname in parent's UTS namespace. This will be
              different from hostname in child's UTS namespace. */

           if (uname(&uts) == -1)
               err(EXIT_FAILURE, "uname");
           printf("uts.nodename in parent: %s\n", uts.nodename);

           if (waitpid(pid, NULL, 0) == -1)    /* Wait for child */
               err(EXIT_FAILURE, "waitpid");
           printf("child has terminated\n");

           exit(EXIT_SUCCESS);
       }

SEE ALSO
       fork(2), futex(2), getpid(2), gettid(2), kcmp(2), mmap(2),
       pidfd_open(2), set_thread_area(2), set_tid_address(2), setns(2),
       tkill(2), unshare(2), wait(2), capabilities(7), namespaces(7),
       pthreads(7)

Linux man-pages 6.7              2024-02-18                          clone(2)
```

---

execve(2)

```
execve(2)                    System Calls Manual                    execve(2)

NAME
       execve - execute program

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       int execve(const char *pathname, char *const _Nullable argv[],
                  char *const _Nullable envp[]);

DESCRIPTION
       execve() executes the program referred to by pathname.  This causes the
```

```
program that is currently being run by the calling process to be
replaced with a new program, with newly initialized stack, heap, and
(initialized and uninitialized) data segments.

pathname must be either a binary executable, or a script starting with
a line of the form:

    #!interpreter [optional-arg]

For details of the latter case, see "Interpreter scripts" below.

argv is an array of pointers to strings passed to the new program as
its command-line arguments.  By convention, the first of these strings
(i.e., argv[0]) should contain the filename associated with the file
being executed.  The argv array must be terminated by a null pointer.
(Thus, in the new program, argv[argc] will be a null pointer.)

envp is an array of pointers to strings, conventionally of the form
key=value, which are passed as the environment of the new program.  The
envp array must be terminated by a null pointer.

This manual page describes the Linux system call in detail; for an
overview of the nomenclature and the many, often preferable,
standardised variants of this function provided by libc, including ones
that search the PATH environment variable, see exec(3).

The argument vector and environment can be accessed by the new
program's main function, when it is defined as:

    int main(int argc, char *argv[], char *envp[])

Note, however, that the use of a third argument to the main function is
not specified in POSIX.1; according to POSIX.1, the environment should
be accessed via the external variable environ(7).

execve() does not return on success, and the text, initialized data,
uninitialized data (bss), and stack of the calling process are
overwritten according to the contents of the newly loaded program.

If the current program is being ptraced, a SIGTRAP signal is sent to it
after a successful execve().

If the set-user-ID bit is set on the program file referred to by
pathname, then the effective user ID of the calling process is changed
to that of the owner of the program file.  Similarly, if the set-group-
ID bit is set on the program file, then the effective group ID of the
calling process is set to the group of the program file.

The aforementioned transformations of the effective IDs are not
performed (i.e., the set-user-ID and set-group-ID bits are ignored) if
any of the following is true:

•  the no_new_privs attribute is set for the calling thread (see
   prctl(2));
```

```
       • the underlying filesystem is mounted nosuid (the MS_NOSUID flag for
         mount(2)); or

       • the calling process is being ptraced.

       The capabilities of the program file (see capabilities(7)) are also
       ignored if any of the above are true.

       The effective user ID of the process is copied to the saved set-user-
       ID; similarly, the effective group ID is copied to the saved set-group-
       ID.  This copying takes place after any effective ID changes that occur
       because of the set-user-ID and set-group-ID mode bits.

       The process's real UID and real GID, as well as its supplementary group
       IDs, are unchanged by a call to execve().

       If the executable is an a.out dynamically linked binary executable
       containing shared-library stubs, the Linux dynamic linker ld.so(8) is
       called at the start of execution to bring needed shared objects into
       memory and link the executable with them.

       If the executable is a dynamically linked ELF executable, the
       interpreter named in the PT_INTERP segment is used to load the needed
       shared objects.  This interpreter is typically /lib/ld-linux.so.2 for
       binaries linked with glibc (see ld-linux.so(8)).

   Effect on process attributes
       All process attributes are preserved during an execve(), except the
       following:

       • The dispositions of any signals that are being caught are reset to
         the default (signal(7)).

       • Any alternate signal stack is not preserved (sigaltstack(2)).

       • Memory mappings are not preserved (mmap(2)).

       • Attached System V shared memory segments are detached (shmat(2)).

       • POSIX shared memory regions are unmapped (shm_open(3)).

       • Open POSIX message queue descriptors are closed (mq_overview(7)).

       • Any open POSIX named semaphores are closed (sem_overview(7)).

       • POSIX timers are not preserved (timer_create(2)).

       • Any open directory streams are closed (opendir(3)).

       • Memory locks are not preserved (mlock(2), mlockall(2)).

       • Exit handlers are not preserved (atexit(3), on_exit(3)).

       • The floating-point environment is reset to the default (see
         fenv(3)).
```

The process attributes in the preceding list are all specified in POSIX.1. The following Linux-specific process attributes are also not preserved during an execve():

- The process's "dumpable" attribute is set to the value 1, unless a set-user-ID program, a set-group-ID program, or a program with capabilities is being executed, in which case the dumpable flag may instead be reset to the value in /proc/sys/fs/suid_dumpable, in the circumstances described under PR_SET_DUMPABLE in prctl(2). Note that changes to the "dumpable" attribute may cause ownership of files in the process's /proc/pid directory to change to root:root, as described in proc(5).

- The prctl(2) PR_SET_KEEPCAPS flag is cleared.

- (Since Linux 2.4.36 / 2.6.23) If a set-user-ID or set-group-ID program is being executed, then the parent death signal set by prctl(2) PR_SET_PDEATHSIG flag is cleared.

- The process name, as set by prctl(2) PR_SET_NAME (and displayed by ps -o comm), is reset to the name of the new executable file.

- The SECBIT_KEEP_CAPS securebits flag is cleared. See capabilities(7).

- The termination signal is reset to SIGCHLD (see clone(2)).

- The file descriptor table is unshared, undoing the effect of the CLONE_FILES flag of clone(2).

Note the following further points:

- All threads other than the calling thread are destroyed during an execve(). Mutexes, condition variables, and other pthreads objects are not preserved.

- The equivalent of setlocale(LC_ALL, "C") is executed at program start-up.

- POSIX.1 specifies that the dispositions of any signals that are ignored or set to the default are left unchanged. POSIX.1 specifies one exception: if SIGCHLD is being ignored, then an implementation may leave the disposition unchanged or reset it to the default; Linux does the former.

- Any outstanding asynchronous I/O operations are canceled (aio_read(3), aio_write(3)).

- For the handling of capabilities during execve(), see capabilities(7).

- By default, file descriptors remain open across an execve(). File descriptors that are marked close-on-exec are closed; see the description of FD_CLOEXEC in fcntl(2). (If a file descriptor is

```
        closed, this will cause the release of all record locks obtained on
        the underlying file by this process.  See fcntl(2) for details.)
        POSIX.1 says that if file descriptors 0, 1, and 2 would otherwise be
        closed after a successful execve(), and the process would gain
        privilege because the set-user-ID or set-group-ID mode bit was set
        on the executed file, then the system may open an unspecified file
        for each of these file descriptors.  As a general principle, no
        portable program, whether privileged or not, can assume that these
        three file descriptors will remain closed across an execve().

Interpreter scripts
    An interpreter script is a text file that has execute permission
    enabled and whose first line is of the form:

        #!interpreter [optional-arg]

    The interpreter must be a valid pathname for an executable file.

    If the pathname argument of execve() specifies an interpreter script,
    then interpreter will be invoked with the following arguments:

        interpreter [optional-arg] pathname arg...

    where pathname is the pathname of the file specified as the first
    argument of execve(), and arg...  is the series of words pointed to by
    the argv argument of execve(), starting at argv[1].  Note that there is
    no way to get the argv[0] that was passed to the execve() call.

    For portable use, optional-arg should either be absent, or be specified
    as a single word (i.e., it should not contain white space); see NOTES
    below.

    Since Linux 2.6.28, the kernel permits the interpreter of a script to
    itself be a script.  This permission is recursive, up to a limit of
    four recursions, so that the interpreter may be a script which is
    interpreted by a script, and so on.

Limits on size of arguments and environment
    Most UNIX implementations impose some limit on the total size of the
    command-line argument (argv) and environment (envp) strings that may be
    passed to a new program.  POSIX.1 allows an implementation to advertise
    this limit using the ARG_MAX constant (either defined in <limits.h> or
    available at run time using the call sysconf(_SC_ARG_MAX)).

    Before Linux 2.6.23, the memory used to store the environment and
    argument strings was limited to 32 pages (defined by the kernel
    constant MAX_ARG_PAGES).  On architectures with a 4-kB page size, this
    yields a maximum size of 128 kB.

    On Linux 2.6.23 and later, most architectures support a size limit
    derived from the soft RLIMIT_STACK resource limit (see getrlimit(2))
    that is in force at the time of the execve() call.  (Architectures with
    no memory management unit are excepted: they maintain the limit that
    was in effect before Linux 2.6.23.)  This change allows programs to
    have a much larger argument and/or environment list.  For these
```

```
       architectures, the total size is limited to 1/4 of the allowed stack
       size.  (Imposing the 1/4-limit ensures that the new program always has
       some stack space.)  Additionally, the total size is limited to 3/4 of
       the value of the kernel constant _STK_LIM (8 MiB).  Since Linux 2.6.25,
       the kernel also places a floor of 32 pages on this size limit, so that,
       even when RLIMIT_STACK is set very low, applications are guaranteed to
       have at least as much argument and environment space as was provided by
       Linux 2.6.22 and earlier.  (This guarantee was not provided in Linux
       2.6.23 and 2.6.24.)  Additionally, the limit per string is 32 pages
       (the kernel constant MAX_ARG_STRLEN), and the maximum number of strings
       is 0x7FFFFFFF.

RETURN VALUE
       On success, execve() does not return, on error -1 is returned, and
       errno is set to indicate the error.

ERRORS
       E2BIG  The total number of bytes in the environment (envp) and argument
              list (argv) is too large, an argument or environment string is
              too long, or the full pathname of the executable is too long.
              The terminating null byte is counted as part of the string
              length.

       EACCES Search permission is denied on a component of the path prefix of
              pathname or the name of a script interpreter.  (See also
              path_resolution(7).)

       EACCES The file or a script interpreter is not a regular file.

       EACCES Execute permission is denied for the file or a script or ELF
              interpreter.

       EACCES The filesystem is mounted noexec.

       EAGAIN (since Linux 3.1)
              Having changed its real UID using one of the set*uid() calls,
              the caller was-and is now still-above its RLIMIT_NPROC resource
              limit (see setrlimit(2)).  For a more detailed explanation of
              this error, see NOTES.

       EFAULT pathname or one of the pointers in the vectors argv or envp
              points outside your accessible address space.

       EINVAL An ELF executable had more than one PT_INTERP segment (i.e.,
              tried to name more than one interpreter).

       EIO    An I/O error occurred.

       EISDIR An ELF interpreter was a directory.

       ELIBBAD
              An ELF interpreter was not in a recognized format.

       ELOOP  Too many symbolic links were encountered in resolving pathname
              or the name of a script or ELF interpreter.
```

```
       ELOOP  The maximum recursion limit was reached during recursive script
              interpretation (see "Interpreter scripts", above).  Before Linux
              3.8, the error produced for this case was ENOEXEC.

       EMFILE The per-process limit on the number of open file descriptors has
              been reached.

       ENAMETOOLONG
              pathname is too long.

       ENFILE The system-wide limit on the total number of open files has been
              reached.

       ENOENT The file pathname or a script or ELF interpreter does not exist.

       ENOEXEC
              An executable is not in a recognized format, is for the wrong
              architecture, or has some other format error that means it
              cannot be executed.

       ENOMEM Insufficient kernel memory was available.

       ENOTDIR
              A component of the path prefix of pathname or a script or ELF
              interpreter is not a directory.

       EPERM  The filesystem is mounted nosuid, the user is not the superuser,
              and the file has the set-user-ID or set-group-ID bit set.

       EPERM  The process is being traced, the user is not the superuser and
              the file has the set-user-ID or set-group-ID bit set.

       EPERM  A "capability-dumb" applications would not obtain the full set
              of permitted capabilities granted by the executable file.  See
              capabilities(7).

       ETXTBSY
              The specified executable was open for writing by one or more
              processes.

VERSIONS
       POSIX does not document the #! behavior, but it exists (with some
       variations) on other UNIX systems.

       On Linux, argv and envp can be specified as NULL.  In both cases, this
       has the same effect as specifying the argument as a pointer to a list
       containing a single null pointer.  Do not take advantage of this
       nonstandard and nonportable misfeature!  On many other UNIX systems,
       specifying argv as NULL will result in an error (EFAULT).  Some other
       UNIX systems treat the envp==NULL case the same as Linux.

       POSIX.1 says that values returned by sysconf(3) should be invariant
       over the lifetime of a process.  However, since Linux 2.6.23, if the
       RLIMIT_STACK resource limit changes, then the value reported by
```

```
       _SC_ARG_MAX will also change, to reflect the fact that the limit on
       space for holding command-line arguments and environment variables has
       changed.

   Interpreter scripts
       The kernel imposes a maximum length on the text that follows the "#!"
       characters at the start of a script; characters beyond the limit are
       ignored.  Before Linux 5.1, the limit is 127 characters.  Since Linux
       5.1, the limit is 255 characters.

       The semantics of the optional-arg argument of an interpreter script
       vary across implementations.  On Linux, the entire string following the
       interpreter name is passed as a single argument to the interpreter, and
       this string can include white space.  However, behavior differs on some
       other systems.  Some systems use the first white space to terminate
       optional-arg.  On some systems, an interpreter script can have multiple
       arguments, and white spaces in optional-arg are used to delimit the
       arguments.

       Linux (like most other modern UNIX systems) ignores the set-user-ID and
       set-group-ID bits on scripts.

STANDARDS
       POSIX.1-2008.

HISTORY
       POSIX.1-2001, SVr4, 4.3BSD.

       With UNIX V6, the argument list of an exec() call was ended by 0, while
       the argument list of main was ended by -1.  Thus, this argument list
       was not directly usable in a further exec() call.  Since UNIX V7, both
       are NULL.

NOTES
       One sometimes sees execve() (and the related functions described in
       exec(3)) described as "executing a new process" (or similar).  This is
       a highly misleading description: there is no new process; many
       attributes of the calling process remain unchanged (in particular, its
       PID).  All that execve() does is arrange for an existing process (the
       calling process) to execute a new program.

       Set-user-ID and set-group-ID processes can not be ptrace(2)d.

       The result of mounting a filesystem nosuid varies across Linux kernel
       versions: some will refuse execution of set-user-ID and set-group-ID
       executables when this would give the user powers they did not have
       already (and return EPERM), some will just ignore the set-user-ID and
       set-group-ID bits and exec() successfully.

       In most cases where execve() fails, control returns to the original
       executable image, and the caller of execve() can then handle the error.
       However, in (rare) cases (typically caused by resource exhaustion),
       failure may occur past the point of no return: the original executable
       image has been torn down, but the new image could not be completely
       built.  In such cases, the kernel kills the process with a SIGSEGV
```

```
       (SIGKILL until Linux 3.17) signal.

   execve() and EAGAIN
       A more detailed explanation of the EAGAIN error that can occur (since
       Linux 3.1) when calling execve() is as follows.

       The EAGAIN error can occur when a preceding call to setuid(2),
       setreuid(2), or setresuid(2) caused the real user ID of the process to
       change, and that change caused the process to exceed its RLIMIT_NPROC
       resource limit (i.e., the number of processes belonging to the new real
       UID exceeds the resource limit).  From Linux 2.6.0 to Linux 3.0, this
       caused the set*uid() call to fail.  (Before Linux 2.6, the resource
       limit was not imposed on processes that changed their user IDs.)

       Since Linux 3.1, the scenario just described no longer causes the
       set*uid() call to fail, because it too often led to security holes
       where buggy applications didn't check the return status and assumed
       that-if the caller had root privileges-the call would always succeed.
       Instead, the set*uid() calls now successfully change the real UID, but
       the kernel sets an internal flag, named PF_NPROC_EXCEEDED, to note that
       the RLIMIT_NPROC resource limit has been exceeded.  If the
       PF_NPROC_EXCEEDED flag is set and the resource limit is still exceeded
       at the time of a subsequent execve() call, that call fails with the
       error EAGAIN.  This kernel logic ensures that the RLIMIT_NPROC resource
       limit is still enforced for the common privileged daemon workflow-
       namely, fork(2) + set*uid() + execve().

       If the resource limit was not still exceeded at the time of the
       execve() call (because other processes belonging to this real UID
       terminated between the set*uid() call and the execve() call), then the
       execve() call succeeds and the kernel clears the PF_NPROC_EXCEEDED
       process flag.  The flag is also cleared if a subsequent call to fork(2)
       by this process succeeds.

EXAMPLES
       The following program is designed to be execed by the second program
       below.  It just echoes its command-line arguments, one per line.

           /* myecho.c */

           #include <stdio.h>
           #include <stdlib.h>

           int
           main(int argc, char *argv[])
           {
               for (size_t j = 0; j < argc; j++)
                   printf("argv[%zu]: %s\n", j, argv[j]);

               exit(EXIT_SUCCESS);
           }

       This program can be used to exec the program named in its command-line
       argument:
```

```
           /* execve.c */

           #include <stdio.h>
           #include <stdlib.h>
           #include <unistd.h>

           int
           main(int argc, char *argv[])
           {
               static char *newargv[] = { NULL, "hello", "world", NULL };
               static char *newenviron[] = { NULL };

               if (argc != 2) {
                   fprintf(stderr, "Usage: %s <file-to-exec>\n", argv[0]);
                   exit(EXIT_FAILURE);
               }

               newargv[0] = argv[1];

               execve(argv[1], newargv, newenviron);
               perror("execve");   /* execve() returns only on error */
               exit(EXIT_FAILURE);
           }

       We can use the second program to exec the first as follows:

           $ cc myecho.c -o myecho
           $ cc execve.c -o execve
           $ ./execve ./myecho
           argv[0]: ./myecho
           argv[1]: hello
           argv[2]: world

       We can also use these programs to demonstrate the use of a script
       interpreter.  To do this we create a script whose "interpreter" is our
       myecho program:

           $ cat > script
           #!./myecho script-arg
           ^D
           $ chmod +x script

       We can then use our program to exec the script:

           $ ./execve ./script
           argv[0]: ./myecho
           argv[1]: script-arg
           argv[2]: ./script
           argv[3]: hello
           argv[4]: world

SEE ALSO
       chmod(2), execveat(2), fork(2), get_robust_list(2), ptrace(2), exec(3),
       fexecve(3), getauxval(3), getopt(3), system(3), capabilities(7),
       credentials(7), environ(7), path_resolution(7), ld.so(8)
```

```
Linux man-pages 6.7              2023-11-01                          execve(2)
```

---

wait4(2)

```
wait4(2)                      System Calls Manual                      wait4(2)

NAME
       wait3, wait4 - wait for process to change state, BSD style

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/wait.h>

       pid_t wait3(int *_Nullable wstatus, int options,
                   struct rusage *_Nullable rusage);
       pid_t wait4(pid_t pid, int *_Nullable wstatus, int options,
                   struct rusage *_Nullable rusage);

   Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

       wait3():
           Since glibc 2.26:
               _DEFAULT_SOURCE
                   || (_XOPEN_SOURCE >= 500 &&
                       ! (_POSIX_C_SOURCE >= 200112L
                          || _XOPEN_SOURCE >= 600))
           From glibc 2.19 to glibc 2.25:
               _DEFAULT_SOURCE || _XOPEN_SOURCE >= 500
           glibc 2.19 and earlier:
               _BSD_SOURCE || _XOPEN_SOURCE >= 500

       wait4():
           Since glibc 2.19:
               _DEFAULT_SOURCE
           glibc 2.19 and earlier:
               _BSD_SOURCE

DESCRIPTION
       These functions are nonstandard; in new programs, the use of waitpid(2)
       or waitid(2) is preferable.

       The wait3() and wait4() system calls are similar to waitpid(2), but
       additionally return resource usage information about the child in the
       structure pointed to by rusage.

       Other than the use of the rusage argument, the following wait3() call:

           wait3(wstatus, options, rusage);

       is equivalent to:
```

```
           waitpid(-1, wstatus, options);

       Similarly, the following wait4() call:

           wait4(pid, wstatus, options, rusage);

       is equivalent to:

           waitpid(pid, wstatus, options);

       In other words, wait3() waits of any child, while wait4() can be used
       to select a specific child, or children, on which to wait.  See wait(2)
       for further details.

       If rusage is not NULL, the struct rusage to which it points will be
       filled with accounting information about the child.  See getrusage(2)
       for details.

RETURN VALUE
       As for waitpid(2).

ERRORS
       As for waitpid(2).

STANDARDS
       None.

HISTORY
       4.3BSD.

       SUSv1 included a specification of wait3(); SUSv2 included wait3(), but
       marked it LEGACY; SUSv3 removed it.

       Including <sys/time.h> is not required these days, but increases
       portability.  (Indeed, <sys/resource.h> defines the rusage structure
       with fields of type struct timeval defined in <sys/time.h>.)

   C library/kernel differences
       On Linux, wait3() is a library function implemented on top of the
       wait4() system call.

SEE ALSO
       fork(2), getrusage(2), sigaction(2), signal(2), wait(2), signal(7)

Linux man-pages 6.7              2023-10-31                          wait4(2)
```

---

signal(7)

```
signal(7)             Miscellaneous Information Manual             signal(7)

NAME
       signal - overview of signals
```

```
DESCRIPTION
       Linux supports both POSIX reliable signals (hereinafter "standard
       signals") and POSIX real-time signals.

   Signal dispositions
       Each signal has a current disposition, which determines how the process
       behaves when it is delivered the signal.

       The entries in the "Action" column of the table below specify the
       default disposition for each signal, as follows:

       Term   Default action is to terminate the process.

       Ign    Default action is to ignore the signal.

       Core   Default action is to terminate the process and dump core (see
              core(5)).

       Stop   Default action is to stop the process.

       Cont   Default action is to continue the process if it is currently
              stopped.

       A process can change the disposition of a signal using sigaction(2) or
       signal(2).  (The latter is less portable when establishing a signal
       handler; see signal(2) for details.)  Using these system calls, a
       process can elect one of the following behaviors to occur on delivery
       of the signal: perform the default action; ignore the signal; or catch
       the signal with a signal handler, a programmer-defined function that is
       automatically invoked when the signal is delivered.

       By default, a signal handler is invoked on the normal process stack.
       It is possible to arrange that the signal handler uses an alternate
       stack; see sigaltstack(2) for a discussion of how to do this and when
       it might be useful.

       The signal disposition is a per-process attribute: in a multithreaded
       application, the disposition of a particular signal is the same for all
       threads.

       A child created via fork(2) inherits a copy of its parent's signal
       dispositions.  During an execve(2), the dispositions of handled signals
       are reset to the default; the dispositions of ignored signals are left
       unchanged.

   Sending a signal
       The following system calls and library functions allow the caller to
       send a signal:

       raise(3)
              Sends a signal to the calling thread.

       kill(2)
              Sends a signal to a specified process, to all members of a
```

```
            specified process group, or to all processes on the system.

    pidfd_send_signal(2)
            Sends a signal to a process identified by a PID file descriptor.

    killpg(3)
            Sends a signal to all of the members of a specified process
            group.

    pthread_kill(3)
            Sends a signal to a specified POSIX thread in the same process
            as the caller.

    tgkill(2)
            Sends a signal to a specified thread within a specific process.
            (This is the system call used to implement pthread_kill(3).)

    sigqueue(3)
            Sends a real-time signal with accompanying data to a specified
            process.

Waiting for a signal to be caught
    The following system calls suspend execution of the calling thread
    until a signal is caught (or an unhandled signal terminates the
    process):

    pause(2)
            Suspends execution until any signal is caught.

    sigsuspend(2)
            Temporarily changes the signal mask (see below) and suspends
            execution until one of the unmasked signals is caught.

Synchronously accepting a signal
    Rather than asynchronously catching a signal via a signal handler, it
    is possible to synchronously accept the signal, that is, to block
    execution until the signal is delivered, at which point the kernel
    returns information about the signal to the caller.  There are two
    general ways to do this:

    •  sigwaitinfo(2), sigtimedwait(2), and sigwait(3) suspend execution
       until one of the signals in a specified set is delivered.  Each of
       these calls returns information about the delivered signal.

    •  signalfd(2) returns a file descriptor that can be used to read
       information about signals that are delivered to the caller.  Each
       read(2) from this file descriptor blocks until one of the signals in
       the set specified in the signalfd(2) call is delivered to the
       caller.  The buffer returned by read(2) contains a structure
       describing the signal.

Signal mask and pending signals
    A signal may be blocked, which means that it will not be delivered
    until it is later unblocked.  Between the time when it is generated and
    when it is delivered a signal is said to be pending.
```

```
    Each thread in a process has an independent signal mask, which
    indicates the set of signals that the thread is currently blocking.  A
    thread can manipulate its signal mask using pthread_sigmask(3).  In a
    traditional single-threaded application, sigprocmask(2) can be used to
    manipulate the signal mask.

    A child created via fork(2) inherits a copy of its parent's signal
    mask; the signal mask is preserved across execve(2).

    A signal may be process-directed or thread-directed.  A process-
    directed signal is one that is targeted at (and thus pending for) the
    process as a whole.  A signal may be process-directed because it was
    generated by the kernel for reasons other than a hardware exception, or
    because it was sent using kill(2) or sigqueue(3).  A thread-directed
    signal is one that is targeted at a specific thread.  A signal may be
    thread-directed because it was generated as a consequence of executing
    a specific machine-language instruction that triggered a hardware
    exception (e.g., SIGSEGV for an invalid memory access, or SIGFPE for a
    math error), or because it was targeted at a specific thread using
    interfaces such as tgkill(2) or pthread_kill(3).

    A process-directed signal may be delivered to any one of the threads
    that does not currently have the signal blocked.  If more than one of
    the threads has the signal unblocked, then the kernel chooses an
    arbitrary thread to which to deliver the signal.

    A thread can obtain the set of signals that it currently has pending
    using sigpending(2).  This set will consist of the union of the set of
    pending process-directed signals and the set of signals pending for the
    calling thread.

    A child created via fork(2) initially has an empty pending signal set;
    the pending signal set is preserved across an execve(2).

Execution of signal handlers
    Whenever there is a transition from kernel-mode to user-mode execution
    (e.g., on return from a system call or scheduling of a thread onto the
    CPU), the kernel checks whether there is a pending unblocked signal for
    which the process has established a signal handler.  If there is such a
    pending signal, the following steps occur:

    (1)  The kernel performs the necessary preparatory steps for execution
         of the signal handler:

         (1.1)  The signal is removed from the set of pending signals.

         (1.2)  If the signal handler was installed by a call to
                sigaction(2) that specified the SA_ONSTACK flag and the
                thread has defined an alternate signal stack (using
                sigaltstack(2)), then that stack is installed.

         (1.3)  Various pieces of signal-related context are saved into a
                special frame that is created on the stack.  The saved
                information includes:
```

```
                - the program counter register (i.e., the address of the
                  next instruction in the main program that should be
                  executed when the signal handler returns);

                - architecture-specific register state required for
                  resuming the interrupted program;

                - the thread's current signal mask;

                - the thread's alternate signal stack settings.

                (If the signal handler was installed using the sigaction(2)
                SA_SIGINFO flag, then the above information is accessible
                via the ucontext_t object that is pointed to by the third
                argument of the signal handler.)

        (1.4)   Any signals specified in act->sa_mask when registering the
                handler with sigprocmask(2) are added to the thread's
                signal mask.  The signal being delivered is also added to
                the signal mask, unless SA_NODEFER was specified when
                registering the handler.  These signals are thus blocked
                while the handler executes.

    (2) The kernel constructs a frame for the signal handler on the stack.
        The kernel sets the program counter for the thread to point to the
        first instruction of the signal handler function, and configures
        the return address for that function to point to a piece of user-
        space code known as the signal trampoline (described in
        sigreturn(2)).

    (3) The kernel passes control back to user-space, where execution
        commences at the start of the signal handler function.

    (4) When the signal handler returns, control passes to the signal
        trampoline code.

    (5) The signal trampoline calls sigreturn(2), a system call that uses
        the information in the stack frame created in step 1 to restore
        the thread to its state before the signal handler was called.  The
        thread's signal mask and alternate signal stack settings are
        restored as part of this procedure.  Upon completion of the call
        to sigreturn(2), the kernel transfers control back to user space,
        and the thread recommences execution at the point where it was
        interrupted by the signal handler.

    Note that if the signal handler does not return (e.g., control is
    transferred out of the handler using siglongjmp(3), or the handler
    executes a new program with execve(2)), then the final step is not
    performed.  In particular, in such scenarios it is the programmer's
    responsibility to restore the state of the signal mask (using
    sigprocmask(2)), if it is desired to unblock the signals that were
    blocked on entry to the signal handler.  (Note that siglongjmp(3) may
    or may not restore the signal mask, depending on the savesigs value
    that was specified in the corresponding call to sigsetjmp(3).)
```

```
    From the kernel's point of view, execution of the signal handler code
    is exactly the same as the execution of any other user-space code.
    That is to say, the kernel does not record any special state
    information indicating that the thread is currently executing inside a
    signal handler.  All necessary state information is maintained in user-
    space registers and the user-space stack.  The depth to which nested
    signal handlers may be invoked is thus limited only by the user-space
    stack (and sensible software design!).

Standard signals
    Linux supports the standard signals listed below.  The second column of
    the table indicates which standard (if any) specified the signal:
    "P1990" indicates that the signal is described in the original
    POSIX.1-1990 standard; "P2001" indicates that the signal was added in
    SUSv2 and POSIX.1-2001.
    Signal      Standard   Action   Comment

    SIGABRT      P1990      Core    Abort signal from abort(3)
    SIGALRM      P1990      Term    Timer signal from alarm(2)
    SIGBUS       P2001      Core    Bus error (bad memory access)
    SIGCHLD      P1990      Ign     Child stopped or terminated
    SIGCLD         -        Ign     A synonym for SIGCHLD
    SIGCONT      P1990      Cont    Continue if stopped
    SIGEMT         -        Term    Emulator trap
    SIGFPE       P1990      Core    Floating-point exception
    SIGHUP       P1990      Term    Hangup detected on controlling terminal
                                    or death of controlling process
    SIGILL       P1990      Core    Illegal Instruction
    SIGINFO        -                A synonym for SIGPWR
    SIGINT       P1990      Term    Interrupt from keyboard
    SIGIO          -        Term    I/O now possible (4.2BSD)
    SIGIOT         -        Core    IOT trap. A synonym for SIGABRT
    SIGKILL      P1990      Term    Kill signal
    SIGLOST        -        Term    File lock lost (unused)
    SIGPIPE      P1990      Term    Broken pipe: write to pipe with no
                                    readers; see pipe(7)
    SIGPOLL      P2001      Term    Pollable event (Sys V);
                                    synonym for SIGIO
    SIGPROF      P2001      Term    Profiling timer expired
    SIGPWR         -        Term    Power failure (System V)
    SIGQUIT      P1990      Core    Quit from keyboard
    SIGSEGV      P1990      Core    Invalid memory reference
    SIGSTKFLT      -        Term    Stack fault on coprocessor (unused)
    SIGSTOP      P1990      Stop    Stop process
    SIGTSTP      P1990      Stop    Stop typed at terminal
    SIGSYS       P2001      Core    Bad system call (SVr4);
                                    see also seccomp(2)
    SIGTERM      P1990      Term    Termination signal
    SIGTRAP      P2001      Core    Trace/breakpoint trap
    SIGTTIN      P1990      Stop    Terminal input for background process
    SIGTTOU      P1990      Stop    Terminal output for background process
    SIGUNUSED      -        Core    Synonymous with SIGSYS
    SIGURG       P2001      Ign     Urgent condition on socket (4.2BSD)
    SIGUSR1      P1990      Term    User-defined signal 1
```

```
       SIGUSR2      P1990      Term    User-defined signal 2
       SIGVTALRM    P2001      Term    Virtual alarm clock (4.2BSD)
       SIGXCPU      P2001      Core    CPU time limit exceeded (4.2BSD);
                                       see setrlimit(2)
       SIGXFSZ      P2001      Core    File size limit exceeded (4.2BSD);
                                       see setrlimit(2)
       SIGWINCH       -        Ign     Window resize signal (4.3BSD, Sun)

       The signals SIGKILL and SIGSTOP cannot be caught, blocked, or ignored.

       Up to and including Linux 2.2, the default behavior for SIGSYS,
       SIGXCPU, SIGXFSZ, and (on architectures other than SPARC and MIPS)
       SIGBUS was to terminate the process (without a core dump).  (On some
       other UNIX systems the default action for SIGXCPU and SIGXFSZ is to
       terminate the process without a core dump.)  Linux 2.4 conforms to the
       POSIX.1-2001 requirements for these signals, terminating the process
       with a core dump.

       SIGEMT is not specified in POSIX.1-2001, but nevertheless appears on
       most other UNIX systems, where its default action is typically to
       terminate the process with a core dump.

       SIGPWR (which is not specified in POSIX.1-2001) is typically ignored by
       default on those other UNIX systems where it appears.

       SIGIO (which is not specified in POSIX.1-2001) is ignored by default on
       several other UNIX systems.

   Queueing and delivery semantics for standard signals
       If multiple standard signals are pending for a process, the order in
       which the signals are delivered is unspecified.

       Standard signals do not queue.  If multiple instances of a standard
       signal are generated while that signal is blocked, then only one
       instance of the signal is marked as pending (and the signal will be
       delivered just once when it is unblocked).  In the case where a
       standard signal is already pending, the siginfo_t structure (see
       sigaction(2)) associated with that signal is not overwritten on arrival
       of subsequent instances of the same signal.  Thus, the process will
       receive the information associated with the first instance of the
       signal.

   Signal numbering for standard signals
       The numeric value for each signal is given in the table below.  As
       shown in the table, many signals have different numeric values on
       different architectures.  The first numeric value in each table row
       shows the signal number on x86, ARM, and most other architectures; the
       second value is for Alpha and SPARC; the third is for MIPS; and the
       last is for PARISC.  A dash (-) denotes that a signal is absent on the
       corresponding architecture.
       Signal        x86/ARM     Alpha/   MIPS   PARISC   Notes
                  most others   SPARC

       SIGHUP           1          1       1       1
       SIGINT           2          2       2       2
```

```
       SIGQUIT          3          3       3       3
       SIGILL           4          4       4       4
       SIGTRAP          5          5       5       5
       SIGABRT          6          6       6       6
       SIGIOT           6          6       6       6
       SIGBUS           7         10      10      10
       SIGEMT           -          7       7       -
       SIGFPE           8          8       8       8
       SIGKILL          9          9       9       9
       SIGUSR1         10         30      16      16
       SIGSEGV         11         11      11      11
       SIGUSR2         12         31      17      17
       SIGPIPE         13         13      13      13
       SIGALRM         14         14      14      14
       SIGTERM         15         15      15      15
       SIGSTKFLT       16         -       -        7
       SIGCHLD         17         20      18      18
       SIGCLD           -         -       18      -
       SIGCONT         18         19      25      26
       SIGSTOP         19         17      23      24
       SIGTSTP         20         18      24      25
       SIGTTIN         21         21      26      27
       SIGTTOU         22         22      27      28
       SIGURG          23         16      21      29
       SIGXCPU         24         24      30      12
       SIGXFSZ         25         25      31      30
       SIGVTALRM       26         26      28      20
       SIGPROF         27         27      29      21
       SIGWINCH        28         28      20      23
       SIGIO           29         23      22      22
       SIGPOLL                                            Same as SIGIO
       SIGPWR          30        29/-     19      19
       SIGINFO          -        29/-     -       -
       SIGLOST          -        -/29     -       -
       SIGSYS          31         12      12      31
       SIGUNUSED       31         -       -       31

       Note the following:

       • Where defined, SIGUNUSED is synonymous with SIGSYS.  Since glibc
         2.26, SIGUNUSED is no longer defined on any architecture.

       • Signal 29 is SIGINFO/SIGPWR (synonyms for the same value) on Alpha
         but SIGLOST on SPARC.

   Real-time signals
       Starting with Linux 2.2, Linux supports real-time signals as originally
       defined in the POSIX.1b real-time extensions (and now included in
       POSIX.1-2001).  The range of supported real-time signals is defined by
       the macros SIGRTMIN and SIGRTMAX.  POSIX.1-2001 requires that an
       implementation support at least _POSIX_RTSIG_MAX (8) real-time signals.

       The Linux kernel supports a range of 33 different real-time signals,
       numbered 32 to 64.  However, the glibc POSIX threads implementation
       internally uses two (for NPTL) or three (for LinuxThreads) real-time
```

signals (see pthreads(7)), and adjusts the value of SIGRTMIN suitably (to 34 or 35). Because the range of available real-time signals varies according to the glibc threading implementation (and this variation can occur at run time according to the available kernel and glibc), and indeed the range of real-time signals varies across UNIX systems, programs should never refer to real-time signals using hard-coded numbers, but instead should always refer to real-time signals using the notation SIGRTMIN+n, and include suitable (run-time) checks that SIGRTMIN+n does not exceed SIGRTMAX.

Unlike standard signals, real-time signals have no predefined meanings: the entire set of real-time signals can be used for application-defined purposes.

The default action for an unhandled real-time signal is to terminate the receiving process.

Real-time signals are distinguished by the following:

- Multiple instances of real-time signals can be queued. By contrast, if multiple instances of a standard signal are delivered while that signal is currently blocked, then only one instance is queued.

- If the signal is sent using sigqueue(3), an accompanying value (either an integer or a pointer) can be sent with the signal. If the receiving process establishes a handler for this signal using the SA_SIGINFO flag to sigaction(2), then it can obtain this data via the si_value field of the siginfo_t structure passed as the second argument to the handler. Furthermore, the si_pid and si_uid fields of this structure can be used to obtain the PID and real user ID of the process sending the signal.

- Real-time signals are delivered in a guaranteed order. Multiple real-time signals of the same type are delivered in the order they were sent. If different real-time signals are sent to a process, they are delivered starting with the lowest-numbered signal. (I.e., low-numbered signals have highest priority.) By contrast, if multiple standard signals are pending for a process, the order in which they are delivered is unspecified.

If both standard and real-time signals are pending for a process, POSIX leaves it unspecified which is delivered first. Linux, like many other implementations, gives priority to standard signals in this case.

According to POSIX, an implementation should permit at least _POSIX_SIGQUEUE_MAX (32) real-time signals to be queued to a process. However, Linux does things differently. Up to and including Linux 2.6.7, Linux imposes a system-wide limit on the number of queued real-time signals for all processes. This limit can be viewed and (with privilege) changed via the /proc/sys/kernel/rtsig-max file. A related file, /proc/sys/kernel/rtsig-nr, can be used to find out how many real-time signals are currently queued. In Linux 2.6.8, these /proc interfaces were replaced by the RLIMIT_SIGPENDING resource limit, which specifies a per-user limit for queued signals; see setrlimit(2) for further details.

```
    The addition of real-time signals required the widening of the signal
    set structure (sigset_t) from 32 to 64 bits.  Consequently, various
    system calls were superseded by new system calls that supported the
    larger signal sets.  The old and new system calls are as follows:
    Linux 2.0 and earlier   Linux 2.2 and later
    sigaction(2)            rt_sigaction(2)
    sigpending(2)           rt_sigpending(2)
    sigprocmask(2)          rt_sigprocmask(2)
    sigreturn(2)            rt_sigreturn(2)
    sigsuspend(2)           rt_sigsuspend(2)
    sigtimedwait(2)         rt_sigtimedwait(2)

Interruption of system calls and library functions by signal handlers
    If a signal handler is invoked while a system call or library function
    call is blocked, then either:

    • the call is automatically restarted after the signal handler
      returns; or

    • the call fails with the error EINTR.

    Which of these two behaviors occurs depends on the interface and
    whether or not the signal handler was established using the SA_RESTART
    flag (see sigaction(2)).  The details vary across UNIX systems; below,
    the details for Linux.

    If a blocked call to one of the following interfaces is interrupted by
    a signal handler, then the call is automatically restarted after the
    signal handler returns if the SA_RESTART flag was used; otherwise the
    call fails with the error EINTR:

    • read(2), readv(2), write(2), writev(2), and ioctl(2) calls on "slow"
      devices.  A "slow" device is one where the I/O call may block for an
      indefinite time, for example, a terminal, pipe, or socket.  If an
      I/O call on a slow device has already transferred some data by the
      time it is interrupted by a signal handler, then the call will
      return a success status (normally, the number of bytes transferred).
      Note that a (local) disk is not a slow device according to this
      definition; I/O operations on disk devices are not interrupted by
      signals.

    • open(2), if it can block (e.g., when opening a FIFO; see fifo(7)).

    • wait(2), wait3(2), wait4(2), waitid(2), and waitpid(2).

    • Socket interfaces: accept(2), connect(2), recv(2), recvfrom(2),
      recvmmsg(2), recvmsg(2), send(2), sendto(2), and sendmsg(2), unless
      a timeout has been set on the socket (see below).

    • File locking interfaces: flock(2) and the F_SETLKW and F_OFD_SETLKW
      operations of fcntl(2)

    • POSIX message queue interfaces: mq_receive(3), mq_timedreceive(3),
      mq_send(3), and mq_timedsend(3).
```

```
       • futex(2) FUTEX_WAIT (since Linux 2.6.22; beforehand, always failed
         with EINTR).

       • getrandom(2).

       • pthread_mutex_lock(3), pthread_cond_wait(3), and related APIs.

       • futex(2) FUTEX_WAIT_BITSET.

       • POSIX semaphore interfaces: sem_wait(3) and sem_timedwait(3) (since
         Linux 2.6.22; beforehand, always failed with EINTR).

       • read(2) from an inotify(7) file descriptor (since Linux 3.8;
         beforehand, always failed with EINTR).

       The following interfaces are never restarted after being interrupted by
       a signal handler, regardless of the use of SA_RESTART; they always fail
       with the error EINTR when interrupted by a signal handler:

       • "Input" socket interfaces, when a timeout (SO_RCVTIMEO) has been set
         on the socket using setsockopt(2): accept(2), recv(2), recvfrom(2),
         recvmmsg(2) (also with a non-NULL timeout argument), and recvmsg(2).

       • "Output" socket interfaces, when a timeout (SO_RCVTIMEO) has been
         set on the socket using setsockopt(2): connect(2), send(2),
         sendto(2), and sendmsg(2).

       • Interfaces used to wait for signals: pause(2), sigsuspend(2),
         sigtimedwait(2), and sigwaitinfo(2).

       • File descriptor multiplexing interfaces: epoll_wait(2),
         epoll_pwait(2), poll(2), ppoll(2), select(2), and pselect(2).

       • System V IPC interfaces: msgrcv(2), msgsnd(2), semop(2), and
         semtimedop(2).

       • Sleep interfaces: clock_nanosleep(2), nanosleep(2), and usleep(3).

       • io_getevents(2).

       The sleep(3) function is also never restarted if interrupted by a
       handler, but gives a success return: the number of seconds remaining to
       sleep.

       In certain circumstances, the seccomp(2) user-space notification
       feature can lead to restarting of system calls that would otherwise
       never be restarted by SA_RESTART; for details, see seccomp_unotify(2).

   Interruption of system calls and library functions by stop signals
       On Linux, even in the absence of signal handlers, certain blocking
       interfaces can fail with the error EINTR after the process is stopped
       by one of the stop signals and then resumed via SIGCONT.  This behavior
       is not sanctioned by POSIX.1, and doesn't occur on other systems.
```

```
       The Linux interfaces that display this behavior are:

       • "Input" socket interfaces, when a timeout (SO_RCVTIMEO) has been set
         on the socket using setsockopt(2): accept(2), recv(2), recvfrom(2),
         recvmmsg(2) (also with a non-NULL timeout argument), and recvmsg(2).

       • "Output" socket interfaces, when a timeout (SO_RCVTIMEO) has been
         set on the socket using setsockopt(2): connect(2), send(2),
         sendto(2), and sendmsg(2), if a send timeout (SO_SNDTIMEO) has been
         set.

       • epoll_wait(2), epoll_pwait(2).

       • semop(2), semtimedop(2).

       • sigtimedwait(2), sigwaitinfo(2).

       • Linux 3.7 and earlier: read(2) from an inotify(7) file descriptor

       • Linux 2.6.21 and earlier: futex(2) FUTEX_WAIT, sem_timedwait(3),
         sem_wait(3).

       • Linux 2.6.8 and earlier: msgrcv(2), msgsnd(2).

       • Linux 2.4 and earlier: nanosleep(2).

STANDARDS
       POSIX.1, except as noted.

NOTES
       For a discussion of async-signal-safe functions, see signal-safety(7).

       The /proc/pid/task/tid/status file contains various fields that show
       the signals that a thread is blocking (SigBlk), catching (SigCgt), or
       ignoring (SigIgn).  (The set of signals that are caught or ignored will
       be the same across all threads in a process.)  Other fields show the
       set of pending signals that are directed to the thread (SigPnd) as well
       as the set of pending signals that are directed to the process as a
       whole (ShdPnd).  The corresponding fields in /proc/pid/status show the
       information for the main thread.  See proc(5) for further details.

BUGS
       There are six signals that can be delivered as a consequence of a
       hardware exception: SIGBUS, SIGEMT, SIGFPE, SIGILL, SIGSEGV, and
       SIGTRAP.  Which of these signals is delivered, for any given hardware
       exception, is not documented and does not always make sense.

       For example, an invalid memory access that causes delivery of SIGSEGV
       on one CPU architecture may cause delivery of SIGBUS on another
       architecture, or vice versa.

       For another example, using the x86 int instruction with a forbidden
       argument (any number other than 3 or 128) causes delivery of SIGSEGV,
       even though SIGILL would make more sense, because of how the CPU
       reports the forbidden operation to the kernel.
```

```
SEE ALSO
       kill(1), clone(2), getrlimit(2), kill(2), pidfd_send_signal(2),
       restart_syscall(2), rt_sigqueueinfo(2), setitimer(2), setrlimit(2),
       sgetmask(2), sigaction(2), sigaltstack(2), signal(2), signalfd(2),
       sigpending(2), sigprocmask(2), sigreturn(2), sigsuspend(2),
       sigwaitinfo(2), abort(3), bsd_signal(3), killpg(3), longjmp(3),
       pthread_sigqueue(3), raise(3), sigqueue(3), sigset(3), sigsetops(3),
       sigvec(3), sigwait(3), strsignal(3), swapcontext(3), sysv_signal(3),
       core(5), proc(5), nptl(7), pthreads(7), sigevent(3type)

Linux man-pages 6.7              2023-10-31                        signal(7)
```

---

signal-safety(7)

```
signal-safety(7)       Miscellaneous Information Manual       signal-safety(7)

NAME
       signal-safety - async-signal-safe functions

DESCRIPTION
       An async-signal-safe function is one that can be safely called from
       within a signal handler.  Many functions are not async-signal-safe.  In
       particular, nonreentrant functions are generally unsafe to call from a
       signal handler.

       The kinds of issues that render a function unsafe can be quickly
       understood when one considers the implementation of the stdio library,
       all of whose functions are not async-signal-safe.

       When performing buffered I/O on a file, the stdio functions must
       maintain a statically allocated data buffer along with associated
       counters and indexes (or pointers) that record the amount of data and
       the current position in the buffer.  Suppose that the main program is
       in the middle of a call to a stdio function such as printf(3) where the
       buffer and associated variables have been partially updated.  If, at
       that moment, the program is interrupted by a signal handler that also
       calls printf(3), then the second call to printf(3) will operate on
       inconsistent data, with unpredictable results.

       To avoid problems with unsafe functions, there are two possible
       choices:

       (a)  Ensure that (1) the signal handler calls only async-signal-safe
            functions, and (2) the signal handler itself is reentrant with
            respect to global variables in the main program.

       (b)  Block signal delivery in the main program when calling functions
            that are unsafe or operating on global data that is also accessed
            by the signal handler.

       Generally, the second choice is difficult in programs of any
       complexity, so the first choice is taken.
```

```
POSIX.1 specifies a set of functions that an implementation must make
async-signal-safe.  (An implementation may provide safe implementations
of additional functions, but this is not required by the standard and
other implementations may not provide the same guarantees.)

In general, a function is async-signal-safe either because it is
reentrant or because it is atomic with respect to signals (i.e., its
execution can't be interrupted by a signal handler).

The set of functions required to be async-signal-safe by POSIX.1 is
shown in the following table.  The functions not otherwise noted were
required to be async-signal-safe in POSIX.1-2001; the table details
changes in the subsequent standards.

Function               Notes
abort(3)               Added in POSIX.1-2001 TC1
accept(2)
access(2)
aio_error(3)
aio_return(3)
aio_suspend(3)         See notes below
alarm(2)
bind(2)
cfgetispeed(3)
cfgetospeed(3)
cfsetispeed(3)
cfsetospeed(3)
chdir(2)
chmod(2)
chown(2)
clock_gettime(2)
close(2)
connect(2)
creat(2)
dup(2)
dup2(2)
execl(3)               Added in POSIX.1-2008; see
                       notes below
execle(3)              See notes below
execv(3)               Added in POSIX.1-2008
execve(2)
_exit(2)
_Exit(2)
faccessat(2)           Added in POSIX.1-2008
fchdir(2)              Added in POSIX.1-2008 TC1
fchmod(2)
fchmodat(2)            Added in POSIX.1-2008
fchown(2)
fchownat(2)            Added in POSIX.1-2008
fcntl(2)
fdatasync(2)
fexecve(3)             Added in POSIX.1-2008
ffs(3)                 Added in POSIX.1-2008 TC2
fork(2)                See notes below
```

```
       fstat(2)
       fstatat(2)              Added in POSIX.1-2008
       fsync(2)
       ftruncate(2)
       futimens(3)             Added in POSIX.1-2008
       getegid(2)
       geteuid(2)
       getgid(2)
       getgroups(2)
       getpeername(2)
       getpgrp(2)
       getpid(2)
       getppid(2)
       getsockname(2)
       getsockopt(2)
       getuid(2)
       htonl(3)                Added in POSIX.1-2008 TC2
       htons(3)                Added in POSIX.1-2008 TC2
       kill(2)
       link(2)
       linkat(2)               Added in POSIX.1-2008
       listen(2)
       longjmp(3)              Added in POSIX.1-2008 TC2;
                               see notes below
       lseek(2)
       lstat(2)
       memccpy(3)              Added in POSIX.1-2008 TC2
       memchr(3)               Added in POSIX.1-2008 TC2
       memcmp(3)               Added in POSIX.1-2008 TC2
       memcpy(3)               Added in POSIX.1-2008 TC2
       memmove(3)              Added in POSIX.1-2008 TC2
       memset(3)               Added in POSIX.1-2008 TC2
       mkdir(2)
       mkdirat(2)              Added in POSIX.1-2008
       mkfifo(3)
       mkfifoat(3)             Added in POSIX.1-2008
       mknod(2)                Added in POSIX.1-2008
       mknodat(2)              Added in POSIX.1-2008
       ntohl(3)                Added in POSIX.1-2008 TC2
       ntohs(3)                Added in POSIX.1-2008 TC2
       open(2)
       openat(2)               Added in POSIX.1-2008
       pause(2)
       pipe(2)
       poll(2)
       posix_trace_event(3)
       pselect(2)
       pthread_kill(3)         Added in POSIX.1-2008 TC1
       pthread_self(3)         Added in POSIX.1-2008 TC1
       pthread_sigmask(3)      Added in POSIX.1-2008 TC1
       raise(3)
       read(2)
       readlink(2)
       readlinkat(2)           Added in POSIX.1-2008
       recv(2)
```

```
recvfrom(2)
recvmsg(2)
rename(2)
renameat(2)            Added in POSIX.1-2008
rmdir(2)
select(2)
sem_post(3)
send(2)
sendmsg(2)
sendto(2)
setgid(2)
setpgid(2)
setsid(2)
setsockopt(2)
setuid(2)
shutdown(2)
sigaction(2)
sigaddset(3)
sigdelset(3)
sigemptyset(3)
sigfillset(3)
sigismember(3)
siglongjmp(3)          Added in POSIX.1-2008 TC2;
                       see notes below
signal(2)
sigpause(3)
sigpending(2)
sigprocmask(2)
sigqueue(2)
sigset(3)
sigsuspend(2)
sleep(3)
sockatmark(3)          Added in POSIX.1-2001 TC2
socket(2)
socketpair(2)
stat(2)
stpcpy(3)              Added in POSIX.1-2008 TC2
stpncpy(3)             Added in POSIX.1-2008 TC2
strcat(3)              Added in POSIX.1-2008 TC2
strchr(3)              Added in POSIX.1-2008 TC2
strcmp(3)              Added in POSIX.1-2008 TC2
strcpy(3)              Added in POSIX.1-2008 TC2
strcspn(3)             Added in POSIX.1-2008 TC2
strlen(3)              Added in POSIX.1-2008 TC2
strncat(3)             Added in POSIX.1-2008 TC2
strncmp(3)             Added in POSIX.1-2008 TC2
strncpy(3)             Added in POSIX.1-2008 TC2
strnlen(3)             Added in POSIX.1-2008 TC2
strpbrk(3)             Added in POSIX.1-2008 TC2
strrchr(3)             Added in POSIX.1-2008 TC2
strspn(3)              Added in POSIX.1-2008 TC2
strstr(3)              Added in POSIX.1-2008 TC2
strtok_r(3)            Added in POSIX.1-2008 TC2
symlink(2)
symlinkat(2)           Added in POSIX.1-2008
```

```
       tcdrain(3)
       tcflow(3)
       tcflush(3)
       tcgetattr(3)
       tcgetpgrp(3)
       tcsendbreak(3)
       tcsetattr(3)
       tcsetpgrp(3)
       time(2)
       timer_getoverrun(2)
       timer_gettime(2)
       timer_settime(2)
       times(2)
       umask(2)
       uname(2)
       unlink(2)
       unlinkat(2)             Added in POSIX.1-2008
       utime(2)
       utimensat(2)            Added in POSIX.1-2008
       utimes(2)               Added in POSIX.1-2008
       wait(2)
       waitpid(2)
       wcpcpy(3)               Added in POSIX.1-2008 TC2
       wcpncpy(3)              Added in POSIX.1-2008 TC2
       wcscat(3)               Added in POSIX.1-2008 TC2
       wcschr(3)               Added in POSIX.1-2008 TC2
       wcscmp(3)               Added in POSIX.1-2008 TC2
       wcscpy(3)               Added in POSIX.1-2008 TC2
       wcscspn(3)              Added in POSIX.1-2008 TC2
       wcslen(3)               Added in POSIX.1-2008 TC2
       wcsncat(3)              Added in POSIX.1-2008 TC2
       wcsncmp(3)              Added in POSIX.1-2008 TC2
       wcsncpy(3)              Added in POSIX.1-2008 TC2
       wcsnlen(3)              Added in POSIX.1-2008 TC2
       wcspbrk(3)              Added in POSIX.1-2008 TC2
       wcsrchr(3)              Added in POSIX.1-2008 TC2
       wcsspn(3)               Added in POSIX.1-2008 TC2
       wcsstr(3)               Added in POSIX.1-2008 TC2
       wcstok(3)               Added in POSIX.1-2008 TC2
       wmemchr(3)              Added in POSIX.1-2008 TC2
       wmemcmp(3)              Added in POSIX.1-2008 TC2
       wmemcpy(3)              Added in POSIX.1-2008 TC2
       wmemmove(3)             Added in POSIX.1-2008 TC2
       wmemset(3)              Added in POSIX.1-2008 TC2
       write(2)

       Notes:

       • POSIX.1-2001 and POSIX.1-2001 TC2 required the functions
         fpathconf(3), pathconf(3), and sysconf(3) to be async-signal-safe,
         but this requirement was removed in POSIX.1-2008.

       • If a signal handler interrupts the execution of an unsafe function,
         and the handler terminates via a call to longjmp(3) or siglongjmp(3)
         and the program subsequently calls an unsafe function, then the
```

```
          behavior of the program is undefined.

      • POSIX.1-2001 TC1 clarified that if an application calls fork(2) from
        a signal handler and any of the fork handlers registered by
        pthread_atfork(3) calls a function that is not async-signal-safe,
        the behavior is undefined.  A future revision of the standard is
        likely to remove fork(2) from the list of async-signal-safe
        functions.

      • Asynchronous signal handlers that call functions which are
        cancelation points and nest over regions of deferred cancelation may
        trigger cancelation whose behavior is as if asynchronous cancelation
        had occurred and may cause application state to become inconsistent.

   errno
       Fetching and setting the value of errno is async-signal-safe provided
       that the signal handler saves errno on entry and restores its value
       before returning.

   Deviations in the GNU C library
       The following known deviations from the standard occur in the GNU C
       library:

       • Before glibc 2.24, execl(3) and execle(3) employed realloc(3)
         internally and were consequently not async-signal-safe.  This was
         fixed in glibc 2.24.

       • The glibc implementation of aio_suspend(3) is not async-signal-safe
         because it uses pthread_mutex_lock(3) internally.

SEE ALSO
       sigaction(2), signal(7), standards(7)

Linux man-pages 6.7              2023-10-31                  signal-safety(7)
```

ELF(5)

```
ELF(5)                        File Formats Manual                        ELF(5)

NAME
       elf - format of Executable and Linking Format (ELF) files

SYNOPSIS
       #include <elf.h>

DESCRIPTION
       The header file <elf.h> defines the format of ELF executable binary
       files.  Amongst these files are normal executable files, relocatable
       object files, core files, and shared objects.

       An executable file using the ELF file format consists of an ELF header,
       followed by a program header table or a section header table, or both.
```

```
    The ELF header is always at offset zero of the file.  The program
    header table and the section header table's offset in the file are
    defined in the ELF header.  The two tables describe the rest of the
    particularities of the file.

    This header file describes the above mentioned headers as C structures
    and also includes structures for dynamic sections, relocation sections
    and symbol tables.

Basic types
    The following types are used for N-bit architectures (N=32,64, ElfN
    stands for Elf32 or Elf64, uintN_t stands for uint32_t or uint64_t):

        ElfN_Addr       Unsigned program address, uintN_t
        ElfN_Off        Unsigned file offset, uintN_t
        ElfN_Section    Unsigned section index, uint16_t
        ElfN_Versym     Unsigned version symbol information, uint16_t
        Elf_Byte        unsigned char
        ElfN_Half       uint16_t
        ElfN_Sword      int32_t
        ElfN_Word       uint32_t
        ElfN_Sxword     int64_t
        ElfN_Xword      uint64_t

    (Note: the *BSD terminology is a bit different.  There, Elf64_Half is
    twice as large as Elf32_Half, and Elf64Quarter is used for uint16_t.
    In order to avoid confusion these types are replaced by explicit ones
    in the below.)

    All data structures that the file format defines follow the "natural"
    size and alignment guidelines for the relevant class.  If necessary,
    data structures contain explicit padding to ensure 4-byte alignment for
    4-byte objects, to force structure sizes to a multiple of 4, and so on.

ELF header (Ehdr)
    The ELF header is described by the type Elf32_Ehdr or Elf64_Ehdr:

        #define EI_NIDENT 16

        typedef struct {
            unsigned char e_ident[EI_NIDENT];
            uint16_t      e_type;
            uint16_t      e_machine;
            uint32_t      e_version;
            ElfN_Addr     e_entry;
            ElfN_Off      e_phoff;
            ElfN_Off      e_shoff;
            uint32_t      e_flags;
            uint16_t      e_ehsize;
            uint16_t      e_phentsize;
            uint16_t      e_phnum;
            uint16_t      e_shentsize;
            uint16_t      e_shnum;
            uint16_t      e_shstrndx;
        } ElfN_Ehdr;
```

```
The fields have the following meanings:

e_ident
       This array of bytes specifies how to interpret the file,
       independent of the processor or the file's remaining contents.
       Within this array everything is named by macros, which start
       with the prefix EI_ and may contain values which start with the
       prefix ELF.  The following macros are defined:

       EI_MAG0
              The first byte of the magic number.  It must be filled
              with ELFMAG0.  (0: 0x7f)

       EI_MAG1
              The second byte of the magic number.  It must be filled
              with ELFMAG1.  (1: 'E')

       EI_MAG2
              The third byte of the magic number.  It must be filled
              with ELFMAG2.  (2: 'L')

       EI_MAG3
              The fourth byte of the magic number.  It must be filled
              with ELFMAG3.  (3: 'F')

       EI_CLASS
              The fifth byte identifies the architecture for this
              binary:

              ELFCLASSNONE  This class is invalid.
              ELFCLASS32    This defines the 32-bit architecture.  It
                            supports machines with files and virtual
                            address spaces up to 4 Gigabytes.
              ELFCLASS64    This defines the 64-bit architecture.

       EI_DATA
              The sixth byte specifies the data encoding of the
              processor-specific data in the file.  Currently, these
              encodings are supported:

                ELFDATANONE   Unknown data format.
                ELFDATA2LSB   Two's complement, little-endian.
                ELFDATA2MSB   Two's complement, big-endian.

       EI_VERSION
              The seventh byte is the version number of the ELF
              specification:

              EV_NONE       Invalid version.
              EV_CURRENT    Current version.

       EI_OSABI
              The eighth byte identifies the operating system and ABI
              to which the object is targeted.  Some fields in other
```

```
                ELF structures have flags and values that have platform-
                specific meanings; the interpretation of those fields is
                determined by the value of this byte.  For example:

                ELFOSABI_NONE        Same as ELFOSABI_SYSV
                ELFOSABI_SYSV        UNIX System V ABI
                ELFOSABI_HPUX        HP-UX ABI
                ELFOSABI_NETBSD      NetBSD ABI
                ELFOSABI_LINUX       Linux ABI
                ELFOSABI_SOLARIS     Solaris ABI
                ELFOSABI_IRIX        IRIX ABI
                ELFOSABI_FREEBSD     FreeBSD ABI
                ELFOSABI_TRU64       TRU64 UNIX ABI
                ELFOSABI_ARM         ARM architecture ABI
                ELFOSABI_STANDALONE  Stand-alone (embedded) ABI

        EI_ABIVERSION
                The ninth byte identifies the version of the ABI to which
                the object is targeted.  This field is used to
                distinguish among incompatible versions of an ABI.  The
                interpretation of this version number is dependent on the
                ABI identified by the EI_OSABI field.  Applications
                conforming to this specification use the value 0.

        EI_PAD Start of padding.  These bytes are reserved and set to
                zero.  Programs which read them should ignore them.  The
                value for EI_PAD will change in the future if currently
                unused bytes are given meanings.

        EI_NIDENT
                The size of the e_ident array.

    e_type This member of the structure identifies the object file type:

        ET_NONE         An unknown type.
        ET_REL          A relocatable file.
        ET_EXEC         An executable file.
        ET_DYN          A shared object.
        ET_CORE         A core file.

    e_machine
        This member specifies the required architecture for an
        individual file.  For example:

        EM_NONE         An unknown machine
        EM_M32          AT&T WE 32100
        EM_SPARC        Sun Microsystems SPARC
        EM_386          Intel 80386
        EM_68K          Motorola 68000
        EM_88K          Motorola 88000
        EM_860          Intel 80860
        EM_MIPS         MIPS RS3000 (big-endian only)
        EM_PARISC       HP/PA
        EM_SPARC32PLUS  SPARC with enhanced instruction set
        EM_PPC          PowerPC
```

```
              EM_PPC64        PowerPC 64-bit
              EM_S390         IBM S/390
              EM_ARM          Advanced RISC Machines
              EM_SH           Renesas SuperH
              EM_SPARCV9      SPARC v9 64-bit
              EM_IA_64        Intel Itanium
              EM_X86_64       AMD x86-64
              EM_VAX          DEC Vax

       e_version
              This member identifies the file version:

              EV_NONE         Invalid version
              EV_CURRENT      Current version

       e_entry
              This member gives the virtual address to which the system first
              transfers control, thus starting the process.  If the file has
              no associated entry point, this member holds zero.

       e_phoff
              This member holds the program header table's file offset in
              bytes.  If the file has no program header table, this member
              holds zero.

       e_shoff
              This member holds the section header table's file offset in
              bytes.  If the file has no section header table, this member
              holds zero.

       e_flags
              This member holds processor-specific flags associated with the
              file.  Flag names take the form EF_`machine_flag'.  Currently,
              no flags have been defined.

       e_ehsize
              This member holds the ELF header's size in bytes.

       e_phentsize
              This member holds the size in bytes of one entry in the file's
              program header table; all entries are the same size.

       e_phnum
              This member holds the number of entries in the program header
              table.  Thus the product of e_phentsize and e_phnum gives the
              table's size in bytes.  If a file has no program header, e_phnum
              holds the value zero.

              If the number of entries in the program header table is larger
              than or equal to PN_XNUM (0xffff), this member holds PN_XNUM
              (0xffff) and the real number of entries in the program header
              table is held in the sh_info member of the initial entry in
              section header table.  Otherwise, the sh_info member of the
              initial entry contains the value zero.
```

```
            PN_XNUM
                   This is defined as 0xffff, the largest number e_phnum can
                   have, specifying where the actual number of program
                   headers is assigned.

     e_shentsize
            This member holds a sections header's size in bytes.  A section
            header is one entry in the section header table; all entries are
            the same size.

     e_shnum
            This member holds the number of entries in the section header
            table.  Thus the product of e_shentsize and e_shnum gives the
            section header table's size in bytes.  If a file has no section
            header table, e_shnum holds the value of zero.

            If the number of entries in the section header table is larger
            than or equal to SHN_LORESERVE (0xff00), e_shnum holds the value
            zero and the real number of entries in the section header table
            is held in the sh_size member of the initial entry in section
            header table.  Otherwise, the sh_size member of the initial
            entry in the section header table holds the value zero.

     e_shstrndx
            This member holds the section header table index of the entry
            associated with the section name string table.  If the file has
            no section name string table, this member holds the value
            SHN_UNDEF.

            If the index of section name string table section is larger than
            or equal to SHN_LORESERVE (0xff00), this member holds SHN_XINDEX
            (0xffff) and the real index of the section name string table
            section is held in the sh_link member of the initial entry in
            section header table.  Otherwise, the sh_link member of the
            initial entry in section header table contains the value zero.

  Program header (Phdr)
     An executable or shared object file's program header table is an array
     of structures, each describing a segment or other information the
     system needs to prepare the program for execution.  An object file
     segment contains one or more sections.  Program headers are meaningful
     only for executable and shared object files.  A file specifies its own
     program header size with the ELF header's e_phentsize and e_phnum
     members.  The ELF program header is described by the type Elf32_Phdr or
     Elf64_Phdr depending on the architecture:

         typedef struct {
             uint32_t   p_type;
             Elf32_Off  p_offset;
             Elf32_Addr p_vaddr;
             Elf32_Addr p_paddr;
             uint32_t   p_filesz;
             uint32_t   p_memsz;
             uint32_t   p_flags;
             uint32_t   p_align;
```

```
    } Elf32_Phdr;

    typedef struct {
        uint32_t   p_type;
        uint32_t   p_flags;
        Elf64_Off  p_offset;
        Elf64_Addr p_vaddr;
        Elf64_Addr p_paddr;
        uint64_t   p_filesz;
        uint64_t   p_memsz;
        uint64_t   p_align;
    } Elf64_Phdr;

The main difference between the 32-bit and the 64-bit program header
lies in the location of the p_flags member in the total struct.

p_type This member of the structure indicates what kind of segment this
       array element describes or how to interpret the array element's
       information.

          PT_NULL
                 The array element is unused and the other members'
                 values are undefined.  This lets the program header
                 have ignored entries.

          PT_LOAD
                 The array element specifies a loadable segment,
                 described by p_filesz and p_memsz.  The bytes from the
                 file are mapped to the beginning of the memory
                 segment.  If the segment's memory size p_memsz is
                 larger than the file size p_filesz, the "extra" bytes
                 are defined to hold the value 0 and to follow the
                 segment's initialized area.  The file size may not be
                 larger than the memory size.  Loadable segment entries
                 in the program header table appear in ascending order,
                 sorted on the p_vaddr member.

          PT_DYNAMIC
                 The array element specifies dynamic linking
                 information.

          PT_INTERP
                 The array element specifies the location and size of a
                 null-terminated pathname to invoke as an interpreter.
                 This segment type is meaningful only for executable
                 files (though it may occur for shared objects).
                 However it may not occur more than once in a file.  If
                 it is present, it must precede any loadable segment
                 entry.

          PT_NOTE
                 The array element specifies the location of notes
                 (ElfN_Nhdr).

          PT_SHLIB
```

```
                This segment type is reserved but has unspecified
                semantics.  Programs that contain an array element of
                this type do not conform to the ABI.

            PT_PHDR
                The array element, if present, specifies the location
                and size of the program header table itself, both in
                the file and in the memory image of the program.  This
                segment type may not occur more than once in a file.
                Moreover, it may occur only if the program header
                table is part of the memory image of the program.  If
                it is present, it must precede any loadable segment
                entry.

            PT_LOPROC
            PT_HIPROC
                Values in the inclusive range [PT_LOPROC, PT_HIPROC]
                are reserved for processor-specific semantics.

            PT_GNU_STACK
                GNU extension which is used by the Linux kernel to
                control the state of the stack via the flags set in
                the p_flags member.

    p_offset
           This member holds the offset from the beginning of the file at
           which the first byte of the segment resides.

    p_vaddr
           This member holds the virtual address at which the first byte of
           the segment resides in memory.

    p_paddr
           On systems for which physical addressing is relevant, this
           member is reserved for the segment's physical address.  Under
           BSD this member is not used and must be zero.

    p_filesz
           This member holds the number of bytes in the file image of the
           segment.  It may be zero.

    p_memsz
           This member holds the number of bytes in the memory image of the
           segment.  It may be zero.

    p_flags
           This member holds a bit mask of flags relevant to the segment:

           PF_X   An executable segment.
           PF_W   A writable segment.
           PF_R   A readable segment.

           A text segment commonly has the flags PF_X and PF_R.  A data
           segment commonly has PF_W and PF_R.
```

```
       p_align
              This member holds the value to which the segments are aligned in
              memory and in the file.  Loadable process segments must have
              congruent values for p_vaddr and p_offset, modulo the page size.
              Values of zero and one mean no alignment is required.
              Otherwise, p_align should be a positive, integral power of two,
              and p_vaddr should equal p_offset, modulo p_align.

   Section header (Shdr)
       A file's section header table lets one locate all the file's sections.
       The section header table is an array of Elf32_Shdr or Elf64_Shdr
       structures.  The ELF header's e_shoff member gives the byte offset from
       the beginning of the file to the section header table.  e_shnum holds
       the number of entries the section header table contains.  e_shentsize
       holds the size in bytes of each entry.

       A section header table index is a subscript into this array.  Some
       section header table indices are reserved: the initial entry and the
       indices between SHN_LORESERVE and SHN_HIRESERVE.  The initial entry is
       used in ELF extensions for e_phnum, e_shnum, and e_shstrndx; in other
       cases, each field in the initial entry is set to zero.  An object file
       does not have sections for these special indices:

       SHN_UNDEF
              This value marks an undefined, missing, irrelevant, or otherwise
              meaningless section reference.

       SHN_LORESERVE
              This value specifies the lower bound of the range of reserved
              indices.

       SHN_LOPROC
       SHN_HIPROC
              Values greater in the inclusive range [SHN_LOPROC, SHN_HIPROC]
              are reserved for processor-specific semantics.

       SHN_ABS
              This value specifies the absolute value for the corresponding
              reference.  For example, a symbol defined relative to section
              number SHN_ABS has an absolute value and is not affected by
              relocation.

       SHN_COMMON
              Symbols defined relative to this section are common symbols,
              such as FORTRAN COMMON or unallocated C external variables.

       SHN_HIRESERVE
              This value specifies the upper bound of the range of reserved
              indices.  The system reserves indices between SHN_LORESERVE and
              SHN_HIRESERVE, inclusive.  The section header table does not
              contain entries for the reserved indices.

       The section header has the following structure:

           typedef struct {
```

```
            uint32_t   sh_name;
            uint32_t   sh_type;
            uint32_t   sh_flags;
            Elf32_Addr sh_addr;
            Elf32_Off  sh_offset;
            uint32_t   sh_size;
            uint32_t   sh_link;
            uint32_t   sh_info;
            uint32_t   sh_addralign;
            uint32_t   sh_entsize;
        } Elf32_Shdr;

        typedef struct {
            uint32_t   sh_name;
            uint32_t   sh_type;
            uint64_t   sh_flags;
            Elf64_Addr sh_addr;
            Elf64_Off  sh_offset;
            uint64_t   sh_size;
            uint32_t   sh_link;
            uint32_t   sh_info;
            uint64_t   sh_addralign;
            uint64_t   sh_entsize;
        } Elf64_Shdr;

    No real differences exist between the 32-bit and 64-bit section
    headers.

    sh_name
           This member specifies the name of the section.  Its value is an
           index into the section header string table section, giving the
           location of a null-terminated string.

    sh_type
           This member categorizes the section's contents and semantics.

           SHT_NULL
                  This value marks the section header as inactive.  It does
                  not have an associated section.  Other members of the
                  section header have undefined values.

           SHT_PROGBITS
                  This section holds information defined by the program,
                  whose format and meaning are determined solely by the
                  program.

           SHT_SYMTAB
                  This section holds a symbol table.  Typically, SHT_SYMTAB
                  provides symbols for link editing, though it may also be
                  used for dynamic linking.  As a complete symbol table, it
                  may contain many symbols unnecessary for dynamic linking.
                  An object file can also contain a SHT_DYNSYM section.

           SHT_STRTAB
                  This section holds a string table.  An object file may
```

```
       have multiple string table sections.

SHT_RELA
       This section holds relocation entries with explicit
       addends, such as type Elf32_Rela for the 32-bit class of
       object files.  An object may have multiple relocation
       sections.

SHT_HASH
       This section holds a symbol hash table.  An object
       participating in dynamic linking must contain a symbol
       hash table.  An object file may have only one hash table.

SHT_DYNAMIC
       This section holds information for dynamic linking.  An
       object file may have only one dynamic section.

SHT_NOTE
       This section holds notes (ElfN_Nhdr).

SHT_NOBITS
       A section of this type occupies no space in the file but
       otherwise resembles SHT_PROGBITS.  Although this section
       contains no bytes, the sh_offset member contains the
       conceptual file offset.

SHT_REL
       This section holds relocation offsets without explicit
       addends, such as type Elf32_Rel for the 32-bit class of
       object files.  An object file may have multiple
       relocation sections.

SHT_SHLIB
       This section is reserved but has unspecified semantics.

SHT_DYNSYM
       This section holds a minimal set of dynamic linking
       symbols.  An object file can also contain a SHT_SYMTAB
       section.

SHT_LOPROC
SHT_HIPROC
       Values in the inclusive range [SHT_LOPROC, SHT_HIPROC]
       are reserved for processor-specific semantics.

SHT_LOUSER
       This value specifies the lower bound of the range of
       indices reserved for application programs.

SHT_HIUSER
       This value specifies the upper bound of the range of
       indices reserved for application programs.  Section types
       between SHT_LOUSER and SHT_HIUSER may be used by the
       application, without conflicting with current or future
       system-defined section types.
```

```
   sh_flags
          Sections support one-bit flags that describe miscellaneous
          attributes.  If a flag bit is set in sh_flags, the attribute is
          "on" for the section.  Otherwise, the attribute is "off" or does
          not apply.  Undefined attributes are set to zero.

          SHF_WRITE
                 This section contains data that should be writable during
                 process execution.

          SHF_ALLOC
                 This section occupies memory during process execution.
                 Some control sections do not reside in the memory image
                 of an object file.  This attribute is off for those
                 sections.

          SHF_EXECINSTR
                 This section contains executable machine instructions.

          SHF_MASKPROC
                 All bits included in this mask are reserved for
                 processor-specific semantics.

   sh_addr
          If this section appears in the memory image of a process, this
          member holds the address at which the section's first byte
          should reside.  Otherwise, the member contains zero.

   sh_offset
          This member's value holds the byte offset from the beginning of
          the file to the first byte in the section.  One section type,
          SHT_NOBITS, occupies no space in the file, and its sh_offset
          member locates the conceptual placement in the file.

   sh_size
          This member holds the section's size in bytes.  Unless the
          section type is SHT_NOBITS, the section occupies sh_size bytes
          in the file.  A section of type SHT_NOBITS may have a nonzero
          size, but it occupies no space in the file.

   sh_link
          This member holds a section header table index link, whose
          interpretation depends on the section type.

   sh_info
          This member holds extra information, whose interpretation
          depends on the section type.

   sh_addralign
          Some sections have address alignment constraints.  If a section
          holds a doubleword, the system must ensure doubleword alignment
          for the entire section.  That is, the value of sh_addr must be
          congruent to zero, modulo the value of sh_addralign.  Only zero
          and positive integral powers of two are allowed.  The value 0 or
```

```
       1 means that the section has no alignment constraints.

sh_entsize
       Some sections hold a table of fixed-sized entries, such as a
       symbol table.  For such a section, this member gives the size in
       bytes for each entry.  This member contains zero if the section
       does not hold a table of fixed-size entries.

Various sections hold program and control information:

.bss   This section holds uninitialized data that contributes to the
       program's memory image.  By definition, the system initializes
       the data with zeros when the program begins to run.  This
       section is of type SHT_NOBITS.  The attribute types are
       SHF_ALLOC and SHF_WRITE.

.comment
       This section holds version control information.  This section is
       of type SHT_PROGBITS.  No attribute types are used.

.ctors This section holds initialized pointers to the C++ constructor
       functions.  This section is of type SHT_PROGBITS.  The attribute
       types are SHF_ALLOC and SHF_WRITE.

.data  This section holds initialized data that contribute to the
       program's memory image.  This section is of type SHT_PROGBITS.
       The attribute types are SHF_ALLOC and SHF_WRITE.

.data1 This section holds initialized data that contribute to the
       program's memory image.  This section is of type SHT_PROGBITS.
       The attribute types are SHF_ALLOC and SHF_WRITE.

.debug This section holds information for symbolic debugging.  The
       contents are unspecified.  This section is of type SHT_PROGBITS.
       No attribute types are used.

.dtors This section holds initialized pointers to the C++ destructor
       functions.  This section is of type SHT_PROGBITS.  The attribute
       types are SHF_ALLOC and SHF_WRITE.

.dynamic
       This section holds dynamic linking information.  The section's
       attributes will include the SHF_ALLOC bit.  Whether the
       SHF_WRITE bit is set is processor-specific.  This section is of
       type SHT_DYNAMIC.  See the attributes above.

.dynstr
       This section holds strings needed for dynamic linking, most
       commonly the strings that represent the names associated with
       symbol table entries.  This section is of type SHT_STRTAB.  The
       attribute type used is SHF_ALLOC.

.dynsym
       This section holds the dynamic linking symbol table.  This
       section is of type SHT_DYNSYM.  The attribute used is SHF_ALLOC.
```

```
.fini  This section holds executable instructions that contribute to
       the process termination code.  When a program exits normally the
       system arranges to execute the code in this section.  This
       section is of type SHT_PROGBITS.  The attributes used are
       SHF_ALLOC and SHF_EXECINSTR.

.gnu.version
       This section holds the version symbol table, an array of
       ElfN_Half elements.  This section is of type SHT_GNU_versym.
       The attribute type used is SHF_ALLOC.

.gnu.version_d
       This section holds the version symbol definitions, a table of
       ElfN_Verdef structures.  This section is of type SHT_GNU_verdef.
       The attribute type used is SHF_ALLOC.

.gnu.version_r
       This section holds the version symbol needed elements, a table
       of ElfN_Verneed structures.  This section is of type
       SHT_GNU_versym.  The attribute type used is SHF_ALLOC.

.got   This section holds the global offset table.  This section is of
       type SHT_PROGBITS.  The attributes are processor-specific.

.hash  This section holds a symbol hash table.  This section is of type
       SHT_HASH.  The attribute used is SHF_ALLOC.

.init  This section holds executable instructions that contribute to
       the process initialization code.  When a program starts to run
       the system arranges to execute the code in this section before
       calling the main program entry point.  This section is of type
       SHT_PROGBITS.  The attributes used are SHF_ALLOC and
       SHF_EXECINSTR.

.interp
       This section holds the pathname of a program interpreter.  If
       the file has a loadable segment that includes the section, the
       section's attributes will include the SHF_ALLOC bit.  Otherwise,
       that bit will be off.  This section is of type SHT_PROGBITS.

.line  This section holds line number information for symbolic
       debugging, which describes the correspondence between the
       program source and the machine code.  The contents are
       unspecified.  This section is of type SHT_PROGBITS.  No
       attribute types are used.

.note  This section holds various notes.  This section is of type
       SHT_NOTE.  No attribute types are used.

.note.ABI-tag
       This section is used to declare the expected run-time ABI of the
       ELF image.  It may include the operating system name and its
       run-time versions.  This section is of type SHT_NOTE.  The only
       attribute used is SHF_ALLOC.
```

```
.note.gnu.build-id
       This section is used to hold an ID that uniquely identifies the
       contents of the ELF image.  Different files with the same build
       ID should contain the same executable content.  See the
       --build-id option to the GNU linker (ld (1)) for more details.
       This section is of type SHT_NOTE.  The only attribute used is
       SHF_ALLOC.

.note.GNU-stack
       This section is used in Linux object files for declaring stack
       attributes.  This section is of type SHT_PROGBITS.  The only
       attribute used is SHF_EXECINSTR.  This indicates to the GNU
       linker that the object file requires an executable stack.

.note.openbsd.ident
       OpenBSD native executables usually contain this section to
       identify themselves so the kernel can bypass any compatibility
       ELF binary emulation tests when loading the file.

.plt   This section holds the procedure linkage table.  This section is
       of type SHT_PROGBITS.  The attributes are processor-specific.

.relNAME
       This section holds relocation information as described below.
       If the file has a loadable segment that includes relocation, the
       section's attributes will include the SHF_ALLOC bit.  Otherwise,
       the bit will be off.  By convention, "NAME" is supplied by the
       section to which the relocations apply.  Thus a relocation
       section for .text normally would have the name .rel.text.  This
       section is of type SHT_REL.

.relaNAME
       This section holds relocation information as described below.
       If the file has a loadable segment that includes relocation, the
       section's attributes will include the SHF_ALLOC bit.  Otherwise,
       the bit will be off.  By convention, "NAME" is supplied by the
       section to which the relocations apply.  Thus a relocation
       section for .text normally would have the name .rela.text.  This
       section is of type SHT_RELA.

.rodata
       This section holds read-only data that typically contributes to
       a nonwritable segment in the process image.  This section is of
       type SHT_PROGBITS.  The attribute used is SHF_ALLOC.

.rodata1
       This section holds read-only data that typically contributes to
       a nonwritable segment in the process image.  This section is of
       type SHT_PROGBITS.  The attribute used is SHF_ALLOC.

.shstrtab
       This section holds section names.  This section is of type
       SHT_STRTAB.  No attribute types are used.
```

```
      .strtab
             This section holds strings, most commonly the strings that
             represent the names associated with symbol table entries.  If
             the file has a loadable segment that includes the symbol string
             table, the section's attributes will include the SHF_ALLOC bit.
             Otherwise, the bit will be off.  This section is of type
             SHT_STRTAB.

      .symtab
             This section holds a symbol table.  If the file has a loadable
             segment that includes the symbol table, the section's attributes
             will include the SHF_ALLOC bit.  Otherwise, the bit will be off.
             This section is of type SHT_SYMTAB.

      .text  This section holds the "text", or executable instructions, of a
             program.  This section is of type SHT_PROGBITS.  The attributes
             used are SHF_ALLOC and SHF_EXECINSTR.

  String and symbol tables
      String table sections hold null-terminated character sequences,
      commonly called strings.  The object file uses these strings to
      represent symbol and section names.  One references a string as an
      index into the string table section.  The first byte, which is index
      zero, is defined to hold a null byte ('\0').  Similarly, a string
      table's last byte is defined to hold a null byte, ensuring null
      termination for all strings.

      An object file's symbol table holds information needed to locate and
      relocate a program's symbolic definitions and references.  A symbol
      table index is a subscript into this array.

          typedef struct {
              uint32_t      st_name;
              Elf32_Addr    st_value;
              uint32_t      st_size;
              unsigned char st_info;
              unsigned char st_other;
              uint16_t      st_shndx;
          } Elf32_Sym;

          typedef struct {
              uint32_t      st_name;
              unsigned char st_info;
              unsigned char st_other;
              uint16_t      st_shndx;
              Elf64_Addr    st_value;
              uint64_t      st_size;
          } Elf64_Sym;

      The 32-bit and 64-bit versions have the same members, just in a
      different order.

      st_name
             This member holds an index into the object file's symbol string
             table, which holds character representations of the symbol
```

```
       names.  If the value is nonzero, it represents a string table
       index that gives the symbol name.  Otherwise, the symbol has no
       name.

   st_value
       This member gives the value of the associated symbol.

   st_size
       Many symbols have associated sizes.  This member holds zero if
       the symbol has no size or an unknown size.

   st_info
       This member specifies the symbol's type and binding attributes:

       STT_NOTYPE
              The symbol's type is not defined.

       STT_OBJECT
              The symbol is associated with a data object.

       STT_FUNC
              The symbol is associated with a function or other
              executable code.

       STT_SECTION
              The symbol is associated with a section.  Symbol table
              entries of this type exist primarily for relocation and
              normally have STB_LOCAL bindings.

       STT_FILE
              By convention, the symbol's name gives the name of the
              source file associated with the object file.  A file
              symbol has STB_LOCAL bindings, its section index is
              SHN_ABS, and it precedes the other STB_LOCAL symbols of
              the file, if it is present.

       STT_LOPROC
       STT_HIPROC
              Values in the inclusive range [STT_LOPROC, STT_HIPROC]
              are reserved for processor-specific semantics.

       STB_LOCAL
              Local symbols are not visible outside the object file
              containing their definition.  Local symbols of the same
              name may exist in multiple files without interfering with
              each other.

       STB_GLOBAL
              Global symbols are visible to all object files being
              combined.  One file's definition of a global symbol will
              satisfy another file's undefined reference to the same
              symbol.

       STB_WEAK
              Weak symbols resemble global symbols, but their
```

```
                definitions have lower precedence.

            STB_LOPROC
            STB_HIPROC
                Values in the inclusive range [STB_LOPROC, STB_HIPROC]
                are reserved for processor-specific semantics.

            There are macros for packing and unpacking the binding and type
            fields:

            ELF32_ST_BIND(info)
            ELF64_ST_BIND(info)
                Extract a binding from an st_info value.

            ELF32_ST_TYPE(info)
            ELF64_ST_TYPE(info)
                Extract a type from an st_info value.

            ELF32_ST_INFO(bind, type)
            ELF64_ST_INFO(bind, type)
                Convert a binding and a type into an st_info value.

    st_other
            This member defines the symbol visibility.

            STV_DEFAULT
                Default symbol visibility rules.  Global and weak symbols
                are available to other modules; references in the local
                module can be interposed by definitions in other modules.
            STV_INTERNAL
                Processor-specific hidden class.
            STV_HIDDEN
                Symbol is unavailable to other modules; references in the
                local module always resolve to the local symbol (i.e.,
                the symbol can't be interposed by definitions in other
                modules).
            STV_PROTECTED
                Symbol is available to other modules, but references in
                the local module always resolve to the local symbol.

            There are macros for extracting the visibility type:

            ELF32_ST_VISIBILITY(other) or ELF64_ST_VISIBILITY(other)

    st_shndx
            Every symbol table entry is "defined" in relation to some
            section.  This member holds the relevant section header table
            index.

Relocation entries (Rel & Rela)
    Relocation is the process of connecting symbolic references with
    symbolic definitions.  Relocatable files must have information that
    describes how to modify their section contents, thus allowing
    executable and shared object files to hold the right information for a
    process's program image.  Relocation entries are these data.
```

```
    Relocation structures that do not need an addend:

        typedef struct {
            Elf32_Addr r_offset;
            uint32_t   r_info;
        } Elf32_Rel;

        typedef struct {
            Elf64_Addr r_offset;
            uint64_t   r_info;
        } Elf64_Rel;

    Relocation structures that need an addend:

        typedef struct {
            Elf32_Addr r_offset;
            uint32_t   r_info;
            int32_t    r_addend;
        } Elf32_Rela;

        typedef struct {
            Elf64_Addr r_offset;
            uint64_t   r_info;
            int64_t    r_addend;
        } Elf64_Rela;

    r_offset
           This member gives the location at which to apply the relocation
           action.  For a relocatable file, the value is the byte offset
           from the beginning of the section to the storage unit affected
           by the relocation.  For an executable file or shared object, the
           value is the virtual address of the storage unit affected by the
           relocation.

    r_info This member gives both the symbol table index with respect to
           which the relocation must be made and the type of relocation to
           apply.  Relocation types are processor-specific.  When the text
           refers to a relocation entry's relocation type or symbol table
           index, it means the result of applying ELF[32|64]_R_TYPE or
           ELF[32|64]_R_SYM, respectively, to the entry's r_info member.

    r_addend
           This member specifies a constant addend used to compute the
           value to be stored into the relocatable field.

Dynamic tags (Dyn)
    The .dynamic section contains a series of structures that hold relevant
    dynamic linking information.  The d_tag member controls the
    interpretation of d_un.

        typedef struct {
            Elf32_Sword    d_tag;
            union {
                Elf32_Word d_val;
```

```
                Elf32_Addr d_ptr;
            } d_un;
        } Elf32_Dyn;
        extern Elf32_Dyn _DYNAMIC[];

        typedef struct {
            Elf64_Sxword    d_tag;
            union {
                Elf64_Xword d_val;
                Elf64_Addr  d_ptr;
            } d_un;
        } Elf64_Dyn;
        extern Elf64_Dyn _DYNAMIC[];

    d_tag  This member may have any of the following values:

           DT_NULL     Marks end of dynamic section

           DT_NEEDED   String table offset to name of a needed library

           DT_PLTRELSZ Size in bytes of PLT relocation entries

           DT_PLTGOT   Address of PLT and/or GOT

           DT_HASH     Address of symbol hash table

           DT_STRTAB   Address of string table

           DT_SYMTAB   Address of symbol table

           DT_RELA     Address of Rela relocation table

           DT_RELASZ   Size in bytes of the Rela relocation table

           DT_RELAENT  Size in bytes of a Rela relocation table entry

           DT_STRSZ    Size in bytes of string table

           DT_SYMENT   Size in bytes of a symbol table entry

           DT_INIT     Address of the initialization function

           DT_FINI     Address of the termination function

           DT_SONAME   String table offset to name of shared object

           DT_RPATH    String table offset to library search path
                       (deprecated)

           DT_SYMBOLIC Alert linker to search this shared object before the
                       executable for symbols

           DT_REL      Address of Rel relocation table

           DT_RELSZ    Size in bytes of Rel relocation table
```

```
            DT_RELENT   Size in bytes of a Rel table entry

            DT_PLTREL   Type of relocation entry to which the PLT refers
                        (Rela or Rel)

            DT_DEBUG    Undefined use for debugging

            DT_TEXTREL  Absence of this entry indicates that no relocation
                        entries should apply to a nonwritable segment

            DT_JMPREL   Address of relocation entries associated solely with
                        the PLT

            DT_BIND_NOW Instruct dynamic linker to process all relocations
                        before transferring control to the executable

            DT_RUNPATH  String table offset to library search path

            DT_LOPROC
            DT_HIPROC   Values in the inclusive range [DT_LOPROC, DT_HIPROC]
                        are reserved for processor-specific semantics

    d_val  This member represents integer values with various
           interpretations.

    d_ptr  This member represents program virtual addresses.  When
           interpreting these addresses, the actual address should be
           computed based on the original file value and memory base
           address.  Files do not contain relocation entries to fixup these
           addresses.

    _DYNAMIC
           Array containing all the dynamic structures in the .dynamic
           section.  This is automatically populated by the linker.

Notes (Nhdr)
    ELF notes allow for appending arbitrary information for the system to
    use.  They are largely used by core files (e_type of ET_CORE), but many
    projects define their own set of extensions.  For example, the GNU tool
    chain uses ELF notes to pass information from the linker to the C
    library.

    Note sections contain a series of notes (see the struct definitions
    below).  Each note is followed by the name field (whose length is
    defined in n_namesz) and then by the descriptor field (whose length is
    defined in n_descsz) and whose starting address has a 4 byte alignment.
    Neither field is defined in the note struct due to their arbitrary
    lengths.

    An example for parsing out two consecutive notes should clarify their
    layout in memory:

        void *memory, *name, *desc;
        Elf64_Nhdr *note, *next_note;
```

```
    /* The buffer is pointing to the start of the section/segment. */
    note = memory;

    /* If the name is defined, it follows the note. */
    name = note->n_namesz == 0 ? NULL : memory + sizeof(*note);

    /* If the descriptor is defined, it follows the name
       (with alignment). */

    desc = note->n_descsz == 0 ? NULL :
           memory + sizeof(*note) + ALIGN_UP(note->n_namesz, 4);

    /* The next note follows both (with alignment). */
    next_note = memory + sizeof(*note) +
                         ALIGN_UP(note->n_namesz, 4) +
                         ALIGN_UP(note->n_descsz, 4);
```

Keep in mind that the interpretation of n_type depends on the namespace defined by the n_namesz field. If the n_namesz field is not set (e.g., is 0), then there are two sets of notes: one for core files and one for all other ELF types. If the namespace is unknown, then tools will usually fallback to these sets of notes as well.

```
    typedef struct {
        Elf32_Word n_namesz;
        Elf32_Word n_descsz;
        Elf32_Word n_type;
    } Elf32_Nhdr;

    typedef struct {
        Elf64_Word n_namesz;
        Elf64_Word n_descsz;
        Elf64_Word n_type;
    } Elf64_Nhdr;
```

n_namesz
: The length of the name field in bytes. The contents will immediately follow this note in memory. The name is null terminated. For example, if the name is "GNU", then n_namesz will be set to 4.

n_descsz
: The length of the descriptor field in bytes. The contents will immediately follow the name field in memory.

n_type
: Depending on the value of the name field, this member may have any of the following values:

  Core files (e_type = ET_CORE)
  : Notes used by all core files. These are highly operating system or architecture specific and often require close coordination with kernels, C libraries, and debuggers. These are used when the namespace is the default (i.e., n_namesz will be set to 0), or a fallback when the

```
       namespace is unknown.

       NT_PRSTATUS          prstatus struct
       NT_FPREGSET          fpregset struct
       NT_PRPSINFO          prpsinfo struct
       NT_PRXREG            prxregset struct
       NT_TASKSTRUCT        task structure
       NT_PLATFORM          String from sysinfo(SI_PLATFORM)
       NT_AUXV              auxv array
       NT_GWINDOWS          gwindows struct
       NT_ASRS              asrset struct
       NT_PSTATUS           pstatus struct
       NT_PSINFO            psinfo struct
       NT_PRCRED            prcred struct
       NT_UTSNAME           utsname struct
       NT_LWPSTATUS         lwpstatus struct
       NT_LWPSINFO          lwpinfo struct
       NT_PRFPXREG          fprxregset struct
       NT_SIGINFO           siginfo_t (size might increase over
                            time)
       NT_FILE              Contains information about mapped
                            files
       NT_PRXFPREG          user_fxsr_struct
       NT_PPC_VMX           PowerPC Altivec/VMX registers
       NT_PPC_SPE           PowerPC SPE/EVR registers
       NT_PPC_VSX           PowerPC VSX registers
       NT_386_TLS           i386 TLS slots (struct user_desc)
       NT_386_IOPERM        x86 io permission bitmap (1=deny)
       NT_X86_XSTATE        x86 extended state using xsave
       NT_S390_HIGH_GPRS    s390 upper register halves
       NT_S390_TIMER        s390 timer register
       NT_S390_TODCMP       s390 time-of-day (TOD) clock
                            comparator register
       NT_S390_TODPREG      s390 time-of-day (TOD) programmable
                            register
       NT_S390_CTRS         s390 control registers
       NT_S390_PREFIX       s390 prefix register
       NT_S390_LAST_BREAK   s390 breaking event address
       NT_S390_SYSTEM_CALL  s390 system call restart data
       NT_S390_TDB          s390 transaction diagnostic block
       NT_ARM_VFP           ARM VFP/NEON registers
       NT_ARM_TLS           ARM TLS register
       NT_ARM_HW_BREAK      ARM hardware breakpoint registers
       NT_ARM_HW_WATCH      ARM hardware watchpoint registers
       NT_ARM_SYSTEM_CALL   ARM system call number

  n_name = GNU
       Extensions used by the GNU tool chain.

       NT_GNU_ABI_TAG
              Operating system (OS) ABI information.  The desc
              field will be 4 words:

              [0]  OS descriptor (ELF_NOTE_OS_LINUX,
                   ELF_NOTE_OS_GNU, and so on)`
```

```
                        [1]  major version of the ABI
                        [2]  minor version of the ABI
                        [3]  subminor version of the ABI

                NT_GNU_HWCAP
                        Synthetic hwcap information.  The desc field begins
                        with two words:

                        [0]  number of entries
                        [1]  bit mask of enabled entries

                        Then follow variable-length entries, one byte
                        followed by a null-terminated hwcap name string.
                        The byte gives the bit number to test if enabled,
                        (1U << bit) & bit mask.

                NT_GNU_BUILD_ID
                        Unique build ID as generated by the GNU ld(1)
                        --build-id option.  The desc consists of any nonzero
                        number of bytes.

                NT_GNU_GOLD_VERSION
                        The desc contains the GNU Gold linker version used.

            Default/unknown namespace (e_type != ET_CORE)
                These are used when the namespace is the default (i.e.,
                n_namesz will be set to 0), or a fallback when the
                namespace is unknown.

                NT_VERSION  A version string of some sort.
                NT_ARCH     Architecture information.

NOTES
       ELF first appeared in System V.  The ELF format is an adopted standard.

       The extensions for e_phnum, e_shnum, and e_shstrndx respectively are
       Linux extensions.  Sun, BSD, and AMD64 also support them; for further
       information, look under SEE ALSO.

SEE ALSO
       as(1), elfedit(1), gdb(1), ld(1), nm(1), objcopy(1), objdump(1),
       patchelf(1), readelf(1), size(1), strings(1), strip(1), execve(2),
       dl_iterate_phdr(3), core(5), ld.so(8)

       Hewlett-Packard, Elf-64 Object File Format.

       Santa Cruz Operation, System V Application Binary Interface.

       UNIX System Laboratories, "Object Files", Executable and Linking Format
       (ELF).

       Sun Microsystems, Linker and Libraries Guide.

       AMD64 ABI Draft, System V Application Binary Interface AMD64
       Architecture Processor Supplement.
```

```
Linux man-pages 6.7              2024-02-25                          ELF(5)
```

---

errno(3)

```
errno(3)                   Library Functions Manual                   errno(3)

NAME
       errno - number of last error

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <errno.h>

DESCRIPTION
       The <errno.h> header file defines the integer variable errno, which is
       set by system calls and some library functions in the event of an error
       to indicate what went wrong.

   errno
       The value in errno is significant only when the return value of the
       call indicated an error (i.e., -1 from most system calls; -1 or NULL
       from most library functions); a function that succeeds is allowed to
       change errno.  The value of errno is never set to zero by any system
       call or library function.

       For some system calls and library functions (e.g., getpriority(2)), -1
       is a valid return on success.  In such cases, a successful return can
       be distinguished from an error return by setting errno to zero before
       the call, and then, if the call returns a status that indicates that an
       error may have occurred, checking to see if errno has a nonzero value.

       errno is defined by the ISO C standard to be a modifiable lvalue of
       type int, and must not be explicitly declared; errno may be a macro.
       errno is thread-local; setting it in one thread does not affect its
       value in any other thread.

   Error numbers and names
       Valid error numbers are all positive numbers.  The <errno.h> header
       file defines symbolic names for each of the possible error numbers that
       may appear in errno.

       All the error names specified by POSIX.1 must have distinct values,
       with the exception of EAGAIN and EWOULDBLOCK, which may be the same.
       On Linux, these two have the same value on all architectures.

       The error numbers that correspond to each symbolic name vary across
       UNIX systems, and even across different architectures on Linux.
       Therefore, numeric values are not included as part of the list of error
       names below.  The perror(3) and strerror(3) functions can be used to
       convert these names to corresponding textual error messages.
```

```
    On any particular Linux system, one can obtain a list of all symbolic
    error names and the corresponding error numbers using the errno(1)
    command (part of the moreutils package):

        $ errno -l
        EPERM 1 Operation not permitted
        ENOENT 2 No such file or directory
        ESRCH 3 No such process
        EINTR 4 Interrupted system call
        EIO 5 Input/output error
        ...

    The errno(1) command can also be used to look up individual error
    numbers and names, and to search for errors using strings from the
    error description, as in the following examples:

        $ errno 2
        ENOENT 2 No such file or directory
        $ errno ESRCH
        ESRCH 3 No such process
        $ errno -s permission
        EACCES 13 Permission denied

List of error names
    In the list of the symbolic error names below, various names are marked
    as follows:

    POSIX.1-2001
           The name is defined by POSIX.1-2001, and is defined in later
           POSIX.1 versions, unless otherwise indicated.

    POSIX.1-2008
           The name is defined in POSIX.1-2008, but was not present in
           earlier POSIX.1 standards.

    C99    The name is defined by C99.

    Below is a list of the symbolic error names that are defined on Linux:

    E2BIG           Argument list too long (POSIX.1-2001).

    EACCES          Permission denied (POSIX.1-2001).

    EADDRINUSE      Address already in use (POSIX.1-2001).

    EADDRNOTAVAIL   Address not available (POSIX.1-2001).

    EAFNOSUPPORT    Address family not supported (POSIX.1-2001).

    EAGAIN          Resource temporarily unavailable (may be the same value
                    as EWOULDBLOCK) (POSIX.1-2001).

    EALREADY        Connection already in progress (POSIX.1-2001).
```

```
       EBADE           Invalid exchange.

       EBADF           Bad file descriptor (POSIX.1-2001).

       EBADFD          File descriptor in bad state.

       EBADMSG         Bad message (POSIX.1-2001).

       EBADR           Invalid request descriptor.

       EBADRQC         Invalid request code.

       EBADSLT         Invalid slot.

       EBUSY           Device or resource busy (POSIX.1-2001).

       ECANCELED       Operation canceled (POSIX.1-2001).

       ECHILD          No child processes (POSIX.1-2001).

       ECHRNG          Channel number out of range.

       ECOMM           Communication error on send.

       ECONNABORTED    Connection aborted (POSIX.1-2001).

       ECONNREFUSED    Connection refused (POSIX.1-2001).

       ECONNRESET      Connection reset (POSIX.1-2001).

       EDEADLK         Resource deadlock avoided (POSIX.1-2001).

       EDEADLOCK       On most architectures, a synonym for EDEADLK.  On some
                       architectures (e.g., Linux MIPS, PowerPC, SPARC), it is
                       a separate error code "File locking deadlock error".

       EDESTADDRREQ    Destination address required (POSIX.1-2001).

       EDOM            Mathematics argument out of domain of function
                       (POSIX.1, C99).

       EDQUOT          Disk quota exceeded (POSIX.1-2001).

       EEXIST          File exists (POSIX.1-2001).

       EFAULT          Bad address (POSIX.1-2001).

       EFBIG           File too large (POSIX.1-2001).

       EHOSTDOWN       Host is down.

       EHOSTUNREACH    Host is unreachable (POSIX.1-2001).

       EHWPOISON       Memory page has hardware error.
```

```
       EIDRM           Identifier removed (POSIX.1-2001).

       EILSEQ          Invalid or incomplete multibyte or wide character
                       (POSIX.1, C99).

                       The text shown here is the glibc error description; in
                       POSIX.1, this error is described as "Illegal byte
                       sequence".

       EINPROGRESS     Operation in progress (POSIX.1-2001).

       EINTR           Interrupted function call (POSIX.1-2001); see
                       signal(7).

       EINVAL          Invalid argument (POSIX.1-2001).

       EIO             Input/output error (POSIX.1-2001).

       EISCONN         Socket is connected (POSIX.1-2001).

       EISDIR          Is a directory (POSIX.1-2001).

       EISNAM          Is a named type file.

       EKEYEXPIRED     Key has expired.

       EKEYREJECTED    Key was rejected by service.

       EKEYREVOKED     Key has been revoked.

       EL2HLT          Level 2 halted.

       EL2NSYNC        Level 2 not synchronized.

       EL3HLT          Level 3 halted.

       EL3RST          Level 3 reset.

       ELIBACC         Cannot access a needed shared library.

       ELIBBAD         Accessing a corrupted shared library.

       ELIBMAX         Attempting to link in too many shared libraries.

       ELIBSCN         .lib section in a.out corrupted

       ELIBEXEC        Cannot exec a shared library directly.

       ELNRNG          Link number out of range.

       ELOOP           Too many levels of symbolic links (POSIX.1-2001).

       EMEDIUMTYPE     Wrong medium type.

       EMFILE          Too many open files (POSIX.1-2001).  Commonly caused by
```

```
                exceeding the RLIMIT_NOFILE resource limit described in
                getrlimit(2).  Can also be caused by exceeding the
                limit specified in /proc/sys/fs/nr_open.

    EMLINK      Too many links (POSIX.1-2001).

    EMSGSIZE    Message too long (POSIX.1-2001).

    EMULTIHOP   Multihop attempted (POSIX.1-2001).

    ENAMETOOLONG Filename too long (POSIX.1-2001).

    ENETDOWN    Network is down (POSIX.1-2001).

    ENETRESET   Connection aborted by network (POSIX.1-2001).

    ENETUNREACH Network unreachable (POSIX.1-2001).

    ENFILE      Too many open files in system (POSIX.1-2001).  On
                Linux, this is probably a result of encountering the
                /proc/sys/fs/file-max limit (see proc(5)).

    ENOANO      No anode.

    ENOBUFS     No buffer space available (POSIX.1 (XSI STREAMS
                option)).

    ENODATA     The named attribute does not exist, or the process has
                no access to this attribute; see xattr(7).

                In POSIX.1-2001 (XSI STREAMS option), this error was
                described as "No message is available on the STREAM
                head read queue".

    ENODEV      No such device (POSIX.1-2001).

    ENOENT      No such file or directory (POSIX.1-2001).

                Typically, this error results when a specified pathname
                does not exist, or one of the components in the
                directory prefix of a pathname does not exist, or the
                specified pathname is a dangling symbolic link.

    ENOEXEC     Exec format error (POSIX.1-2001).

    ENOKEY      Required key not available.

    ENOLCK      No locks available (POSIX.1-2001).

    ENOLINK     Link has been severed (POSIX.1-2001).

    ENOMEDIUM   No medium found.

    ENOMEM      Not enough space/cannot allocate memory (POSIX.1-2001).
```

```
       ENOMSG          No message of the desired type (POSIX.1-2001).

       ENONET          Machine is not on the network.

       ENOPKG          Package not installed.

       ENOPROTOOPT     Protocol not available (POSIX.1-2001).

       ENOSPC          No space left on device (POSIX.1-2001).

       ENOSR           No STREAM resources (POSIX.1 (XSI STREAMS option)).

       ENOSTR          Not a STREAM (POSIX.1 (XSI STREAMS option)).

       ENOSYS          Function not implemented (POSIX.1-2001).

       ENOTBLK         Block device required.

       ENOTCONN        The socket is not connected (POSIX.1-2001).

       ENOTDIR         Not a directory (POSIX.1-2001).

       ENOTEMPTY       Directory not empty (POSIX.1-2001).

       ENOTRECOVERABLE State not recoverable (POSIX.1-2008).

       ENOTSOCK        Not a socket (POSIX.1-2001).

       ENOTSUP         Operation not supported (POSIX.1-2001).

       ENOTTY          Inappropriate I/O control operation (POSIX.1-2001).

       ENOTUNIQ        Name not unique on network.

       ENXIO           No such device or address (POSIX.1-2001).

       EOPNOTSUPP      Operation not supported on socket (POSIX.1-2001).

                       (ENOTSUP and EOPNOTSUPP have the same value on Linux,
                       but according to POSIX.1 these error values should be
                       distinct.)

       EOVERFLOW       Value too large to be stored in data type
                       (POSIX.1-2001).

       EOWNERDEAD      Owner died (POSIX.1-2008).

       EPERM           Operation not permitted (POSIX.1-2001).

       EPFNOSUPPORT    Protocol family not supported.

       EPIPE           Broken pipe (POSIX.1-2001).

       EPROTO          Protocol error (POSIX.1-2001).
```

```
       EPROTONOSUPPORT Protocol not supported (POSIX.1-2001).

       EPROTOTYPE      Protocol wrong type for socket (POSIX.1-2001).

       ERANGE          Result too large (POSIX.1, C99).

       EREMCHG         Remote address changed.

       EREMOTE         Object is remote.

       EREMOTEIO       Remote I/O error.

       ERESTART        Interrupted system call should be restarted.

       ERFKILL         Operation not possible due to RF-kill.

       EROFS           Read-only filesystem (POSIX.1-2001).

       ESHUTDOWN       Cannot send after transport endpoint shutdown.

       ESPIPE          Invalid seek (POSIX.1-2001).

       ESOCKTNOSUPPORT Socket type not supported.

       ESRCH           No such process (POSIX.1-2001).

       ESTALE          Stale file handle (POSIX.1-2001).

                       This error can occur for NFS and for other filesystems.

       ESTRPIPE        Streams pipe error.

       ETIME           Timer expired (POSIX.1 (XSI STREAMS option)).

                       (POSIX.1 says "STREAM ioctl(2) timeout".)

       ETIMEDOUT       Connection timed out (POSIX.1-2001).

       ETOOMANYREFS    Too many references: cannot splice.

       ETXTBSY         Text file busy (POSIX.1-2001).

       EUCLEAN         Structure needs cleaning.

       EUNATCH         Protocol driver not attached.

       EUSERS          Too many users.

       EWOULDBLOCK     Operation would block (may be same value as EAGAIN)
                       (POSIX.1-2001).

       EXDEV           Invalid cross-device link (POSIX.1-2001).

       EXFULL          Exchange full.
```

```
NOTES
       A common mistake is to do

           if (somecall() == -1) {
               printf("somecall() failed\n");
               if (errno == ...) { ... }
           }

       where errno no longer needs to have the value it had upon return from
       somecall() (i.e., it may have been changed by the printf(3)).  If the
       value of errno should be preserved across a library call, it must be
       saved:

           if (somecall() == -1) {
               int errsv = errno;
               printf("somecall() failed\n");
               if (errsv == ...) { ... }
           }

       Note that the POSIX threads APIs do not set errno on error.  Instead,
       on failure they return an error number as the function result.  These
       error numbers have the same meanings as the error numbers returned in
       errno by other APIs.

       On some ancient systems, <errno.h> was not present or did not declare
       errno, so that it was necessary to declare errno manually (i.e., extern
       int errno).  Do not do this.  It long ago ceased to be necessary, and
       it will cause problems with modern versions of the C library.

SEE ALSO
       errno(1), err(3), error(3), perror(3), strerror(3)

Linux man-pages 6.7              2023-10-31                          errno(3)
```

---

fcntl(2)

```
fcntl(2)                     System Calls Manual                     fcntl(2)

NAME
       fcntl - manipulate file descriptor

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <fcntl.h>

       int fcntl(int fd, int op, ... /* arg */ );

DESCRIPTION
       fcntl() performs one of the operations described below on the open file
       descriptor fd.  The operation is determined by op.
```

```
    fcntl() can take an optional third argument.  Whether or not this
    argument is required is determined by op.  The required argument type
    is indicated in parentheses after each op name (in most cases, the
    required type is int, and we identify the argument using the name arg),
    or void is specified if the argument is not required.

    Certain of the operations below are supported only since a particular
    Linux kernel version.  The preferred method of checking whether the
    host kernel supports a particular operation is to invoke fcntl() with
    the desired op value and then test whether the call failed with EINVAL,
    indicating that the kernel does not recognize this value.

Duplicating a file descriptor
    F_DUPFD (int)
           Duplicate the file descriptor fd using the lowest-numbered
           available file descriptor greater than or equal to arg.  This is
           different from dup2(2), which uses exactly the file descriptor
           specified.

           On success, the new file descriptor is returned.

           See dup(2) for further details.

    F_DUPFD_CLOEXEC (int; since Linux 2.6.24)
           As for F_DUPFD, but additionally set the close-on-exec flag for
           the duplicate file descriptor.  Specifying this flag permits a
           program to avoid an additional fcntl() F_SETFD operation to set
           the FD_CLOEXEC flag.  For an explanation of why this flag is
           useful, see the description of O_CLOEXEC in open(2).

File descriptor flags
    The following operations manipulate the flags associated with a file
    descriptor.  Currently, only one such flag is defined: FD_CLOEXEC, the
    close-on-exec flag.  If the FD_CLOEXEC bit is set, the file descriptor
    will automatically be closed during a successful execve(2).  (If the
    execve(2) fails, the file descriptor is left open.)  If the FD_CLOEXEC
    bit is not set, the file descriptor will remain open across an
    execve(2).

    F_GETFD (void)
           Return (as the function result) the file descriptor flags; arg
           is ignored.

    F_SETFD (int)
           Set the file descriptor flags to the value specified by arg.

    In multithreaded programs, using fcntl() F_SETFD to set the close-on-
    exec flag at the same time as another thread performs a fork(2) plus
    execve(2) is vulnerable to a race condition that may unintentionally
    leak the file descriptor to the program executed in the child process.
    See the discussion of the O_CLOEXEC flag in open(2) for details and a
    remedy to the problem.

File status flags
```

```
    Each open file description has certain associated status flags,
    initialized by open(2) and possibly modified by fcntl().  Duplicated
    file descriptors (made with dup(2), fcntl(F_DUPFD), fork(2), etc.)
    refer to the same open file description, and thus share the same file
    status flags.

    The file status flags and their semantics are described in open(2).

    F_GETFL (void)
           Return (as the function result) the file access mode and the
           file status flags; arg is ignored.

    F_SETFL (int)
           Set the file status flags to the value specified by arg.  File
           access mode (O_RDONLY, O_WRONLY, O_RDWR) and file creation flags
           (i.e., O_CREAT, O_EXCL, O_NOCTTY, O_TRUNC) in arg are ignored.
           On Linux, this operation can change only the O_APPEND, O_ASYNC,
           O_DIRECT, O_NOATIME, and O_NONBLOCK flags.  It is not possible
           to change the O_DSYNC and O_SYNC flags; see BUGS, below.

Advisory record locking
    Linux implements traditional ("process-associated") UNIX record locks,
    as standardized by POSIX.  For a Linux-specific alternative with better
    semantics, see the discussion of open file description locks below.

    F_SETLK, F_SETLKW, and F_GETLK are used to acquire, release, and test
    for the existence of record locks (also known as byte-range, file-
    segment, or file-region locks).  The third argument, lock, is a pointer
    to a structure that has at least the following fields (in unspecified
    order).

        struct flock {
            ...
            short l_type;    /* Type of lock: F_RDLCK,
                                F_WRLCK, F_UNLCK */
            short l_whence;  /* How to interpret l_start:
                                SEEK_SET, SEEK_CUR, SEEK_END */
            off_t l_start;   /* Starting offset for lock */
            off_t l_len;     /* Number of bytes to lock */
            pid_t l_pid;     /* PID of process blocking our lock
                                (set by F_GETLK and F_OFD_GETLK) */
            ...
        };

    The l_whence, l_start, and l_len fields of this structure specify the
    range of bytes we wish to lock.  Bytes past the end of the file may be
    locked, but not bytes before the start of the file.

    l_start is the starting offset for the lock, and is interpreted
    relative to either: the start of the file (if l_whence is SEEK_SET);
    the current file offset (if l_whence is SEEK_CUR); or the end of the
    file (if l_whence is SEEK_END).  In the final two cases, l_start can be
    a negative number provided the offset does not lie before the start of
    the file.
```

```
       l_len specifies the number of bytes to be locked.  If l_len is
       positive, then the range to be locked covers bytes l_start up to and
       including l_start+l_len-1.  Specifying 0 for l_len has the special
       meaning: lock all bytes starting at the location specified by l_whence
       and l_start through to the end of file, no matter how large the file
       grows.

       POSIX.1-2001 allows (but does not require) an implementation to support
       a negative l_len value; if l_len is negative, the interval described by
       lock covers bytes l_start+l_len up to and including l_start-1.  This is
       supported since Linux 2.4.21 and Linux 2.5.49.

       The l_type field can be used to place a read (F_RDLCK) or a write
       (F_WRLCK) lock on a file.  Any number of processes may hold a read lock
       (shared lock) on a file region, but only one process may hold a write
       lock (exclusive lock).  An exclusive lock excludes all other locks,
       both shared and exclusive.  A single process can hold only one type of
       lock on a file region; if a new lock is applied to an already-locked
       region, then the existing lock is converted to the new lock type.
       (Such conversions may involve splitting, shrinking, or coalescing with
       an existing lock if the byte range specified by the new lock does not
       precisely coincide with the range of the existing lock.)

       F_SETLK (struct flock *)
              Acquire a lock (when l_type is F_RDLCK or F_WRLCK) or release a
              lock (when l_type is F_UNLCK) on the bytes specified by the
              l_whence, l_start, and l_len fields of lock.  If a conflicting
              lock is held by another process, this call returns -1 and sets
              errno to EACCES or EAGAIN.  (The error returned in this case
              differs across implementations, so POSIX requires a portable
              application to check for both errors.)

       F_SETLKW (struct flock *)
              As for F_SETLK, but if a conflicting lock is held on the file,
              then wait for that lock to be released.  If a signal is caught
              while waiting, then the call is interrupted and (after the
              signal handler has returned) returns immediately (with return
              value -1 and errno set to EINTR; see signal(7)).

       F_GETLK (struct flock *)
              On input to this call, lock describes a lock we would like to
              place on the file.  If the lock could be placed, fcntl() does
              not actually place it, but returns F_UNLCK in the l_type field
              of lock and leaves the other fields of the structure unchanged.

              If one or more incompatible locks would prevent this lock being
              placed, then fcntl() returns details about one of those locks in
              the l_type, l_whence, l_start, and l_len fields of lock.  If the
              conflicting lock is a traditional (process-associated) record
              lock, then the l_pid field is set to the PID of the process
              holding that lock.  If the conflicting lock is an open file
              description lock, then l_pid is set to -1.  Note that the
              returned information may already be out of date by the time the
              caller inspects it.
```

```
       In order to place a read lock, fd must be open for reading.  In order
       to place a write lock, fd must be open for writing.  To place both
       types of lock, open a file read-write.

       When placing locks with F_SETLKW, the kernel detects deadlocks, whereby
       two or more processes have their lock requests mutually blocked by
       locks held by the other processes.  For example, suppose process A
       holds a write lock on byte 100 of a file, and process B holds a write
       lock on byte 200.  If each process then attempts to lock the byte
       already locked by the other process using F_SETLKW, then, without
       deadlock detection, both processes would remain blocked indefinitely.
       When the kernel detects such deadlocks, it causes one of the blocking
       lock requests to immediately fail with the error EDEADLK; an
       application that encounters such an error should release some of its
       locks to allow other applications to proceed before attempting regain
       the locks that it requires.  Circular deadlocks involving more than two
       processes are also detected.  Note, however, that there are limitations
       to the kernel's deadlock-detection algorithm; see BUGS.

       As well as being removed by an explicit F_UNLCK, record locks are
       automatically released when the process terminates.

       Record locks are not inherited by a child created via fork(2), but are
       preserved across an execve(2).

       Because of the buffering performed by the stdio(3) library, the use of
       record locking with routines in that package should be avoided; use
       read(2) and write(2) instead.

       The record locks described above are associated with the process
       (unlike the open file description locks described below).  This has
       some unfortunate consequences:

       •  If a process closes any file descriptor referring to a file, then
          all of the process's locks on that file are released, regardless of
          the file descriptor(s) on which the locks were obtained.  This is
          bad: it means that a process can lose its locks on a file such as
          /etc/passwd or /etc/mtab when for some reason a library function
          decides to open, read, and close the same file.

       •  The threads in a process share locks.  In other words, a
          multithreaded program can't use record locking to ensure that
          threads don't simultaneously access the same region of a file.

       Open file description locks solve both of these problems.

   Open file description locks (non-POSIX)
       Open file description locks are advisory byte-range locks whose
       operation is in most respects identical to the traditional record locks
       described above.  This lock type is Linux-specific, and available since
       Linux 3.15.  (There is a proposal with the Austin Group to include this
       lock type in the next revision of POSIX.1.)  For an explanation of open
       file descriptions, see open(2).

       The principal difference between the two lock types is that whereas
```

```
       traditional record locks are associated with a process, open file
       description locks are associated with the open file description on
       which they are acquired, much like locks acquired with flock(2).
       Consequently (and unlike traditional advisory record locks), open file
       description locks are inherited across fork(2) (and clone(2) with
       CLONE_FILES), and are only automatically released on the last close of
       the open file description, instead of being released on any close of
       the file.

       Conflicting lock combinations (i.e., a read lock and a write lock or
       two write locks) where one lock is an open file description lock and
       the other is a traditional record lock conflict even when they are
       acquired by the same process on the same file descriptor.

       Open file description locks placed via the same open file description
       (i.e., via the same file descriptor, or via a duplicate of the file
       descriptor created by fork(2), dup(2), fcntl() F_DUPFD, and so on) are
       always compatible: if a new lock is placed on an already locked region,
       then the existing lock is converted to the new lock type.  (Such
       conversions may result in splitting, shrinking, or coalescing with an
       existing lock as discussed above.)

       On the other hand, open file description locks may conflict with each
       other when they are acquired via different open file descriptions.
       Thus, the threads in a multithreaded program can use open file
       description locks to synchronize access to a file region by having each
       thread perform its own open(2) on the file and applying locks via the
       resulting file descriptor.

       As with traditional advisory locks, the third argument to fcntl(),
       lock, is a pointer to an flock structure.  By contrast with traditional
       record locks, the l_pid field of that structure must be set to zero
       when using the operations described below.

       The operations for working with open file description locks are
       analogous to those used with traditional locks:

       F_OFD_SETLK (struct flock *)
              Acquire an open file description lock (when l_type is F_RDLCK or
              F_WRLCK) or release an open file description lock (when l_type
              is F_UNLCK) on the bytes specified by the l_whence, l_start, and
              l_len fields of lock.  If a conflicting lock is held by another
              process, this call returns -1 and sets errno to EAGAIN.

       F_OFD_SETLKW (struct flock *)
              As for F_OFD_SETLK, but if a conflicting lock is held on the
              file, then wait for that lock to be released.  If a signal is
              caught while waiting, then the call is interrupted and (after
              the signal handler has returned) returns immediately (with
              return value -1 and errno set to EINTR; see signal(7)).

       F_OFD_GETLK (struct flock *)
              On input to this call, lock describes an open file description
              lock we would like to place on the file.  If the lock could be
              placed, fcntl() does not actually place it, but returns F_UNLCK
```

```
              in the l_type field of lock and leaves the other fields of the
              structure unchanged.  If one or more incompatible locks would
              prevent this lock being placed, then details about one of these
              locks are returned via lock, as described above for F_GETLK.

      In the current implementation, no deadlock detection is performed for
      open file description locks.  (This contrasts with process-associated
      record locks, for which the kernel does perform deadlock detection.)

  Mandatory locking
      Warning: the Linux implementation of mandatory locking is unreliable.
      See BUGS below.  Because of these bugs, and the fact that the feature
      is believed to be little used, since Linux 4.5, mandatory locking has
      been made an optional feature, governed by a configuration option
      (CONFIG_MANDATORY_FILE_LOCKING).  This feature is no longer supported
      at all in Linux 5.15 and above.

      By default, both traditional (process-associated) and open file
      description record locks are advisory.  Advisory locks are not enforced
      and are useful only between cooperating processes.

      Both lock types can also be mandatory.  Mandatory locks are enforced
      for all processes.  If a process tries to perform an incompatible
      access (e.g., read(2) or write(2)) on a file region that has an
      incompatible mandatory lock, then the result depends upon whether the
      O_NONBLOCK flag is enabled for its open file description.  If the
      O_NONBLOCK flag is not enabled, then the system call is blocked until
      the lock is removed or converted to a mode that is compatible with the
      access.  If the O_NONBLOCK flag is enabled, then the system call fails
      with the error EAGAIN.

      To make use of mandatory locks, mandatory locking must be enabled both
      on the filesystem that contains the file to be locked, and on the file
      itself.  Mandatory locking is enabled on a filesystem using the "-o
      mand" option to mount(8), or the MS_MANDLOCK flag for mount(2).
      Mandatory locking is enabled on a file by disabling group execute
      permission on the file and enabling the set-group-ID permission bit
      (see chmod(1) and chmod(2)).

      Mandatory locking is not specified by POSIX.  Some other systems also
      support mandatory locking, although the details of how to enable it
      vary across systems.

  Lost locks
      When an advisory lock is obtained on a networked filesystem such as NFS
      it is possible that the lock might get lost.  This may happen due to
      administrative action on the server, or due to a network partition
      (i.e., loss of network connectivity with the server) which lasts long
      enough for the server to assume that the client is no longer
      functioning.

      When the filesystem determines that a lock has been lost, future
      read(2) or write(2) requests may fail with the error EIO.  This error
      will persist until the lock is removed or the file descriptor is
      closed.  Since Linux 3.12, this happens at least for NFSv4 (including
```

```
      all minor versions).

      Some versions of UNIX send a signal (SIGLOST) in this circumstance.
      Linux does not define this signal, and does not provide any
      asynchronous notification of lost locks.

  Managing signals
      F_GETOWN, F_SETOWN, F_GETOWN_EX, F_SETOWN_EX, F_GETSIG, and F_SETSIG
      are used to manage I/O availability signals:

      F_GETOWN (void)
             Return (as the function result) the process ID or process group
             ID currently receiving SIGIO and SIGURG signals for events on
             file descriptor fd.  Process IDs are returned as positive
             values; process group IDs are returned as negative values (but
             see BUGS below).  arg is ignored.

      F_SETOWN (int)
             Set the process ID or process group ID that will receive SIGIO
             and SIGURG signals for events on the file descriptor fd.  The
             target process or process group ID is specified in arg.  A
             process ID is specified as a positive value; a process group ID
             is specified as a negative value.  Most commonly, the calling
             process specifies itself as the owner (that is, arg is specified
             as getpid(2)).

             As well as setting the file descriptor owner, one must also
             enable generation of signals on the file descriptor.  This is
             done by using the fcntl() F_SETFL operation to set the O_ASYNC
             file status flag on the file descriptor.  Subsequently, a SIGIO
             signal is sent whenever input or output becomes possible on the
             file descriptor.  The fcntl() F_SETSIG operation can be used to
             obtain delivery of a signal other than SIGIO.

             Sending a signal to the owner process (group) specified by
             F_SETOWN is subject to the same permissions checks as are
             described for kill(2), where the sending process is the one that
             employs F_SETOWN (but see BUGS below).  If this permission check
             fails, then the signal is silently discarded.  Note: The
             F_SETOWN operation records the caller's credentials at the time
             of the fcntl() call, and it is these saved credentials that are
             used for the permission checks.

             If the file descriptor fd refers to a socket, F_SETOWN also
             selects the recipient of SIGURG signals that are delivered when
             out-of-band data arrives on that socket.  (SIGURG is sent in any
             situation where select(2) would report the socket as having an
             "exceptional condition".)

             The following was true in Linux 2.6.x up to and including Linux
             2.6.11:

                    If a nonzero value is given to F_SETSIG in a
                    multithreaded process running with a threading library
                    that supports thread groups (e.g., NPTL), then a positive
```

```
                value given to F_SETOWN has a different meaning: instead
                of being a process ID identifying a whole process, it is
                a thread ID identifying a specific thread within a
                process.  Consequently, it may be necessary to pass
                F_SETOWN the result of gettid(2) instead of getpid(2) to
                get sensible results when F_SETSIG is used.  (In current
                Linux threading implementations, a main thread's thread
                ID is the same as its process ID.  This means that a
                single-threaded program can equally use gettid(2) or
                getpid(2) in this scenario.)  Note, however, that the
                statements in this paragraph do not apply to the SIGURG
                signal generated for out-of-band data on a socket: this
                signal is always sent to either a process or a process
                group, depending on the value given to F_SETOWN.

        The above behavior was accidentally dropped in Linux 2.6.12, and
        won't be restored.  From Linux 2.6.32 onward, use F_SETOWN_EX to
        target SIGIO and SIGURG signals at a particular thread.

    F_GETOWN_EX (struct f_owner_ex *) (since Linux 2.6.32)
        Return the current file descriptor owner settings as defined by
        a previous F_SETOWN_EX operation.  The information is returned
        in the structure pointed to by arg, which has the following
        form:

            struct f_owner_ex {
                int   type;
                pid_t pid;
            };

        The type field will have one of the values F_OWNER_TID,
        F_OWNER_PID, or F_OWNER_PGRP.  The pid field is a positive
        integer representing a thread ID, process ID, or process group
        ID.  See F_SETOWN_EX for more details.

    F_SETOWN_EX (struct f_owner_ex *) (since Linux 2.6.32)
        This operation performs a similar task to F_SETOWN.  It allows
        the caller to direct I/O availability signals to a specific
        thread, process, or process group.  The caller specifies the
        target of signals via arg, which is a pointer to a f_owner_ex
        structure.  The type field has one of the following values,
        which define how pid is interpreted:

        F_OWNER_TID
                Send the signal to the thread whose thread ID (the value
                returned by a call to clone(2) or gettid(2)) is specified
                in pid.

        F_OWNER_PID
                Send the signal to the process whose ID is specified in
                pid.

        F_OWNER_PGRP
                Send the signal to the process group whose ID is
                specified in pid.  (Note that, unlike with F_SETOWN, a
```

```
                process group ID is specified as a positive value here.)

       F_GETSIG (void)
              Return (as the function result) the signal sent when input or
              output becomes possible.  A value of zero means SIGIO is sent.
              Any other value (including SIGIO) is the signal sent instead,
              and in this case additional info is available to the signal
              handler if installed with SA_SIGINFO.  arg is ignored.

       F_SETSIG (int)
              Set the signal sent when input or output becomes possible to the
              value given in arg.  A value of zero means to send the default
              SIGIO signal.  Any other value (including SIGIO) is the signal
              to send instead, and in this case additional info is available
              to the signal handler if installed with SA_SIGINFO.

              By using F_SETSIG with a nonzero value, and setting SA_SIGINFO
              for the signal handler (see sigaction(2)), extra information
              about I/O events is passed to the handler in a siginfo_t
              structure.  If the si_code field indicates the source is
              SI_SIGIO, the si_fd field gives the file descriptor associated
              with the event.  Otherwise, there is no indication which file
              descriptors are pending, and you should use the usual mechanisms
              (select(2), poll(2), read(2) with O_NONBLOCK set etc.) to
              determine which file descriptors are available for I/O.

              Note that the file descriptor provided in si_fd is the one that
              was specified during the F_SETSIG operation.  This can lead to
              an unusual corner case.  If the file descriptor is duplicated
              (dup(2) or similar), and the original file descriptor is closed,
              then I/O events will continue to be generated, but the si_fd
              field will contain the number of the now closed file descriptor.

              By selecting a real time signal (value >= SIGRTMIN), multiple
              I/O events may be queued using the same signal numbers.
              (Queuing is dependent on available memory.)  Extra information
              is available if SA_SIGINFO is set for the signal handler, as
              above.

              Note that Linux imposes a limit on the number of real-time
              signals that may be queued to a process (see getrlimit(2) and
              signal(7)) and if this limit is reached, then the kernel reverts
              to delivering SIGIO, and this signal is delivered to the entire
              process rather than to a specific thread.

       Using these mechanisms, a program can implement fully asynchronous I/O
       without using select(2) or poll(2) most of the time.

       The use of O_ASYNC is specific to BSD and Linux.  The only use of
       F_GETOWN and F_SETOWN specified in POSIX.1 is in conjunction with the
       use of the SIGURG signal on sockets.  (POSIX does not specify the SIGIO
       signal.)  F_GETOWN_EX, F_SETOWN_EX, F_GETSIG, and F_SETSIG are Linux-
       specific.  POSIX has asynchronous I/O and the aio_sigevent structure to
       achieve similar things; these are also available in Linux as part of
       the GNU C Library (glibc).
```

```
Leases
    F_SETLEASE and F_GETLEASE (Linux 2.4 onward) are used to establish a
    new lease, and retrieve the current lease, on the open file description
    referred to by the file descriptor fd.  A file lease provides a
    mechanism whereby the process holding the lease (the "lease holder") is
    notified (via delivery of a signal) when a process (the "lease
    breaker") tries to open(2) or truncate(2) the file referred to by that
    file descriptor.

    F_SETLEASE (int)
           Set or remove a file lease according to which of the following
           values is specified in the integer arg:

           F_RDLCK
                  Take out a read lease.  This will cause the calling
                  process to be notified when the file is opened for
                  writing or is truncated.  A read lease can be placed only
                  on a file descriptor that is opened read-only.

           F_WRLCK
                  Take out a write lease.  This will cause the caller to be
                  notified when the file is opened for reading or writing
                  or is truncated.  A write lease may be placed on a file
                  only if there are no other open file descriptors for the
                  file.

           F_UNLCK
                  Remove our lease from the file.

    Leases are associated with an open file description (see open(2)).
    This means that duplicate file descriptors (created by, for example,
    fork(2) or dup(2)) refer to the same lease, and this lease may be
    modified or released using any of these descriptors.  Furthermore, the
    lease is released by either an explicit F_UNLCK operation on any of
    these duplicate file descriptors, or when all such file descriptors
    have been closed.

    Leases may be taken out only on regular files.  An unprivileged process
    may take out a lease only on a file whose UID (owner) matches the
    filesystem UID of the process.  A process with the CAP_LEASE capability
    may take out leases on arbitrary files.

    F_GETLEASE (void)
           Indicates what type of lease is associated with the file
           descriptor fd by returning either F_RDLCK, F_WRLCK, or F_UNLCK,
           indicating, respectively, a read lease , a write lease, or no
           lease.  arg is ignored.

    When a process (the "lease breaker") performs an open(2) or truncate(2)
    that conflicts with a lease established via F_SETLEASE, the system call
    is blocked by the kernel and the kernel notifies the lease holder by
    sending it a signal (SIGIO by default).  The lease holder should
    respond to receipt of this signal by doing whatever cleanup is required
    in preparation for the file to be accessed by another process (e.g.,
```

```
       flushing cached buffers) and then either remove or downgrade its lease.
       A lease is removed by performing an F_SETLEASE operation specifying arg
       as F_UNLCK.  If the lease holder currently holds a write lease on the
       file, and the lease breaker is opening the file for reading, then it is
       sufficient for the lease holder to downgrade the lease to a read lease.
       This is done by performing an F_SETLEASE operation specifying arg as
       F_RDLCK.

       If the lease holder fails to downgrade or remove the lease within the
       number of seconds specified in /proc/sys/fs/lease-break-time, then the
       kernel forcibly removes or downgrades the lease holder's lease.

       Once a lease break has been initiated, F_GETLEASE returns the target
       lease type (either F_RDLCK or F_UNLCK, depending on what would be
       compatible with the lease breaker) until the lease holder voluntarily
       downgrades or removes the lease or the kernel forcibly does so after
       the lease break timer expires.

       Once the lease has been voluntarily or forcibly removed or downgraded,
       and assuming the lease breaker has not unblocked its system call, the
       kernel permits the lease breaker's system call to proceed.

       If the lease breaker's blocked open(2) or truncate(2) is interrupted by
       a signal handler, then the system call fails with the error EINTR, but
       the other steps still occur as described above.  If the lease breaker
       is killed by a signal while blocked in open(2) or truncate(2), then the
       other steps still occur as described above.  If the lease breaker
       specifies the O_NONBLOCK flag when calling open(2), then the call
       immediately fails with the error EWOULDBLOCK, but the other steps still
       occur as described above.

       The default signal used to notify the lease holder is SIGIO, but this
       can be changed using the F_SETSIG operation to fcntl().  If a F_SETSIG
       operation is performed (even one specifying SIGIO), and the signal
       handler is established using SA_SIGINFO, then the handler will receive
       a siginfo_t structure as its second argument, and the si_fd field of
       this argument will hold the file descriptor of the leased file that has
       been accessed by another process.  (This is useful if the caller holds
       leases against multiple files.)

   File and directory change notification (dnotify)
       F_NOTIFY (int)
              (Linux 2.4 onward) Provide notification when the directory
              referred to by fd or any of the files that it contains is
              changed.  The events to be notified are specified in arg, which
              is a bit mask specified by ORing together zero or more of the
              following bits:

              DN_ACCESS
                     A file was accessed (read(2), pread(2), readv(2), and
                     similar)
              DN_MODIFY
                     A file was modified (write(2), pwrite(2), writev(2),
                     truncate(2), ftruncate(2), and similar).
              DN_CREATE
```

```
                A file was created (open(2), creat(2), mknod(2),
                mkdir(2), link(2), symlink(2), rename(2) into this
                directory).
            DN_DELETE
                A file was unlinked (unlink(2), rename(2) to another
                directory, rmdir(2)).
            DN_RENAME
                A file was renamed within this directory (rename(2)).
            DN_ATTRIB
                The attributes of a file were changed (chown(2),
                chmod(2), utime(2), utimensat(2), and similar).

            (In order to obtain these definitions, the _GNU_SOURCE feature
            test macro must be defined before including any header files.)

            Directory notifications are normally "one-shot", and the
            application must reregister to receive further notifications.
            Alternatively, if DN_MULTISHOT is included in arg, then
            notification will remain in effect until explicitly removed.

            A series of F_NOTIFY requests is cumulative, with the events in
            arg being added to the set already monitored.  To disable
            notification of all events, make an F_NOTIFY call specifying arg
            as 0.

            Notification occurs via delivery of a signal.  The default
            signal is SIGIO, but this can be changed using the F_SETSIG
            operation to fcntl().  (Note that SIGIO is one of the nonqueuing
            standard signals; switching to the use of a real-time signal
            means that multiple notifications can be queued to the process.)
            In the latter case, the signal handler receives a siginfo_t
            structure as its second argument (if the handler was established
            using SA_SIGINFO) and the si_fd field of this structure contains
            the file descriptor which generated the notification (useful
            when establishing notification on multiple directories).

            Especially when using DN_MULTISHOT, a real time signal should be
            used for notification, so that multiple notifications can be
            queued.

            NOTE: New applications should use the inotify interface
            (available since Linux 2.6.13), which provides a much superior
            interface for obtaining notifications of filesystem events.  See
            inotify(7).

  Changing the capacity of a pipe
      F_SETPIPE_SZ (int; since Linux 2.6.35)
            Change the capacity of the pipe referred to by fd to be at least
            arg bytes.  An unprivileged process can adjust the pipe capacity
            to any value between the system page size and the limit defined
            in /proc/sys/fs/pipe-max-size (see proc(5)).  Attempts to set
            the pipe capacity below the page size are silently rounded up to
            the page size.  Attempts by an unprivileged process to set the
            pipe capacity above the limit in /proc/sys/fs/pipe-max-size
            yield the error EPERM; a privileged process (CAP_SYS_RESOURCE)
```

```
              can override the limit.

              When allocating the buffer for the pipe, the kernel may use a
              capacity larger than arg, if that is convenient for the
              implementation.  (In the current implementation, the allocation
              is the next higher power-of-two page-size multiple of the
              requested size.)  The actual capacity (in bytes) that is set is
              returned as the function result.

              Attempting to set the pipe capacity smaller than the amount of
              buffer space currently used to store data produces the error
              EBUSY.

              Note that because of the way the pages of the pipe buffer are
              employed when data is written to the pipe, the number of bytes
              that can be written may be less than the nominal size, depending
              on the size of the writes.

       F_GETPIPE_SZ (void; since Linux 2.6.35)
              Return (as the function result) the capacity of the pipe
              referred to by fd.

   File Sealing
       File seals limit the set of allowed operations on a given file.  For
       each seal that is set on a file, a specific set of operations will fail
       with EPERM on this file from now on.  The file is said to be sealed.
       The default set of seals depends on the type of the underlying file and
       filesystem.  For an overview of file sealing, a discussion of its
       purpose, and some code examples, see memfd_create(2).

       Currently, file seals can be applied only to a file descriptor returned
       by memfd_create(2) (if the MFD_ALLOW_SEALING was employed).  On other
       filesystems, all fcntl() operations that operate on seals will return
       EINVAL.

       Seals are a property of an inode.  Thus, all open file descriptors
       referring to the same inode share the same set of seals.  Furthermore,
       seals can never be removed, only added.

       F_ADD_SEALS (int; since Linux 3.17)
              Add the seals given in the bit-mask argument arg to the set of
              seals of the inode referred to by the file descriptor fd.  Seals
              cannot be removed again.  Once this call succeeds, the seals are
              enforced by the kernel immediately.  If the current set of seals
              includes F_SEAL_SEAL (see below), then this call will be
              rejected with EPERM.  Adding a seal that is already set is a no-
              op, in case F_SEAL_SEAL is not set already.  In order to place a
              seal, the file descriptor fd must be writable.

       F_GET_SEALS (void; since Linux 3.17)
              Return (as the function result) the current set of seals of the
              inode referred to by fd.  If no seals are set, 0 is returned.
              If the file does not support sealing, -1 is returned and errno
              is set to EINVAL.
```

```
       The following seals are available:

       F_SEAL_SEAL
              If this seal is set, any further call to fcntl() with
              F_ADD_SEALS fails with the error EPERM.  Therefore, this seal
              prevents any modifications to the set of seals itself.  If the
              initial set of seals of a file includes F_SEAL_SEAL, then this
              effectively causes the set of seals to be constant and locked.

       F_SEAL_SHRINK
              If this seal is set, the file in question cannot be reduced in
              size.  This affects open(2) with the O_TRUNC flag as well as
              truncate(2) and ftruncate(2).  Those calls fail with EPERM if
              you try to shrink the file in question.  Increasing the file
              size is still possible.

       F_SEAL_GROW
              If this seal is set, the size of the file in question cannot be
              increased.  This affects write(2) beyond the end of the file,
              truncate(2), ftruncate(2), and fallocate(2).  These calls fail
              with EPERM if you use them to increase the file size.  If you
              keep the size or shrink it, those calls still work as expected.

       F_SEAL_WRITE
              If this seal is set, you cannot modify the contents of the file.
              Note that shrinking or growing the size of the file is still
              possible and allowed.  Thus, this seal is normally used in
              combination with one of the other seals.  This seal affects
              write(2) and fallocate(2) (only in combination with the
              FALLOC_FL_PUNCH_HOLE flag).  Those calls fail with EPERM if this
              seal is set.  Furthermore, trying to create new shared, writable
              memory-mappings via mmap(2) will also fail with EPERM.

              Using the F_ADD_SEALS operation to set the F_SEAL_WRITE seal
              fails with EBUSY if any writable, shared mapping exists.  Such
              mappings must be unmapped before you can add this seal.
              Furthermore, if there are any asynchronous I/O operations
              (io_submit(2)) pending on the file, all outstanding writes will
              be discarded.

       F_SEAL_FUTURE_WRITE (since Linux 5.1)
              The effect of this seal is similar to F_SEAL_WRITE, but the
              contents of the file can still be modified via shared writable
              mappings that were created prior to the seal being set.  Any
              attempt to create a new writable mapping on the file via mmap(2)
              will fail with EPERM.  Likewise, an attempt to write to the file
              via write(2) will fail with EPERM.

              Using this seal, one process can create a memory buffer that it
              can continue to modify while sharing that buffer on a "read-
              only" basis with other processes.

   File read/write hints
       Write lifetime hints can be used to inform the kernel about the
       relative expected lifetime of writes on a given inode or via a
```

```
       particular open file description.  (See open(2) for an explanation of
       open file descriptions.)  In this context, the term "write lifetime"
       means the expected time the data will live on media, before being
       overwritten or erased.

       An application may use the different hint values specified below to
       separate writes into different write classes, so that multiple users or
       applications running on a single storage back-end can aggregate their
       I/O patterns in a consistent manner.  However, there are no functional
       semantics implied by these flags, and different I/O classes can use the
       write lifetime hints in arbitrary ways, so long as the hints are used
       consistently.

       The following operations can be applied to the file descriptor, fd:

       F_GET_RW_HINT (uint64_t *; since Linux 4.13)
              Returns the value of the read/write hint associated with the
              underlying inode referred to by fd.

       F_SET_RW_HINT (uint64_t *; since Linux 4.13)
              Sets the read/write hint value associated with the underlying
              inode referred to by fd.  This hint persists until either it is
              explicitly modified or the underlying filesystem is unmounted.

       F_GET_FILE_RW_HINT (uint64_t *; since Linux 4.13)
              Returns the value of the read/write hint associated with the
              open file description referred to by fd.

       F_SET_FILE_RW_HINT (uint64_t *; since Linux 4.13)
              Sets the read/write hint value associated with the open file
              description referred to by fd.

       If an open file description has not been assigned a read/write hint,
       then it shall use the value assigned to the inode, if any.

       The following read/write hints are valid since Linux 4.13:

       RWH_WRITE_LIFE_NOT_SET
              No specific hint has been set.  This is the default value.

       RWH_WRITE_LIFE_NONE
              No specific write lifetime is associated with this file or
              inode.

       RWH_WRITE_LIFE_SHORT
              Data written to this inode or via this open file description is
              expected to have a short lifetime.

       RWH_WRITE_LIFE_MEDIUM
              Data written to this inode or via this open file description is
              expected to have a lifetime longer than data written with
              RWH_WRITE_LIFE_SHORT.

       RWH_WRITE_LIFE_LONG
              Data written to this inode or via this open file description is
```

```
              expected to have a lifetime longer than data written with
              RWH_WRITE_LIFE_MEDIUM.

       RWH_WRITE_LIFE_EXTREME
              Data written to this inode or via this open file description is
              expected to have a lifetime longer than data written with
              RWH_WRITE_LIFE_LONG.

       All the write-specific hints are relative to each other, and no
       individual absolute meaning should be attributed to them.

RETURN VALUE
       For a successful call, the return value depends on the operation:

       F_DUPFD
              The new file descriptor.

       F_GETFD
              Value of file descriptor flags.

       F_GETFL
              Value of file status flags.

       F_GETLEASE
              Type of lease held on file descriptor.

       F_GETOWN
              Value of file descriptor owner.

       F_GETSIG
              Value of signal sent when read or write becomes possible, or
              zero for traditional SIGIO behavior.

       F_GETPIPE_SZ
       F_SETPIPE_SZ
              The pipe capacity.

       F_GET_SEALS
              A bit mask identifying the seals that have been set for the
              inode referred to by fd.

       All other operations
              Zero.

       On error, -1 is returned, and errno is set to indicate the error.

ERRORS
       EACCES or EAGAIN
              Operation is prohibited by locks held by other processes.

       EAGAIN The operation is prohibited because the file has been memory-
              mapped by another process.

       EBADF  fd is not an open file descriptor
```

```
       EBADF  op is F_SETLK or F_SETLKW and the file descriptor open mode
              doesn't match with the type of lock requested.

       EBUSY  op is F_SETPIPE_SZ and the new pipe capacity specified in arg is
              smaller than the amount of buffer space currently used to store
              data in the pipe.

       EBUSY  op is F_ADD_SEALS, arg includes F_SEAL_WRITE, and there exists a
              writable, shared mapping on the file referred to by fd.

       EDEADLK
              It was detected that the specified F_SETLKW operation would
              cause a deadlock.

       EFAULT lock is outside your accessible address space.

       EINTR  op is F_SETLKW or F_OFD_SETLKW and the operation was interrupted
              by a signal; see signal(7).

       EINTR  op is F_GETLK, F_SETLK, F_OFD_GETLK, or F_OFD_SETLK, and the
              operation was interrupted by a signal before the lock was
              checked or acquired.  Most likely when locking a remote file
              (e.g., locking over NFS), but can sometimes happen locally.

       EINVAL The value specified in op is not recognized by this kernel.

       EINVAL op is F_ADD_SEALS and arg includes an unrecognized sealing bit.

       EINVAL op is F_ADD_SEALS or F_GET_SEALS and the filesystem containing
              the inode referred to by fd does not support sealing.

       EINVAL op is F_DUPFD and arg is negative or is greater than the maximum
              allowable value (see the discussion of RLIMIT_NOFILE in
              getrlimit(2)).

       EINVAL op is F_SETSIG and arg is not an allowable signal number.

       EINVAL op is F_OFD_SETLK, F_OFD_SETLKW, or F_OFD_GETLK, and l_pid was
              not specified as zero.

       EMFILE op is F_DUPFD and the per-process limit on the number of open
              file descriptors has been reached.

       ENOLCK Too many segment locks open, lock table is full, or a remote
              locking protocol failed (e.g., locking over NFS).

       ENOTDIR
              F_NOTIFY was specified in op, but fd does not refer to a
              directory.

       EPERM  op is F_SETPIPE_SZ and the soft or hard user pipe limit has been
              reached; see pipe(7).

       EPERM  Attempted to clear the O_APPEND flag on a file that has the
              append-only attribute set.
```

```
       EPERM  op was F_ADD_SEALS, but fd was not open for writing or the
              current set of seals on the file already includes F_SEAL_SEAL.

STANDARDS
       POSIX.1-2008.

       F_GETOWN_EX, F_SETOWN_EX, F_SETPIPE_SZ, F_GETPIPE_SZ, F_GETSIG,
       F_SETSIG, F_NOTIFY, F_GETLEASE, and F_SETLEASE are Linux-specific.
       (Define the _GNU_SOURCE macro to obtain these definitions.)

       F_OFD_SETLK, F_OFD_SETLKW, and F_OFD_GETLK are Linux-specific (and one
       must define _GNU_SOURCE to obtain their definitions), but work is being
       done to have them included in the next version of POSIX.1.

       F_ADD_SEALS and F_GET_SEALS are Linux-specific.

HISTORY
       SVr4, 4.3BSD, POSIX.1-2001.

       Only the operations F_DUPFD, F_GETFD, F_SETFD, F_GETFL, F_SETFL,
       F_GETLK, F_SETLK, and F_SETLKW are specified in POSIX.1-2001.

       F_GETOWN and F_SETOWN are specified in POSIX.1-2001.  (To get their
       definitions, define either _XOPEN_SOURCE with the value 500 or greater,
       or _POSIX_C_SOURCE with the value 200809L or greater.)

       F_DUPFD_CLOEXEC is specified in POSIX.1-2008.  (To get this definition,
       define _POSIX_C_SOURCE with the value 200809L or greater, or
       _XOPEN_SOURCE with the value 700 or greater.)

NOTES
       The errors returned by dup2(2) are different from those returned by
       F_DUPFD.

   File locking
       The original Linux fcntl() system call was not designed to handle large
       file offsets (in the flock structure).  Consequently, an fcntl64()
       system call was added in Linux 2.4.  The newer system call employs a
       different structure for file locking, flock64, and corresponding
       operations, F_GETLK64, F_SETLK64, and F_SETLKW64.  However, these
       details can be ignored by applications using glibc, whose fcntl()
       wrapper function transparently employs the more recent system call
       where it is available.

   Record locks
       Since Linux 2.0, there is no interaction between the types of lock
       placed by flock(2) and fcntl().

       Several systems have more fields in struct flock such as, for example,
       l_sysid (to identify the machine where the lock is held).  Clearly,
       l_pid alone is not going to be very useful if the process holding the
       lock may live on a different machine; on Linux, while present on some
       architectures (such as MIPS32), this field is not used.
```

```
       The original Linux fcntl() system call was not designed to handle large
       file offsets (in the flock structure).  Consequently, an fcntl64()
       system call was added in Linux 2.4.  The newer system call employs a
       different structure for file locking, flock64, and corresponding
       operations, F_GETLK64, F_SETLK64, and F_SETLKW64.  However, these
       details can be ignored by applications using glibc, whose fcntl()
       wrapper function transparently employs the more recent system call
       where it is available.

   Record locking and NFS
       Before Linux 3.12, if an NFSv4 client loses contact with the server for
       a period of time (defined as more than 90 seconds with no
       communication), it might lose and regain a lock without ever being
       aware of the fact.  (The period of time after which contact is assumed
       lost is known as the NFSv4 leasetime.  On a Linux NFS server, this can
       be determined by looking at /proc/fs/nfsd/nfsv4leasetime, which
       expresses the period in seconds.  The default value for this file is
       90.)  This scenario potentially risks data corruption, since another
       process might acquire a lock in the intervening period and perform file
       I/O.

       Since Linux 3.12, if an NFSv4 client loses contact with the server, any
       I/O to the file by a process which "thinks" it holds a lock will fail
       until that process closes and reopens the file.  A kernel parameter,
       nfs.recover_lost_locks, can be set to 1 to obtain the pre-3.12
       behavior, whereby the client will attempt to recover lost locks when
       contact is reestablished with the server.  Because of the attendant
       risk of data corruption, this parameter defaults to 0 (disabled).

BUGS
   F_SETFL
       It is not possible to use F_SETFL to change the state of the O_DSYNC
       and O_SYNC flags.  Attempts to change the state of these flags are
       silently ignored.

   F_GETOWN
       A limitation of the Linux system call conventions on some architectures
       (notably i386) means that if a (negative) process group ID to be
       returned by F_GETOWN falls in the range -1 to -4095, then the return
       value is wrongly interpreted by glibc as an error in the system call;
       that is, the return value of fcntl() will be -1, and errno will contain
       the (positive) process group ID.  The Linux-specific F_GETOWN_EX
       operation avoids this problem.  Since glibc 2.11, glibc makes the
       kernel F_GETOWN problem invisible by implementing F_GETOWN using
       F_GETOWN_EX.

   F_SETOWN
       In Linux 2.4 and earlier, there is bug that can occur when an
       unprivileged process uses F_SETOWN to specify the owner of a socket
       file descriptor as a process (group) other than the caller.  In this
       case, fcntl() can return -1 with errno set to EPERM, even when the
       owner process (group) is one that the caller has permission to send
       signals to.  Despite this error return, the file descriptor owner is
       set, and signals will be sent to the owner.
```

```
   Deadlock detection
       The deadlock-detection algorithm employed by the kernel when dealing
       with F_SETLKW requests can yield both false negatives (failures to
       detect deadlocks, leaving a set of deadlocked processes blocked
       indefinitely) and false positives (EDEADLK errors when there is no
       deadlock).  For example, the kernel limits the lock depth of its
       dependency search to 10 steps, meaning that circular deadlock chains
       that exceed that size will not be detected.  In addition, the kernel
       may falsely indicate a deadlock when two or more processes created
       using the clone(2) CLONE_FILES flag place locks that appear (to the
       kernel) to conflict.

   Mandatory locking
       The Linux implementation of mandatory locking is subject to race
       conditions which render it unreliable: a write(2) call that overlaps
       with a lock may modify data after the mandatory lock is acquired; a
       read(2) call that overlaps with a lock may detect changes to data that
       were made only after a write lock was acquired.  Similar races exist
       between mandatory locks and mmap(2).  It is therefore inadvisable to
       rely on mandatory locking.

SEE ALSO
       dup2(2), flock(2), open(2), socket(2), lockf(3), capabilities(7),
       feature_test_macros(7), lslocks(8)

       locks.txt, mandatory-locking.txt, and dnotify.txt in the Linux kernel
       source directory Documentation/filesystems/ (on older kernels, these
       files are directly under the Documentation/ directory, and
       mandatory-locking.txt is called mandatory.txt)

Linux man-pages 6.7              2024-03-03                          fcntl(2)
```

---

fsync(2)

```
fsync(2)                      System Calls Manual                     fsync(2)

NAME
       fsync, fdatasync - synchronize a file's in-core state with storage
       device

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       int fsync(int fd);

       int fdatasync(int fd);

   Feature Test Macro Requirements for glibc (see feature_test_macros(7)):
```

```
       fsync():
           glibc 2.16 and later:
               No feature test macros need be defined
           glibc up to and including 2.15:
               _BSD_SOURCE || _XOPEN_SOURCE
                   || /* Since glibc 2.8: */ _POSIX_C_SOURCE >= 200112L

       fdatasync():
           _POSIX_C_SOURCE >= 199309L || _XOPEN_SOURCE >= 500

DESCRIPTION
       fsync() transfers ("flushes") all modified in-core data of (i.e.,
       modified buffer cache pages for) the file referred to by the file
       descriptor fd to the disk device (or other permanent storage device) so
       that all changed information can be retrieved even if the system
       crashes or is rebooted.  This includes writing through or flushing a
       disk cache if present.  The call blocks until the device reports that
       the transfer has completed.

       As well as flushing the file data, fsync() also flushes the metadata
       information associated with the file (see inode(7)).

       Calling fsync() does not necessarily ensure that the entry in the
       directory containing the file has also reached disk.  For that an
       explicit fsync() on a file descriptor for the directory is also needed.

       fdatasync() is similar to fsync(), but does not flush modified metadata
       unless that metadata is needed in order to allow a subsequent data
       retrieval to be correctly handled.  For example, changes to st_atime or
       st_mtime (respectively, time of last access and time of last
       modification; see inode(7)) do not require flushing because they are
       not necessary for a subsequent data read to be handled correctly.  On
       the other hand, a change to the file size (st_size, as made by say
       ftruncate(2)), would require a metadata flush.

       The aim of fdatasync() is to reduce disk activity for applications that
       do not require all metadata to be synchronized with the disk.

RETURN VALUE
       On success, these system calls return zero.  On error, -1 is returned,
       and errno is set to indicate the error.

ERRORS
       EBADF  fd is not a valid open file descriptor.

       EINTR  The function was interrupted by a signal; see signal(7).

       EIO    An error occurred during synchronization.  This error may relate
              to data written to some other file descriptor on the same file.
              Since Linux 4.13, errors from write-back will be reported to all
              file descriptors that might have written the data which
              triggered the error.  Some filesystems (e.g., NFS) keep close
              track of which data came through which file descriptor, and give
              more precise reporting.  Other filesystems (e.g., most local
              filesystems) will report errors to all file descriptors that
```

```
              were open on the file when the error was recorded.

       ENOSPC Disk space was exhausted while synchronizing.

       EROFS
       EINVAL fd is bound to a special file (e.g., a pipe, FIFO, or socket)
              which does not support synchronization.

       ENOSPC
       EDQUOT fd is bound to a file on NFS or another filesystem which does
              not allocate space at the time of a write(2) system call, and
              some previous write failed due to insufficient storage space.

VERSIONS
       On POSIX systems on which fdatasync() is available,
       _POSIX_SYNCHRONIZED_IO is defined in <unistd.h> to a value greater than
       0.  (See also sysconf(3).)

STANDARDS
       POSIX.1-2008.

HISTORY
       POSIX.1-2001, 4.2BSD.

       In Linux 2.2 and earlier, fdatasync() is equivalent to fsync(), and so
       has no performance advantage.

       The fsync() implementations in older kernels and lesser used
       filesystems do not know how to flush disk caches.  In these cases disk
       caches need to be disabled using hdparm(8) or sdparm(8) to guarantee
       safe operation.

       Under AT&T UNIX System V Release 4 fd needs to be opened for writing.
       This is by itself incompatible with the original BSD interface and
       forbidden by POSIX, but nevertheless survives in HP-UX and AIX.

SEE ALSO
       sync(1), bdflush(2), open(2), posix_fadvise(2), pwritev(2), sync(2),
       sync_file_range(2), fflush(3), fileno(3), hdparm(8), mount(8)

Linux man-pages 6.7              2023-10-31                          fsync(2)
```

---

rename(2)

```
rename(2)                    System Calls Manual                    rename(2)

NAME
       rename, renameat, renameat2 - change the name or location of a file

LIBRARY
       Standard C library (libc, -lc)
```

```
SYNOPSIS
       #include <stdio.h>

       int rename(const char *oldpath, const char *newpath);

       #include <fcntl.h>           /* Definition of AT_* constants */
       #include <stdio.h>

       int renameat(int olddirfd, const char *oldpath,
                    int newdirfd, const char *newpath);
       int renameat2(int olddirfd, const char *oldpath,
                    int newdirfd, const char *newpath, unsigned int flags);

   Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

       renameat():
           Since glibc 2.10:
               _POSIX_C_SOURCE >= 200809L
           Before glibc 2.10:
               _ATFILE_SOURCE

       renameat2():
           _GNU_SOURCE

DESCRIPTION
       rename() renames a file, moving it between directories if required.
       Any other hard links to the file (as created using link(2)) are
       unaffected.  Open file descriptors for oldpath are also unaffected.

       Various restrictions determine whether or not the rename operation
       succeeds: see ERRORS below.

       If newpath already exists, it will be atomically replaced, so that
       there is no point at which another process attempting to access newpath
       will find it missing.  However, there will probably be a window in
       which both oldpath and newpath refer to the file being renamed.

       If oldpath and newpath are existing hard links referring to the same
       file, then rename() does nothing, and returns a success status.

       If newpath exists but the operation fails for some reason, rename()
       guarantees to leave an instance of newpath in place.

       oldpath can specify a directory.  In this case, newpath must either not
       exist, or it must specify an empty directory.

       If oldpath refers to a symbolic link, the link is renamed; if newpath
       refers to a symbolic link, the link will be overwritten.

   renameat()
       The renameat() system call operates in exactly the same way as
       rename(), except for the differences described here.

       If the pathname given in oldpath is relative, then it is interpreted
       relative to the directory referred to by the file descriptor olddirfd
```

```
      (rather than relative to the current working directory of the calling
      process, as is done by rename() for a relative pathname).

      If oldpath is relative and olddirfd is the special value AT_FDCWD, then
      oldpath is interpreted relative to the current working directory of the
      calling process (like rename()).

      If oldpath is absolute, then olddirfd is ignored.

      The interpretation of newpath is as for oldpath, except that a relative
      pathname is interpreted relative to the directory referred to by the
      file descriptor newdirfd.

      See openat(2) for an explanation of the need for renameat().

  renameat2()
      renameat2() has an additional flags argument.  A renameat2() call with
      a zero flags argument is equivalent to renameat().

      The flags argument is a bit mask consisting of zero or more of the
      following flags:

      RENAME_EXCHANGE
             Atomically exchange oldpath and newpath.  Both pathnames must
             exist but may be of different types (e.g., one could be a non-
             empty directory and the other a symbolic link).

      RENAME_NOREPLACE
             Don't overwrite newpath of the rename.  Return an error if
             newpath already exists.

             RENAME_NOREPLACE can't be employed together with
             RENAME_EXCHANGE.

             RENAME_NOREPLACE requires support from the underlying
             filesystem.  Support for various filesystems was added as
             follows:

             • ext4 (Linux 3.15);

             • btrfs, tmpfs, and cifs (Linux 3.17);

             • xfs (Linux 4.0);

             • Support for many other filesystems was added in Linux 4.9,
               including ext2, minix, reiserfs, jfs, vfat, and bpf.

      RENAME_WHITEOUT (since Linux 3.18)
             This operation makes sense only for overlay/union filesystem
             implementations.

             Specifying RENAME_WHITEOUT creates a "whiteout" object at the
             source of the rename at the same time as performing the rename.
             The whole operation is atomic, so that if the rename succeeds
             then the whiteout will also have been created.
```

```
              A "whiteout" is an object that has special meaning in
              union/overlay filesystem constructs.  In these constructs,
              multiple layers exist and only the top one is ever modified.  A
              whiteout on an upper layer will effectively hide a matching file
              in the lower layer, making it appear as if the file didn't
              exist.

              When a file that exists on the lower layer is renamed, the file
              is first copied up (if not already on the upper layer) and then
              renamed on the upper, read-write layer.  At the same time, the
              source file needs to be "whiteouted" (so that the version of the
              source file in the lower layer is rendered invisible).  The
              whole operation needs to be done atomically.

              When not part of a union/overlay, the whiteout appears as a
              character device with a {0,0} device number.  (Note that other
              union/overlay implementations may employ different methods for
              storing whiteout entries; specifically, BSD union mount employs
              a separate inode type, DT_WHT, which, while supported by some
              filesystems available in Linux, such as CODA and XFS, is ignored
              by the kernel's whiteout support code, as of Linux 4.19, at
              least.)

              RENAME_WHITEOUT requires the same privileges as creating a
              device node (i.e., the CAP_MKNOD capability).

              RENAME_WHITEOUT can't be employed together with RENAME_EXCHANGE.

              RENAME_WHITEOUT requires support from the underlying filesystem.
              Among the filesystems that support it are tmpfs (since Linux
              3.18), ext4 (since Linux 3.18), XFS (since Linux 4.1), f2fs
              (since Linux 4.2), btrfs (since Linux 4.7), and ubifs (since
              Linux 4.9).

RETURN VALUE
       On success, zero is returned.  On error, -1 is returned, and errno is
       set to indicate the error.

ERRORS
       EACCES Write permission is denied for the directory containing oldpath
              or newpath, or, search permission is denied for one of the
              directories in the path prefix of oldpath or newpath, or oldpath
              is a directory and does not allow write permission (needed to
              update the ..  entry).  (See also path_resolution(7).)

       EBUSY  The rename fails because oldpath or newpath is a directory that
              is in use by some process (perhaps as current working directory,
              or as root directory, or because it was open for reading) or is
              in use by the system (for example as a mount point), while the
              system considers this an error.  (Note that there is no
              requirement to return EBUSY in such cases-there is nothing wrong
              with doing the rename anyway-but it is allowed to return EBUSY
              if the system cannot otherwise handle such situations.)
```

```
       EDQUOT The user's quota of disk blocks on the filesystem has been
              exhausted.

       EFAULT oldpath or newpath points outside your accessible address space.

       EINVAL The new pathname contained a path prefix of the old, or, more
              generally, an attempt was made to make a directory a
              subdirectory of itself.

       EISDIR newpath is an existing directory, but oldpath is not a
              directory.

       ELOOP  Too many symbolic links were encountered in resolving oldpath or
              newpath.

       EMLINK oldpath already has the maximum number of links to it, or it was
              a directory and the directory containing newpath has the maximum
              number of links.

       ENAMETOOLONG
              oldpath or newpath was too long.

       ENOENT The link named by oldpath does not exist; or, a directory
              component in newpath does not exist; or, oldpath or newpath is
              an empty string.

       ENOMEM Insufficient kernel memory was available.

       ENOSPC The device containing the file has no room for the new directory
              entry.

       ENOTDIR
              A component used as a directory in oldpath or newpath is not, in
              fact, a directory.  Or, oldpath is a directory, and newpath
              exists but is not a directory.

       ENOTEMPTY or EEXIST
              newpath is a nonempty directory, that is, contains entries other
              than "." and "..".

       EPERM or EACCES
              The directory containing oldpath has the sticky bit (S_ISVTX)
              set and the process's effective user ID is neither the user ID
              of the file to be deleted nor that of the directory containing
              it, and the process is not privileged (Linux: does not have the
              CAP_FOWNER capability); or newpath is an existing file and the
              directory containing it has the sticky bit set and the process's
              effective user ID is neither the user ID of the file to be
              replaced nor that of the directory containing it, and the
              process is not privileged (Linux: does not have the CAP_FOWNER
              capability); or the filesystem containing oldpath does not
              support renaming of the type requested.

       EROFS  The file is on a read-only filesystem.
```

```
       EXDEV  oldpath and newpath are not on the same mounted filesystem.
              (Linux permits a filesystem to be mounted at multiple points,
              but rename() does not work across different mount points, even
              if the same filesystem is mounted on both.)

       The following additional errors can occur for renameat() and
       renameat2():

       EBADF  oldpath (newpath) is relative but olddirfd (newdirfd) is not a
              valid file descriptor.

       ENOTDIR
              oldpath is relative and olddirfd is a file descriptor referring
              to a file other than a directory; or similar for newpath and
              newdirfd

       The following additional errors can occur for renameat2():

       EEXIST flags contains RENAME_NOREPLACE and newpath already exists.

       EINVAL An invalid flag was specified in flags.

       EINVAL Both RENAME_NOREPLACE and RENAME_EXCHANGE were specified in
              flags.

       EINVAL Both RENAME_WHITEOUT and RENAME_EXCHANGE were specified in
              flags.

       EINVAL The filesystem does not support one of the flags in flags.

       ENOENT flags contains RENAME_EXCHANGE and newpath does not exist.

       EPERM  RENAME_WHITEOUT was specified in flags, but the caller does not
              have the CAP_MKNOD capability.

STANDARDS
       rename()
              C11, POSIX.1-2008.

       renameat()
              POSIX.1-2008.

       renameat2()
              Linux.

HISTORY
       rename()
              4.3BSD, C89, POSIX.1-2001.

       renameat()
              Linux 2.6.16, glibc 2.4.

       renameat2()
              Linux 3.15, glibc 2.28.
```

```
   glibc notes
       On older kernels where renameat() is unavailable, the glibc wrapper
       function falls back to the use of rename().  When oldpath and newpath
       are relative pathnames, glibc constructs pathnames based on the
       symbolic links in /proc/self/fd that correspond to the olddirfd and
       newdirfd arguments.

BUGS
       On NFS filesystems, you can not assume that if the operation failed,
       the file was not renamed.  If the server does the rename operation and
       then crashes, the retransmitted RPC which will be processed when the
       server is up again causes a failure.  The application is expected to
       deal with this.  See link(2) for a similar problem.

SEE ALSO
       mv(1), rename(1), chmod(2), link(2), symlink(2), unlink(2),
       path_resolution(7), symlink(7)

Linux man-pages 6.7              2023-10-31                          rename(2)
```

---

nanosleep(2)

```
nanosleep(2)                  System Calls Manual                 nanosleep(2)

NAME
       nanosleep - high-resolution sleep

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <time.h>

       int nanosleep(const struct timespec *duration,
                     struct timespec *_Nullable rem);

   Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

       nanosleep():
           _POSIX_C_SOURCE >= 199309L

DESCRIPTION
       nanosleep() suspends the execution of the calling thread until either
       at least the time specified in *duration has elapsed, or the delivery
       of a signal that triggers the invocation of a handler in the calling
       thread or that terminates the process.

       If the call is interrupted by a signal handler, nanosleep() returns -1,
       sets errno to EINTR, and writes the remaining time into the structure
       pointed to by rem unless rem is NULL.  The value of *rem can then be
       used to call nanosleep() again and complete the specified pause (but
       see NOTES).
```

```
       The timespec(3) structure is used to specify intervals of time with
       nanosecond precision.

       The value of the nanoseconds field must be in the range [0, 999999999].

       Compared to sleep(3) and usleep(3), nanosleep() has the following
       advantages: it provides a higher resolution for specifying the sleep
       interval; POSIX.1 explicitly specifies that it does not interact with
       signals; and it makes the task of resuming a sleep that has been
       interrupted by a signal handler easier.

RETURN VALUE
       On successfully sleeping for the requested duration, nanosleep()
       returns 0.  If the call is interrupted by a signal handler or
       encounters an error, then it returns -1, with errno set to indicate the
       error.

ERRORS
       EFAULT Problem with copying information from user space.

       EINTR  The pause has been interrupted by a signal that was delivered to
              the thread (see signal(7)).  The remaining sleep time has been
              written into *rem so that the thread can easily call nanosleep()
              again and continue with the pause.

       EINVAL The value in the tv_nsec field was not in the range [0,
              999999999] or tv_sec was negative.

VERSIONS
       POSIX.1 specifies that nanosleep() should measure time against the
       CLOCK_REALTIME clock.  However, Linux measures the time using the
       CLOCK_MONOTONIC clock.  This probably does not matter, since the
       POSIX.1 specification for clock_settime(2) says that discontinuous
       changes in CLOCK_REALTIME should not affect nanosleep():

              Setting the value of the CLOCK_REALTIME clock via
              clock_settime(2) shall have no effect on threads that are
              blocked waiting for a relative time service based upon this
              clock, including the nanosleep() function; ...  Consequently,
              these time services shall expire when the requested duration
              elapses, independently of the new or old value of the clock.

STANDARDS
       POSIX.1-2008.

HISTORY
       POSIX.1-2001.

       In order to support applications requiring much more precise pauses
       (e.g., in order to control some time-critical hardware), nanosleep()
       would handle pauses of up to 2 milliseconds by busy waiting with
       microsecond precision when called from a thread scheduled under a real-
       time policy like SCHED_FIFO or SCHED_RR.  This special extension was
       removed in Linux 2.5.39, and is thus not available in Linux 2.6.0 and
```

```
       later kernels.

NOTES
       If the duration is not an exact multiple of the granularity underlying
       clock (see time(7)), then the interval will be rounded up to the next
       multiple.  Furthermore, after the sleep completes, there may still be a
       delay before the CPU becomes free to once again execute the calling
       thread.

       The fact that nanosleep() sleeps for a relative interval can be
       problematic if the call is repeatedly restarted after being interrupted
       by signals, since the time between the interruptions and restarts of
       the call will lead to drift in the time when the sleep finally
       completes.  This problem can be avoided by using clock_nanosleep(2)
       with an absolute time value.

BUGS
       If a program that catches signals and uses nanosleep() receives signals
       at a very high rate, then scheduling delays and rounding errors in the
       kernel's calculation of the sleep interval and the returned remain
       value mean that the remain value may steadily increase on successive
       restarts of the nanosleep() call.  To avoid such problems, use
       clock_nanosleep(2) with the TIMER_ABSTIME flag to sleep to an absolute
       deadline.

       In Linux 2.4, if nanosleep() is stopped by a signal (e.g., SIGTSTP),
       then the call fails with the error EINTR after the thread is resumed by
       a SIGCONT signal.  If the system call is subsequently restarted, then
       the time that the thread spent in the stopped state is not counted
       against the sleep interval.  This problem is fixed in Linux 2.6.0 and
       later kernels.

SEE ALSO
       clock_nanosleep(2), restart_syscall(2), sched_setscheduler(2),
       timer_create(2), sleep(3), timespec(3), usleep(3), time(7)

Linux man-pages 6.7              2024-03-03                      nanosleep(2)
```